A Survey of Small Language Models for Agentic AI: Architectures, Compression, Deployment, and Evaluation

Mohammed Gamal Ragab [1,2*], Hitham Alhussian [1,2*], Said Jadid Abdulkadir [1,2], and W. Eltayeb Ahmed [4]

[1]Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[2]Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar 32610, Perak, Malaysia
[4]Department of Mathematics and Statistics, College of Science, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia
[*]Corresponding authors: mohammed.ragab@utp.edu.my (M.G.R.); seddig.alhussian@utp.edu.my (H.A.)

Abstract

Large language models (LLMs) fit poorly, on cost, memory, and latency, with a growing class of workloads: AI agents that issue many narrow, repetitive, well-specified model calls. Small language models (SLMs) have narrowed the capability gap through distillation, curated and synthetic data, and quantization. This survey defines an SLM as an autoregressive decoder-only model of at most about twenty billion parameters both active per token, which bounds compute, and in total, which bounds storage; models meeting only the first bound are treated separately as small-active-parameter mixtures of experts. We catalog 179 SLMs and 30 such mixtures released from 2023 to September 2026, check every record's developer source, and release the inventory, the checks, and the scripts that apply the inclusion rule. We review compression along separate axes of parameters, memory, training compute, and latency. We quantify why the vendor evaluation record cannot support cross-model comparison: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five. We then state the agentic case as a conditional, falsifiable hypothesis: for call types that are bounded, recurrent enough to clear a break-even volume counting retries, checks, and escalations, short-context, and checkable for correctness, a specialized SLM should be the default, with escalation to a frontier model when a check fails. Grading the agent-specific evidence, we find per-call competence reported for specialized sub-10B models but qualified by independent studies, contested evidence on end-to-end success, and no measurement of cost per completed task for an SLM-first agent. We state the workloads for which the hypothesis fails and specify a pre-registered comparison, with its decision rule and sample size, that would refute it for a given workload.

Highlights

- Fixes a two-condition definition of a small language model that separates per-token cost (active parameters) from residency cost (total parameters), and distinguishes small-active-parameter mixtures of experts from SLMs proper.
- Catalogs 209 documented variants (178 SLMs and 31 small-active mixtures of experts, 2023–2026), selected by a mechanically applied inclusion rule and released in machine-readable form with a per-record inclusion decision.
- Audits the vendor-reported evaluation record for 18 SLMs and shows that it does not support the cross-model accuracy comparisons routinely built from it, pointing to single-harness third-party evaluations instead.
- Reviews compression for the sub-10B regime, separating parameter, memory, training-compute, and latency effects, with quality reported alongside, rather than collapsing them into one ratio.
- States the agentic case as a conditional, falsifiable hypothesis whose conditions can be checked in advance, grades the agent-specific evidence including contrary results, and sets out the workload classes for which it fails.
- Specifies six experiments that would supply the controlled evidence the field currently lacks, including the cost-per-completed-task comparison the economic argument turns on.

Keywords: Small language models • Agentic AI • Model compression • Edge AI • On-device inference • LLM–SLM collaboration

1 Introduction

Large language models (LLMs) such as GPT-4 [1], Claude [2], Gemini [3], and Llama [4] show broad competence across reasoning, code, and multilingual tasks, at a cost that scaling laws [5, 6] made predictable but not small: multi-thousand-GPU training runs, tens of gigabytes of accelerator memory per served replica, and a network round trip, a per-token charge, and a transfer of prompt content for every call. Training GPT-3 is estimated to have

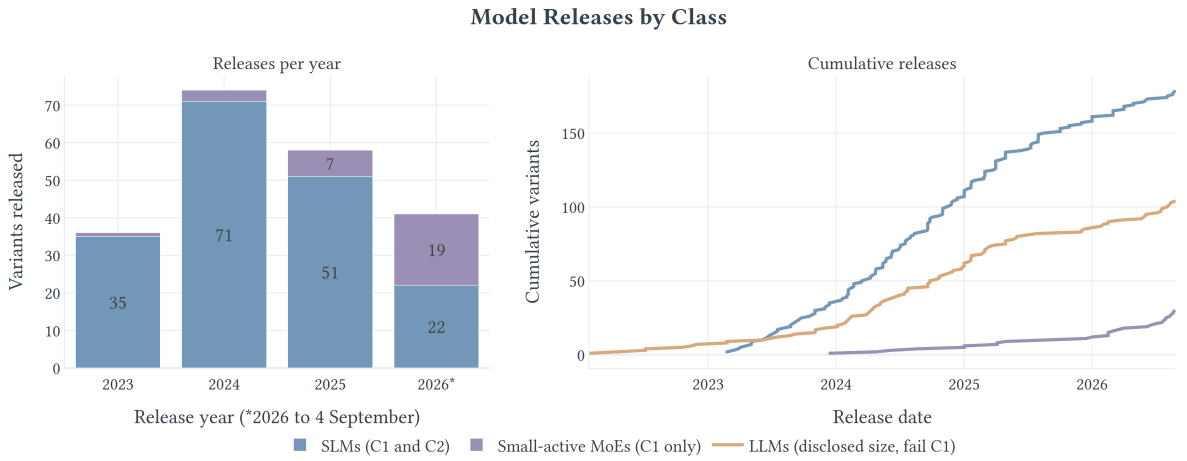


Figure 1. Documented variants by class: releases per year (left; 2026 to 4 September) and cumulative releases (right), from version 3.6 of the released inventory. SLMs meet both C1 and C2; small-active-parameter MoEs meet C1 only and are plotted separately; the comparison curve counts the 104 records of disclosed size that fail C1. The 119 records of undisclosed size cannot be classified and are not plotted.

consumed 1,287 MWh [7]. Per-query serving energy is contested: a 2023 estimate put a frontier query at about ten times a web search [8], while a 2025 production measurement reports 0.24 Wh for the median text prompt of one large assistant [9].

1.1 The Rise of Small Language Models

Against this trajectory, small language models (SLMs) have narrowed the capability gap at fixed size. The Chinchilla analysis [6] showed that most large models were undertrained; Phi showed that curated and synthetic "textbook-quality" data lets a 3.8B model rival much larger ones [10]; Gemma distilled from larger teachers [11]; and Qwen2.5 trained sub-10B models on 18 trillion tokens [12]. The 2025–2026 releases add hybrid state-space architectures [13], controllable reasoning [14], and mobile-first multimodal families [15, 16]. Figure 1 shows the record of the released inventory (Appendix R): 179 SLMs and, counted separately, 30 small-active-parameter mixtures of experts (both defined below) released between 2023 and 4 September 2026. The release rate of SLMs proper peaked in 2024 (71 variants, against 51 in 2025 and 22 in 2026 to date); what grew in 2026 is the small-active-parameter class (19 of the 41 releases of that year), which Section 1.2 keeps apart because it does not share the deployment properties of an SLM. The seed sources of the inventory changed in 2026 (Section 1.5), so year-to-year differences partly reflect coverage.

1.2 Definitions and Terminology

"Small" is used in the literature to mean at least four different things, so we fix the terms before using them.

1.2.1 The Operative Definition

A model is a *small language model* in this survey when it satisfies both of the following:

C1 (compute envelope)
: It is an autoregressive decoder-only model (attention, state-space, or a hybrid) whose *active* parameter count per generated token is at most roughly twenty billion.

C2 (storage envelope)
: Its *total* parameter count is also at most roughly twenty billion, so that its weights, about 11–15 GB at the 4.4–5.8 effective bits per weight of common 4-bit formats (Appendix I.5), can be stored and downloaded like a consumer application and, on runtimes that keep every weight in memory, held on one 16–24 GB consumer GPU or laptop.

C1 bounds per-token compute, latency, and energy (Figure 2). C2 bounds storage and download size, and resident memory only on runtimes that keep all weights resident: runtimes that stream weights or experts from flash [17], as Apple's 20B-total, 1–4B-active on-device model does [18], decouple residency from total size, so for such models resident memory has to be reported separately rather than read off C2. The two conditions coincide for dense models, which is why a single threshold has sufficed in most prior work, but they come apart for mixture of experts

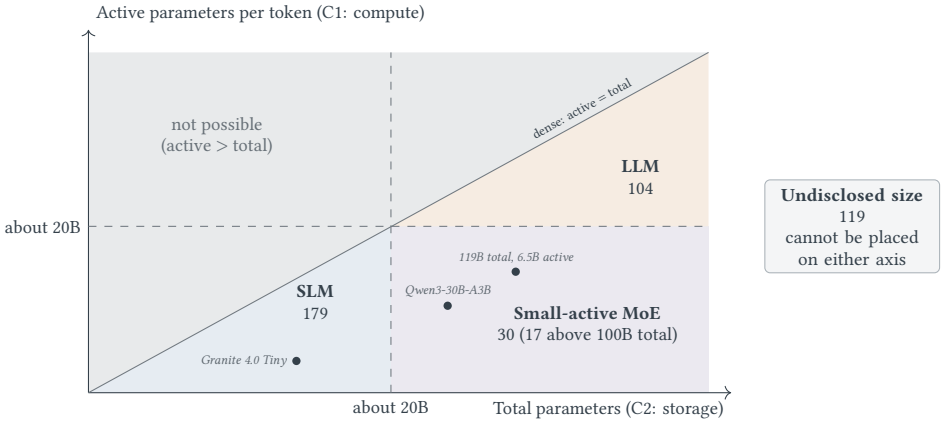


Figure 2. The two bounds of the definition. C1 bounds active parameters per token, which set per-token compute, latency, and energy; C2 bounds total parameters, which set storage and download size (and resident memory when every weight is kept in memory). Dense models lie on the diagonal. Counts are inventory records (version 3.6); example points are illustrative placements on schematic logarithmic axes, not a plot of the inventory.

(MoE) models: a 119B-total model that activates 6.5B parameters per token has the per-token cost of a 6.5B model and the storage cost of a 119B one. The twenty-billion figure is a convention, not a natural kind. Applied to the records of disclosed size in the released file, a bound of 10B, 13B, 20B, or 32B total parameters would admit 143, 160, 179, or 209 of them respectively, and every record carries its counts so that any other threshold can be applied.

1.2.2 Four Distinguished Terms

SLM A model satisfying C1 and C2. Every deployment, storage, and on-device claim in this paper applies to this class.

Compact model
An SLM at or below roughly 3B total parameters, whose 4-bit weights fit in about 2 GB of the 4–8 GB a current phone leaves to applications. Most documented phone deployments use this band; the exceptions keep part of the model off the accelerator, through per-layer embeddings (Gemma 3n and Gemma 4 E-series) or flash-resident experts (Apple's third-generation on-device model).

Edge model
A model, of any size, for which the developer documents and ships an artifact for phone-, single-board-computer-, or neural processing unit (NPU)-class hardware. "Edge" is a claim about the deployment target, not about parameter count.

Small-active-parameter MoE
A model satisfying C1 but not C2. These have SLM-like per-token cost and LLM-like storage cost. They are recorded in the inventory but counted, plotted, and discussed separately, and are excluded from every claim about storage, download size, resident memory, or on-device feasibility. The class is wide: 17 of its 30 members exceed 100B total parameters, so it includes frontier-scale models such as Llama 4 Maverick and Qwen3.5-397B-A17B as well as 21–35B models.

The inventory holds 179 SLMs and 30 small-active-parameter MoEs; wherever this paper counts SLMs, it means the 179. Per-layer-embedding "effective" sizes (Gemma 3n, Gemma 4 E2B/E4B) are not sparse activation, so C1 is evaluated on those models' total counts. The unit counted is the *variant*: a separately released artifact of one size within a family, which may be a distinct post-training or domain specialization released as its own repository (for example Phi-4 and Phi-4-reasoning), with checkpoint revisions of the same artifact counted once. Counted instead by distinct family and size, the 179 SLMs are 136 family–size pairs. Appendix A lists the reporting dimensions the inventory records and their coverage (total parameters 209/209, context length 135/209, peak runtime memory not reported by vendors), and the artifact vocabulary.

1.3 The Agentic Turn and the Thesis

A large share of model calls is now issued by AI agents: control loops in which a model is called repeatedly to choose a tool, fill its arguments, extract a field, classify an intent, or check a result [19, 20]. Many such calls are narrow, repetitive, and format-constrained. Belcak, Heinrich, Diao, et al. [19] argue that for these calls SLMs should be the default, and Sharma and Mehta [21] survey the evidence and propose an SLM-default, LLM-fallback design. We state the position as a conditional hypothesis whose terms can be measured.

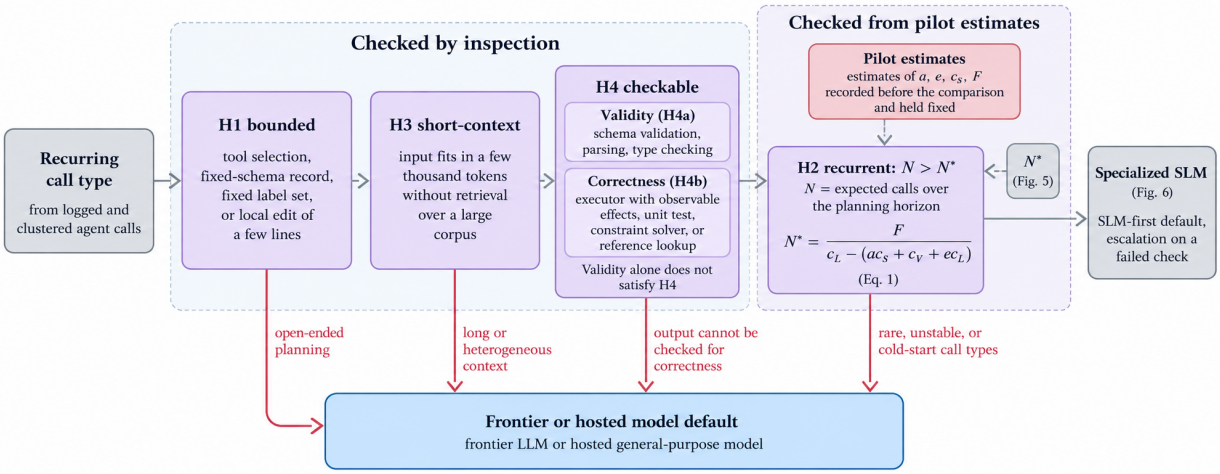


Figure 3. Assessing one recurring agent call type against hypothesis H at design time. H1, H3, and H4 are checked by inspecting the call type; H2 is checked against pilot estimates recorded before any comparison. A call type that passes every condition defaults to a specialized SLM with escalation on a failed check (Figure 6); a failure at any condition, labelled with the workload class that typically fails there (Section 7.4), leaves a frontier or hosted model as the default.

H (default hypothesis). Consider a type of language-model call inside an agent with the following properties.

H1 (bounded)
The output is a tool selection, a record conforming to a fixed schema, a label from a fixed set, or a local edit of a few lines.

H2 (recurrent)
The expected number of such calls over the planning horizon, N, exceeds the break-even volume

$$N^* = \frac{F}{c_L - (a\,c_S + c_V + e\,c_L)}, \tag{1}$$

where F is the one-time and maintenance cost of specializing for the call type, c_L and c_S are the per-call serving costs of the large and the small model, $a \geq 1$ is the expected number of small-model attempts per call, c_V is the per-call cost of the check, and e is the escalation rate. If the denominator is not positive, specialization never pays.

H3 (short-context)
The input needed fits within a few thousand tokens without retrieval over a large corpus.

H4 (checkable)
Two checks exist at a cost small relative to c_L: a *validity* check (H4a: schema validation, parsing, type checking), and a *correctness* check (H4b: an executor with observable effects, a unit test, a constraint solver, or a lookup against a reference) that detects outputs which are well formed but wrong. Validity checking alone does not satisfy H4.

For such a call type, a specialized SLM is the appropriate default: the call goes to the SLM first and is escalated to a frontier LLM when a check fails. "Appropriate default" means that the SLM-first system is non-inferior to an all-LLM system in task success, within a margin δ fixed in advance, at a lower cost per completed task than every comparator of Section 9.3.

H1, H3, and H4 can be checked by inspecting a call type before any model is trained (Figure 3). H2 cannot be checked from first principles, because a, e, and c_S depend on the specialized model: it is assessed from pilot estimates recorded before the comparison and held fixed, so that a failed comparison cannot be reclassified afterwards as a failure of H2. H is refuted for a workload class meeting H1–H4 if a pre-registered comparison of the kind specified in Section 9.3 finds the SLM-first system inferior by more than δ, or more costly per completed task than any comparator, and it is rejected as a general design rule if it fails on a majority of at least three pre-registered workloads with distinct call types. It does not claim that SLMs match frontier models in general, that agents can dispense with large models, or any particular share of agent traffic meeting H1–H4. Section 7.4 sets out the workload classes for which the conditions fail. Section 7.3 grades the evidence bearing on H: it is strongest for per-call competence

of specialized sub-10B models, contested for end-to-end success, and absent for cost per completed task in an SLM-first agent.

1.4 Scope and Contributions

Existing SLM surveys [22–25] provide taxonomies, technique reviews, and measurements but treat agentic deployment only in passing. The agentic case has been made as a position [19, 26] and as a survey [21], which already proposes SLM-default, LLM-fallback systems with verifier cascades, cost per successful task as the deciding metric, and limits where fallback remains necessary. We share that direction. This paper adds the following:

A two-bound definition
that separates per-token cost (C1) from storage cost (C2) and so treats mixtures of experts consistently (Section 1.2).

A verified inventory
of 179 SLMs and 30 small-active-parameter MoEs (2023–2026), in which every record's developer source has been checked and every inclusion decision is recorded with its reason, released with the scripts that apply the rule and generate every table and figure (Appendix B).

An audit of the vendor evaluation record
that quantifies why a cross-model table cannot be built from it: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them, and no stated protocol is shared by more than five (Section 5).

A compression review
that reports reductions along separate axes of parameters, memory, training compute, and latency (Section 4).

An operational version of the agentic hypothesis
with a break-even condition that includes retries, checking, and escalation, a correctness requirement on the check, graded agent-specific evidence including contrary results, and the workload classes for which it fails (Section 7).

A falsification design
that specifies comparators, the statistical decision rule, and the sample size needed to decide H for a workload, with five further verification studies (Section 9).

1.5 Survey Method and Sources

This is a structured narrative survey, not a systematic review; its aims are representative coverage of the design space, a verified inventory, and an audit of a defined cohort. It addresses five research questions: the design space of SLMs released 2023–2026 (RQ1); which compression techniques pay in the sub-10B regime, along which axis, and at what quality cost (RQ2); what the vendor evaluation record can and cannot support (RQ3); what agent-specific evidence says about small models (RQ4); and under which conditions a small model is the appropriate default inside an agent (RQ5).

Model inventory (RQ1). A seed list compiled in March 2026 from public model listings and aggregators, 205 of whose records were compiled with the assistance of a large language model, was updated on 5 September 2026 against the OpenRouter catalogue and vendor announcements, and every record was then classified against C1 and C2. On 8 October 2026 every record's developer source was checked through the Hugging Face Hub API: 197 of the 209 included records link an existing repository owned by the developer, and the other 12 link the developer's own announcement, paper, or code repository. Twelve records first sourced from an aggregator or a press article were re-sourced to the developer, and one for which no developer source exists was excluded. C1, C2, and inclusion criteria M1 (a developer source states size and architecture) and M4 (a dated release) are computed from recorded fields by a released script; the exclusions of derivative fine-tunes, API aliases, and encoder-decoder models are named editorial decisions listed in that script. The released file records, per record, the check, its date, and the Hub parameter count where available. Coverage is strongest for developers that publish on the Hub. Appendix B gives the criteria, the seven excluded records, and the count history.

Technique literature (RQ2, RQ4, RQ5). Works were identified from the prior surveys cited above, by backward and forward citation tracing, and by targeted searches of arXiv and the ACL Anthology, last run on 8 October 2026 for agent-specific evaluation, routing, function calling, and cost-per-outcome measurement. Screening decisions were not recorded for this stream, so statements that we "found no" study are bounded by that search.

Evaluation record (RQ3). For each model of the 18-model cohort, the primary release document and official model card were consulted; the coverage ledger of Section 5.2 was transcribed from them on 8 October 2026. No benchmark was executed for this survey.

Appraisal. Each empirical claim carries an evidence grade (Section 5.1.2) reflecting its design and the independence of its source.

1.6 Organization

Section 2 reviews related work. Section 3 presents the taxonomy, Section 4 compression, Section 5 the evaluation record and the grading scheme, and Section 6 deployment. Section 7 assesses H on agent-specific evidence and states its counterexamples. Section 8 discusses implications, applications, and limitations, Section 9 sets out open problems and the verification agenda, and Section 10 concludes. The electronic supplement (Appendix A to Appendix R) holds the reporting dimensions, inclusion criteria and released artifacts, detailed technique reviews, the audit ledger, reference tables, domain applications, and the generated variant inventory.

2 Related Work and Research Gaps

2.1 From Scaling to Small Models

The Transformer [27] and the pre-train-then-adapt paradigm [28, 29] led to a scaling race that the Chinchilla study [6] partly redirected: a 70B model trained on 1.4 trillion tokens outperformed the 280B Gopher, implying that most large models were undertrained and that smaller, longer-trained models could match them. Subsequent families pushed this further at small scale through curated and synthetic data [10, 30], distillation from larger teachers [11, 31], and pruning plus distillation from a larger sibling, as for Llama 3.2's 1B and 3B edge models [32]. Three bodies of technique underlie the current generation: distillation and synthetic-data training [30, 33]; post-training compression and parameter-efficient adaptation [34–37]; and on-device runtimes [38–40]. Sections 4 and 6 review them, and Appendix C and D summarize the principal families and the longer background.

2.2 LLM–SLM Collaboration and Agentic Systems

A line of work studies how small and large models cooperate [41, 42]. Cascades and routers are the most relevant to agents: FrugalGPT [43] learns a cascade over commercial APIs, Hybrid LLM [44] routes by predicted difficulty, AutoMix [45] escalates on self-verification, RouteLLM [46] learns from preference data, RouterBench [47] benchmarks routers on 405K inference outcomes, and Division-of-Thoughts [48] routes sub-questions between an on-device and a cloud model. Speculative decoding [49, 50] uses a small model to accelerate a large one without delegating answers. On the agentic side, function-calling data pipelines report specialized sub-10B models among the leading entries of the Berkeley Function-Calling Leaderboard [51–53], and agent evaluation has begun to report cost alongside accuracy [54–56].

2.3 Gaps in Existing Literature

General SLM surveys [22–24] provide technique taxonomies, application coverage, and in the case of Lu, Li, Cai, et al. [22] a measurement study of 70 models; Gupta, Thomas, Asnani, et al. [25], updated in 2026, survey about 160 papers on 1–8B models. None separates active from total parameters or foregrounds agentic deployment. Srivastava, Cao, and Wang [57] benchmark 72 small models on 17 reasoning benchmarks with three repeats, including models derived by quantization, pruning, and distillation, and find that reasoning tracks training method and data more than scale and that quantization preserves reasoning while pruning disrupts it; it is the closest thing in the literature to the controlled evaluation whose absence Section 5 documents, but covers reasoning only. Feng, Fang, Ma, and Wang [58] survey efficient reasoning, and Chen, Liu, Han, Zhang, and Liu [20] survey multi-agent systems by application.

Two works state the agentic case directly. Belcak, Heinrich, Diao, et al. [19] argue it as a position, with a conversion algorithm and estimates of how much agent traffic small models could handle, and Tian, Cai, Zhao, and Lane [26] argue that personal agents must move to the edge. The closest prior work is the survey of Sharma and Mehta [21]. It reviews 1–12B (occasionally about 20B) models for agentic workloads, proposes an SLM-default, LLM-fallback architecture with uncertainty-aware routing, retries, and verifier repair (their Section VIII), proposes cost per successful task as the deciding metric, and names where large models still win: open-domain synthesis, complex multi-hop reasoning, safety-critical judgment, and long-context work (their Section XI). We share that design and metric. It does not separate active from total parameters, grade its evidence or report contrary results, examine whether the vendor evaluation record supports comparison, or state a decision rule under which its position would be refuted, and its estimate that cost per successful task falls 10–30× (their Section X) is not a reported measurement. Those are the points on which this paper differs: the C1/C2 definition, the verified inventory, the quantified audit of the evaluation record, graded evidence including contrary results, a break-even condition that includes retries, checking, and escalation together with a correctness requirement on the check, and a falsification design with comparators, a decision rule, and a sample size. Table 1 summarizes the positioning.

Table 1. Coverage of this survey relative to prior surveys and position papers (Y full, P partial, – absent), as judged by the present authors from each work's text (grade D). *Active/total*: separates active from total parameters. *Evaluation critique*: analysis of whether the reporting record supports cross-model comparison. *Graded evidence*: an explicit strength-of-evidence grade per claim. *Agentic focus*: small models inside agents as a primary topic. *Decision rule*: a stated condition under which the position is refuted.

Work	Taxonomy	Compression	Active/ total	Evaluation critique	Graded evidence	Agentic focus	Decision rule
Lu, Li, Cai, et al. [22]	Y	P	–	P	–	–	–
Nguyen, Shen, Aponte, et al. [23]	Y	Y	–	–	–	–	–
Wang, Zhang, Zhang, et al. [24]	Y	Y	–	–	–	P	–
Gupta, Thomas, Asnani, et al. [25]	Y	P	–	–	–	–	–
Srivastava, Cao, and Wang [57]	P	P	–	Y	–	–	–
Belcak, Heinrich, Diao, et al. [19]	–	–	–	P	–	Y	–
Tian, Cai, Zhao, and Lane [26]	–	–	–	–	–	Y	P
Sharma and Mehta [21]	P	P	–	–	–	Y	–
This survey	Y	Y	Y	Y	Y	Y	Y

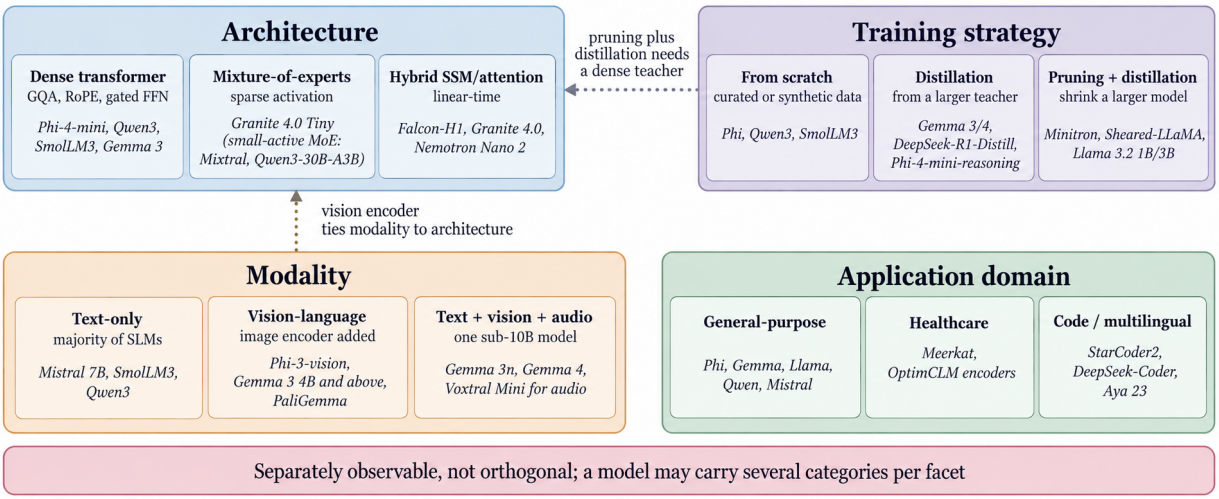


Figure 4. The four primary facets of the taxonomy. Facets are separately observable but not orthogonal, and assignment is multi-label. Representative models are italicised; Appendix F develops each facet and Table 12 gives the tabular form.

3 A Faceted Taxonomy of Small Language Models

3.1 The Scheme

We index the SLM design space along four *primary* facets, architecture, training strategy, modality, and application domain (Figure 4), and five *secondary* facets that govern adoption: licensing, data transparency, safety alignment, reasoning mode, and deployment target (Table 14 in Appendix J). The facets are *separately observable*, in that each can be read off a model's primary source, but they are *not orthogonal*: pruning-plus-distillation presupposes a dense teacher, and a vision encoder ties modality to architecture. The scheme is also *multi-label*: Gemma 3 is distilled *and* trained on a curated corpus, Granite 4.0 is a hybrid state-space model *and* a mixture of experts. It is an indexing scheme illustrated with exemplars, not a partition of the inventory. The released inventory records per variant the fields its sources state directly (modality, access, license, dense or mixture-of-experts, post-training status, context length), and the figures are drawn from those fields; architecture and training facets are assigned in this paper to representative models only. The application-domain facet is the weakest of the four, since it usually describes use rather than the artifact, so we assign it only when the developer trained the released artifact for a domain and says so.

3.2 Architecture

Dense transformers remain the majority, from SmolLM2-135M [59] to 7–14B models. The recurring refinements are grouped-query attention [60], which shrinks the key–value (KV) cache by the ratio of query to KV heads; rotary position embeddings with an enlarged base or rescaling for long context; and gated feed-forward layers (SwiGLU in most families, GeGLU in Gemma). Among SLMs whose records state a context length, the median grew from 2K

tokens in 2023 ($n = 5$) to 16K in 2024 ($n = 39$) and 128K in 2025 ($n = 46$) and 2026 ($n = 20$); coverage is thin for 2023, so the starting point is uncertain (Appendix E).

Mixtures of experts [61, 62] decouple total from active parameters, and they sit on the fault line that C2 marks. Qwen3-30B-A3B [14] costs about what a 3B dense model costs per token but needs 17–22 GB of 4-bit weight storage, so it is a small-active-parameter MoE in this paper, not an SLM; Granite 4.0 Tiny (about 7B total, 1B active) [63] satisfies both conditions. The 2026 releases concentrated in this class, with Nemotron 3.5 Lightning (30B/3B), Laguna XS 2.1 (33B/3B), Ling 3.0 Flash (124B/5.1B), and DeepSeek V4 Flash (284B/13B) appearing between June and August 2026.

Hybrid state-space models combine attention with linear-time state-space layers [64]. Jamba [65] interleaves Mamba and attention blocks and reports a KV cache eight times smaller than Mixtral's at 256K context; Falcon-H1 [13], Granite 4.0 [63], Nemotron Nano 2 [66], and the Qwen3.5 small series [67] brought the design to production at small scale in 2025–2026. Appendix F gives the architectural detail for each class.

3.3 Training Strategy, Modality, and Domain

Three training routes recur and are often combined: training from scratch on curated or synthetic data (Phi, SmolLM, Qwen), distillation from a larger teacher (Gemma, the DeepSeek-R1 students, Phi-4-mini-reasoning), and pruning a larger model followed by distillation (Minitron, Sheared-LLaMA, Llama 3.2 1B and 3B). Text-only models still dominate the inventory, vision–language models are common at 2–12B, and audio and natively multimodal mobile models (Qwen2.5-Omni, Gemma 3n and 4, Voxtral Mini) grew in 2025–2026 (Appendix E). Most SLMs are released as general-purpose models; developer-trained domain models concentrate in code, healthcare, and multilingual use. Appendix F lists representative models for each category.

4 Model Compression Techniques for SLMs

Distillation, quantization, and pruning are the principal routes from a large or full-precision model to one deployable on edge hardware, and compound pipelines combine them. This section states how their results are reported here, summarizes what each technique delivers, and grades what the literature establishes about its quality cost. Appendix G and H review the individual methods.

4.1 How Compression Results Are Reported Here

Compression results are routinely summarized as one "compression ratio" against one "accuracy retention" percentage. That summary does not survive scrutiny. *There is no single compression ratio*: quantization reduces bits per weight and leaves the parameter count untouched; pruning reduces the parameter count, and memory only where the runtime exploits the sparsity pattern; distillation reduces the training compute needed to reach a given quality, while the student's size is the practitioner's choice, so a "10–100×" distillation ratio describes design choices, not a property of the method. *There is no baseline-free retention*: a figure such as "about 96%" is uninterpretable without the baseline precision, model, size, task, metric, and aggregation rule, each of which changes it materially [35, 57]. Table 2 therefore separates what each technique reduces along four axes that are not interchangeable and are not summed, and Table 3 states what the literature establishes about quality cost as a direction with its scope and grade. Where no study supports a number, none is given, and Section 9.3 specifies the measurement that would.

4.2 Distillation, Quantization, and Pruning

Knowledge distillation [33] trains a student to match a teacher's output distribution, its intermediate representations (white-box), or its generated text and reasoning traces (black-box), as in Orca [78] and the DeepSeek-R1 students [79]; synthetic-data training, the mechanism behind Phi [30], is distillation through the data. *Quantization* reduces numerical precision. Post-training methods need only a small calibration set: GPTQ [34] uses second-order information, AWQ [35] protects activation-salient channels, and LLM.int8() and SmoothQuant handle activation outliers at 8 bits [70, 71]. GPT-generated unified format (GGUF) [38] is the format in which most sub-4B models are distributed for CPU deployment. Quantization-aware training recovers part of what post-training quantization loses at 2–3 bits but needs the training stack, which is why it appears mainly in vendor pipelines [16], and its limit case, ternary weights trained from scratch [68], is a training recipe rather than a compression of an existing model. *Pruning* removes individual weights (SparseGPT [73], Wanda [80]) or whole structures (LLM-Pruner [81], ShortGPT [82]); structured pruning followed by continued training is a cheaper route to a small model than training from scratch [83].

The one controlled comparison in the sub-10B regime concerns the choice between the first two kinds of post-training compression. Zhou, Kurz, and Zhao [74] compare GPTQ and AWQ against SparseGPT and Wanda on six models of 0.5–3.8B across seven languages and seven tasks, and find that quantization preserves fidelity, multilingual perplexity, and reasoning accuracy better than pruning, with the advantage shrinking on knowledge-

Table 2. What each compression technique reduces, by axis. Entries follow from how the technique works, not from measurements. "Weight memory" is the realized footprint on a runtime that exploits the format; "training compute" is the cost of producing the compressed artifact relative to training an equivalent model from scratch; "latency" is decode-time speedup on memory-bandwidth-bound hardware. A dash means the technique does not act on that axis.

Technique	Parameters	Weight memory	Training compute	Decode latency	Retraining
PTQ, 8-bit	–	2× vs. FP16	–	In proportion to bandwidth saved	No
PTQ, 4-bit	–	2.8–3.6× (Table 8)	–	In proportion to bandwidth saved, batch 1	No
PTQ, 2-bit	–	Below the nominal 8× by the share kept at higher precision	–	As above	quantization-aware training (QAT) in practice
Ternary weights (1.58-bit)	–	About 10× nominal; trained in the format [68]	–	Needs kernels for the format	Trained from scratch
Unstructured pruning, 50%	2×	Only on sparse-capable runtimes	–	None on dense kernels	Calibration only
Structured pruning	Set by target	Proportional to parameters removed	–	Proportional to parameters removed	To recover quality
Knowledge distillation	Set by student	Proportional to student size	Large; the student is trained	Proportional to student size	Student training
Prune + distill	Set by target	As for structured pruning	Up to 40× fewer tokens per derived model [69]	As for structured pruning	Yes

and-reasoning tasks such as OpenBookQA and with Wanda's parity with SparseGPT at large scale not holding at this size; Srivastava, Cao, and Wang [57], across 72 small models, likewise find that quantization preserves reasoning while pruning disrupts it. This is the one grade-A claim in the paper. Its scope is post-training methods at their usual operating points, 4-bit weights against 50% sparsity, which are not equal reductions: 4-bit weights cut memory by about 3× on any runtime, while 50% unstructured sparsity saves memory only on runtimes that exploit it. The practical reading is that to compress an existing small model after training, quantize first; structured pruning followed by distillation, which derives a new model, is outside the comparison.

4.3 Combined Pipelines

The strongest reported results combine stages. Minitron [69] prunes Nemotron-4 15B in depth and width and retrains the pruned students by distillation from the original on a small fraction of its data; deriving the 8B and 4B models this way needs up to 40× fewer training tokens per model than training from scratch, a 1.8× compute saving for the family, and the derived models are reported to compare with similarly sized community models (C). A follow-up applies the recipe to Llama 3.1 8B and Mistral NeMo 12B [75], and Meta derived Llama 3.2 1B and 3B by pruning Llama 3.1 8B and distilling from the 8B and 70B models [32]. Quantization is applied afterwards for deployment and is not part of these recipes. In clinical NLP, OptimCLM [76] composes distillation, pruning, and 8-bit quantization of BERT encoders (C); it shows that the stages compose, not that they do for generative models.

5 The Public Evaluation Record for Small Language Models

This section examines what the public evaluation record can support about SLM capability. The values discussed are *reported* figures from vendor technical reports, model cards, community leaderboards, and third-party studies. We set out the cohort, sources, and grading scheme (Section 5.1), quantify why a cross-model table cannot be assembled from vendor reporting (Section 5.2), and explain why these benchmarks do not answer the agentic question in any case (Section 5.3).

5.1 Cohort, Sources, and Grading

5.1.1 Cohort and Sources

The cohort was chosen purposively: 18 models with public weights or an evaluation card, below 10B parameters (Phi-4 at 14B retained as an upper-bound reference), at least one score on an established benchmark, release no later than March 2025 so that at least eighteen months of third-party evaluation had accumulated by the snapshot, and a spread of training methods (Table 9 in Appendix J). It spans 0.5B to 14B and eight developers, and it contains none of the 2025–2026 hybrid, reasoning, or mobile-multimodal families, so the audit describes reporting practice for 2023–2025 releases. Values were sought from vendor technical reports, then official model cards, then single-harness

Table 3. What the reviewed literature establishes about the quality cost of each technique in the sub-10B regime, with its scope. Grades follow Section 5.1.2.

Technique	What the literature supports	Scope of the evidence	Grade
PTQ, 8-bit	Effectively lossless on the knowledge and language-modeling benchmarks reported	Method authors' evaluations [70, 71]	C
PTQ, 4-bit	Reasoning accuracy largely preserved in an independent benchmark of small models; quantization methods tuned on large models transfer imperfectly to sub-1B models, with sensitivity depending on architecture	72 models, 17 reasoning benchmarks, three repeats [57]; two models of 0.14B and 0.5B [72]	B
PTQ, 4-bit, by method	activation-aware weight quantization (AWQ) retains more accuracy than GPTQ at matched bit-width	Author-reported [35]	C
PTQ, 2-bit	post-training quantization (PTQ) degrades unacceptably; QAT recovers part of the loss	Vendor pipeline [16]	C
Unstructured pruning, 50–60%	One-shot pruning with negligible perplexity increase at 175B	OPT-175B, BLOOM-176B [73]	C
Quantization vs. post-training pruning	Quantization (GPTQ, AWQ) preserves fidelity and reasoning better than post-training pruning (SparseGPT, Wanda); the advantage shrinks on knowledge-and-reasoning tasks	6 models, 0.5–3.8B, 7 languages, 7 tasks [74]; corroborated across 72 models [57]	A
Prune + distill	Pruning Nemotron-4 15B to 8B and 4B and distilling on under 3% of the original data needs up to 40× fewer training tokens per derived model (1.8× less compute for the family)	One pipeline, one family [69, 75]	C
Domain compound pipeline	Distilling clinical BERT encoders with pruning and 8-bit quantization gives up to 22.88× compression with under 5% macro-AUROC loss	One study, encoder classifiers [76]	C
Fairness under compression	Degradation is uneven across languages; pruning is worse than quantization on multilingual tasks	Two sources [74, 77]	D

community evaluations such as HELM [84] and the Open LLM Leaderboard [85], then peer-reviewed third-party studies. A reported number is comparable with another only if five conditions match: benchmark version, prompt template, answer-extraction rule, decoding settings, and harness [86]; Appendix I defines each.

5.1.2 Evidence Grading

Because this survey mixes controlled studies, vendor self-reports, independent evaluations, and our own synthesis, each empirical claim carries a grade. The grade is set by two properties of the strongest source behind the claim: its *design* (a controlled comparison, in which the compared conditions differ only in the variable of interest under one harness with a matched baseline; a single-harness evaluation of many systems; a self-report; or an estimate or synthesis) and its *independence* (whether the reporting party has a stake in the result).

A **Controlled, independent**
: A controlled comparison reported by a party without a stake in the outcome.

B **Independent**
: Any other evaluation or documented count by a party without a stake: a single-harness evaluation across models, an independent benchmark, or a reproduction.

C **Reported**
: Stated by the developer of the artifact or the author of the technique, including developer-run comparisons on a single harness.

D **Interpretation**
: Our synthesis across sources, an inference from evidence gathered for another question, or an estimate in a position paper.

Table 4 grades the principal claims, and Tables 3 and 5 grade the technique and agent evidence. A grade-C claim is unverified rather than false; where vendor reports and independent reproductions of the same model exist, the vendor figure is usually the more favorable, so the likely direction of error is known.

5.2 Why a Common Benchmark Table Cannot Be Built from Vendor Reporting

We originally intended a single table placing the cohort side by side on a shared benchmark set, assembled from the vendors' documents. The vendor record cannot support it, and we report the measured reasons instead. The released coverage ledger (`audit_coverage_ledger.csv`; Appendix I.2) records, for the 14 cohort models from Microsoft,

Table 4. Evidence grade of the principal claims of this survey (A controlled and independent, B independent, C reported by the interested party, D interpretation or estimate). "None" marks a quantity for which no study was located by the search of Section 1.5.

Claim	Grade	Where
For the 14 cohort models of the four largest vendors, no benchmark is reported for all, and no stated protocol is shared by more than five (released ledger)	B	§5.2
Reasoning ability of small models tracks training method and data more than scale	B	§2.3
The strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not (across sources and time)	D	§5.2
INT8 post-training quantization is effectively lossless on the benchmarks its method authors report	C	§4
At 4 bits, reasoning is largely preserved; methods tuned on large models transfer imperfectly to sub-1B models	B	§4
At 0.5–3.8B, post-training quantization preserves fidelity better than post-training pruning	A	§4.2
4-bit weight footprint is 2.8–3.6× below FP16 (arithmetic from parameter count and effective bits)	D	§5.4
Specialized sub-10B function-calling models placed among the leading single-turn entries in 2024	C	§7.3.1
Specialized small models are reported ahead of frontier models on multi-turn function calling and τ-bench	C	§7.3.1
Without task-specific fine-tuning, small models often fail the required call format	B	§7.3.1
Serving a 7B model is 10–30× cheaper in latency, energy, and FLOPs than a 70–175B model	D	§7.2.2
40–70% of the LLM calls in three open-source agents could be handled by specialized SLMs	D	§7.1
End-to-end success on realistic agent benchmarks was low for all models when the benchmarks were introduced	B	§7.3.3
Open models of 70B or less trailed commercial APIs end to end (2023; size confounded with openness and training)	B	§7.3.3
Current specialized sub-10B agents trail frontier agents end to end	D	§7.3.3
Cost per correct answer favors lightweight models on basic quantitative tasks and large models on knowledge-intensive ones (single-turn)	B	§7.3.4
Cost per completed task, SLM-first agent against all-LLM agent on one workload	None	§7.3.4
Hypothesis H	D	§1.3

Alibaba, Meta, and Google, whether each model's primary release document reports an instruction-tuned value on each of the eight benchmarks that recur in their headline reporting (Massive Multitask Language Understanding (MMLU), MMLU-Pro, ARC-Challenge, HellaSwag, Grade School Math 8K (GSM8K), MATH, HumanEval, MBPP), and under which stated protocol.

Vendors do not report a common benchmark set. Of the 112 model–benchmark cells, 69 carry an instruction-tuned value, and no benchmark is reported for all 14 models; the best covered, GSM8K and MATH, are reported for 11.

Where they report the same benchmark, they rarely use the same protocol. Only 45 of the 69 reported cells state a protocol at all; the Qwen2.5 report, for example, gives no shot counts for its instruction-tuned tables. Among stated protocols, no benchmark has a protocol shared by more than five of the 14 models: MMLU is 5-shot for Phi and Llama 3.2 and 0-shot for Gemma 3; GSM8K is 8-shot with chain-of-thought for Phi and Llama 3.2 and 0-shot for Gemma 3; ARC-Challenge is 10-shot for Phi and 0-shot for Llama 3.2. Shot count alone moves multiple-choice accuracy by several points [86], so a column headed by one protocol would be false for most of its cells. This finding needs no rule beyond protocol matching.

Pre-trained and instruction-tuned checkpoints are reported separately. For 10 of the 14 models, the vendor tabulates the pre-trained and the instruction-tuned checkpoint in different tables under different protocols, and secondary sources often copy a figure without its checkpoint. For Gemma 3-4B, the technical report gives GSM8K 38.4 for the pre-trained checkpoint (8-shot) and 89.2 for the instruction-tuned one (0-shot with chain-of-thought), and MMLU-Pro 29.2 and 43.6 respectively [31]; a row mixing the two misstates a gap by tens of points.

Matched comparisons exist but are vendor-run. Some vendors publish single-harness comparison tables: Microsoft compares Phi-4-mini with ten other models, five of them in this cohort, on an internal platform that keeps prompts and few-shot examples identical across models [87], and Meta reports Llama 3.2 1B and 3B at BF16 beside their quantized versions on the same harness [88]. These meet the harness condition but are run by an interested party

that also chose the comparators and benchmarks, so they are grade C evidence and cannot be merged across vendors.

The general point, that scores are not comparable across evaluation setups, is the premise of HELM, the Open LLM Leaderboard, and the reproducibility analysis of Biderman, Schoelkopf, Sutawika, et al. [86]; the contribution here is to quantify it for the SLM cohort. Single-harness third-party evaluations are the right basis for comparison: the Open LLM Leaderboard v2 ran every submission through one configuration on six benchmarks [85], Lu, Li, Cai, et al. [22] evaluate 70 models of 0.1–5B under one setup including on-device latency, and Srivastava, Cao, and Wang [57] evaluate 72 models under one reasoning protocol. None covers the whole cohort, and their benchmark sets differ from the vendors', so they replace the vendor table rather than filling it in. What can be said from the vendor record without distortion is qualitative (D): the strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not, and no single SLM leads across all task families.

5.3 Why These Benchmarks Do Not Answer the Agentic Question

Every benchmark above measures a model answering a question in isolation. An agent asks a model to select one of several tools, to emit arguments that satisfy a schema, to stay well formed across a long tail of inputs, to recover when a tool returns an error, and to do so repeatedly. A model can score well on MMLU and still emit malformed calls often enough to be unusable in a loop, and a small model tuned for function calling can score poorly on MMLU and be adequate as an executor. Even an internally consistent version of this record would therefore be the wrong evidence for H, and Section 7.3 assembles agent-specific evidence instead.

5.4 Quantization Impact

Four points summarize the compression evidence for the cohort's size range (Table 3): INT8 post-training quantization is reported as effectively lossless (C); at 4 bits, reasoning is largely preserved in an independent benchmark of 72 small models, while methods tuned on large models transfer imperfectly to sub-1B models (B); AWQ is reported to retain more accuracy than GPTQ (C); and post-training quantization preserves fidelity better than post-training pruning at 0.5–3.8B (A). Per-model degradation figures are not reported consistently enough to tabulate. The memory consequence is arithmetic: 4-bit formats store 4.4–5.8 effective bits per weight, so weights shrink 2.8–3.6× rather than 4×; Qwen2.5-7B falls from 15.2 GB to 4.2–4.5 GB (Table 8 in Appendix I.5).

5.5 Multilingual and Safety Reporting

Multilingual and safety reporting is sparser than benchmark reporting and not comparable across models. Models whose reports document large non-English pretraining mixtures, such as Qwen2.5 [12], Gemma 3 [31], and Aya 23 8B [89], report stronger non-English performance than English-centric families (C). Per-model toxicity evaluations are often absent for research-grade small models, alignment can be eroded by fine-tuning [90], and compression degrades performance unevenly across languages [74, 77]. An agent that routes non-English or safety-sensitive calls to a small English-centric model inherits these gaps, which argues for capability-aware rather than size-only routing.

6 Deployment on Resource-Constrained Environments

6.1 On-Device Deployment and What It Guarantees

On-device inference removes the network round trip, the per-token charge, and the transmission of the prompt to an inference provider, and it works offline. It does not by itself keep user data on the device. Whether data leaves depends on whether the application emits telemetry or crash reports containing prompt content, logs conversations to a synchronized store, escalates calls to a cloud model (in which case, under difficulty-based routing, the escalated calls may be exactly the sensitive ones), or sends the same content to a hosted retrieval index or remote tool. On-device inference removes one egress channel, and eliminates egress only in an application whose whole pipeline is local and whose telemetry is absent or content-free. Apple's design [16] treats the escalation path as the hard part and builds Private Cloud Compute around it. Every privacy claim in this paper is subject to this scope condition.

6.1.1 Platforms and Device Tiers

Apple's Foundation Models are the most fully documented on-device deployment: the 2025 generation pairs a roughly 3B on-device model, quantized with 2-bit quantization-aware training and KV-cache sharing, with a server model on Private Cloud Compute [16], and the 2026 generation adds a 20B-total on-device model that activates 1–4B parameters and keeps its full weights in flash rather than DRAM [18]. Apple describes the 2026 family as built in collaboration with Google [18], following a joint statement that its next foundation models would be based on Gemini models and cloud technology [91]. Gemini Nano plays the on-device role on Android [3]. A current phone leaves roughly 4–8 GB to applications, so a 3B model at 4 bits (about 2 GB) fits, subject to thermal throttling and

battery cost. Single-board computers and Jetson-class modules run quantized 1–7B models, and microcontrollers are out of reach for any model in the inventory (Appendix K). Table 13 (Appendix J) compares the main runtimes: llama.cpp, ONNX Runtime, MLX, TensorRT-LLM, ExecuTorch, MLC-LLM, and WebLLM.

6.2 Calculated Quantities Versus Measured Ones

Weight memory at a precision, KV-cache size per token, and their ratio follow from parameter count, bit-width, and head configuration, so they are reliable within their assumptions (Table 8). Throughput, time-to-first-token, sustained rates under thermal load, energy per token, battery cost, and total cost of ownership are measured quantities that depend on batch size, context length, kernels, achieved bandwidth, runtime version, thermal envelope, and for energy the grid mix and the amortization of the device. Published figures for them are not comparable across sources, and we do not tabulate them; where one appears, it is attributed to its source and hardware.

One widely repeated claim needs particular care: that quantizing from 16 to 4 bits gives a near-linear 4× speedup. It holds only for single-stream decoding at batch size 1 on memory-bandwidth-bound hardware, where every generated token reads the full weight set. Under batching the bottleneck moves toward compute, during the compute-bound prefill phase the gain largely disappears, on hardware without native low-precision kernels dequantization consumes part of it, and the memory saving is itself 2.8–3.6× rather than 4×. Meta's measurements on a OnePlus 12 phone show the size of the shortfall: 4-bit Llama 3.2 1B and 3B decode 2.6× faster than their BF16 versions, and their resident memory falls by 40–50% [88] (C). The defensible statement is that 4-bit quantization converts a storage saving into a decode-time saving in proportion to the bandwidth it frees, and that single-stream interactive use, the dominant on-device regime, is where it does so.

Two further constraints bind on the edge. Prefill is compute-bound and sets time-to-first-token, which grows with context length; decode is bandwidth-bound and sets tokens per second. Grouped-query attention (GQA) and local–global attention shrink the KV cache that every decode step reads, which is why they matter more on a device than on a server. For a bounded call that a small model completes correctly, these properties make the small model cheaper per call and faster once the network hop is counted; whether it is cheaper per completed task, once retries, checks, and escalations are counted, is the question Section 7 takes up.

7 When Should a Small Model Be the Default in an Agent?

The preceding sections show that SLMs can be compressed to run on commodity and edge hardware at a characterized cost (Sections 4–6), and that the general benchmark record can establish neither their relative capability nor their fitness for agents (Section 5). This section assesses hypothesis H (Section 1.3) on agent-specific evidence. The premises and the conversion pipeline are those of Belcak, Heinrich, Diao, et al. [19], and the SLM-default, LLM-fallback design with cost per successful task as its metric is shared with Sharma and Mehta [21]; what this section adds is the grading of each line of evidence, including contrary results, the break-even and checking conditions of H, and the counterexamples of Section 7.4.

7.1 What Agents Delegate

An agent is a control loop in which language models operate alongside tools, memory, and a scaffold of prompts and parsers. The model is mostly asked to perform bounded sub-tasks: choose a tool and its arguments, extract fields, classify an intent, rewrite a query, summarize a passage, produce a short patch, or check a step. Belcak, Heinrich, Diao, et al. [19] characterize such calls as narrow, repetitive, and format-constrained, and estimate, by inspecting three open-source agents, that about 60% of the LLM queries in MetaGPT, 40% in Open Operator, and 70% in Cradle could be handled by appropriately specialized SLMs. These are the authors' judgments of replaceability, not measurements and not an application of H1–H4, so we grade them D. We found no trace study measuring the distribution of call types in deployed agents by count, cost, or contribution to task success.

7.2 The Premises

7.2.1 P1: Capability Sufficiency

The first premise is that a specialized SLM is good enough for the individual bounded call, not for the completed trajectory. Section 7.3.1 sets out the evidence, which is favorable for tool selection and argument filling but mostly developer-reported. Two further results bear on it. Controllable reasoning extends what can be delegated: Qwen3 [14] exposes a thinking budget, and distilled reasoning models such as Phi-4-mini-reasoning [92] report competition-mathematics results at 3–8B that earlier models of that size did not reach (C). And specialization, not size alone, appears to carry the gain: a controlled study of Qwen3 models from 0.6B to 32B, published after our snapshot, finds supervised fine-tuning with low-rank adaptation (LoRA) the strongest in-distribution adaptation

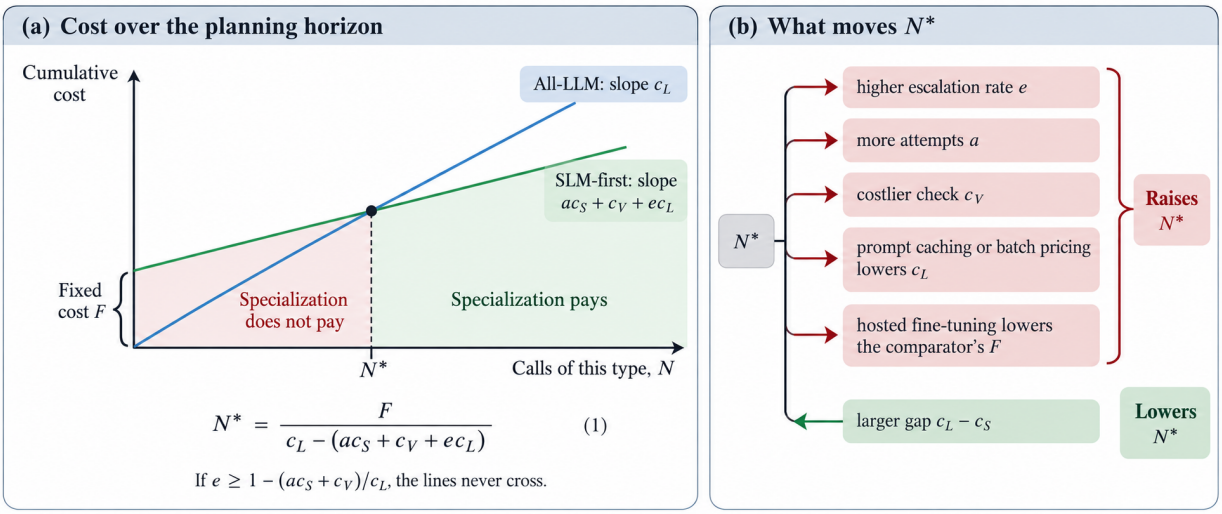


Figure 5. Break-even volume for specializing one call type (Equation 1). (a) An SLM-first system pays a fixed cost F and a per-call cost that includes attempts, the check, and escalations, so it is cheaper than an all-LLM system only above N^*; if $e \geq 1 - (a\,c_S + c_V)/c_L$, the lines never cross. (b) Factors that move N^*. Schematic: no axis values or prices are implied, and no measured N^* exists for any workload.

for tool calling across the whole range [93] (B). All of the per-call evidence concerns models below 10B or a single family.

7.2.2 P2: Economics

Belcak, Heinrich, Diao, et al. [19] estimate that serving a 7B SLM is 10–30× cheaper in latency, energy, and FLOPs than serving a 70–175B LLM, and Sharma and Mehta [21] estimate that cost per successful task falls by a similar factor; both are estimates rather than measurements (D). The break-even condition H2 (Equation 1, Figure 5) is how we state the premise instead, because four things that a per-token ratio omits decide the outcome. First, the unit is cost per *completed task*: a small model that needs a attempts per call, a check costing c_V, and escalation of a fraction e of calls to the large model saves only $c_L - (a\,c_S + c_V + e\,c_L)$ per call, and saves nothing once $e \geq 1 - (a\,c_S + c_V)/c_L$. Second, specialization has fixed costs F: logging and curating traces, removing personal data, fine-tuning per call type, evaluation, router calibration, retraining under drift, and maintaining a fleet of specialists. Third, the large-model cost c_L is not a list price: agent calls repeat long system prompts and tool schemas, which is the case that provider-side prompt caching and asynchronous batch pricing discount, and hosted fine-tuning of a provider's small model gives specialization without a fleet. Each of these lowers c_L or F for the comparator and so raises N^*, as does any further fall in hosted prices at fixed capability. Fourth, the comparator for server-side calls is not only a frontier model: a hosted general-purpose small model or a small-active-parameter MoE has small-model per-token cost and no specialization cost. The one measurement of cost per correct answer across model classes finds that lightweight models are cost-optimal on basic quantitative tasks and large models on knowledge-intensive ones [56] (B), which supports a conditional premise and not a general one. Finally, "zero marginal cost" on a device means no per-query charge to the operator, not zero cost to a user who pays in battery and thermal budget; offline, the large model cannot run at all, which makes the question one of capability rather than cost.

7.2.3 P3: Control

An agent needs its model to emit exactly the format the scaffold expects. Constrained decoding enforces well-formedness outside the model [94, 95], so syntactic validity for a supported schema does not depend on model size. This removes an objection to small models rather than giving them an advantage, since a large model can be constrained equally. What remains for P3 is the argument of Belcak, Heinrich, Diao, et al. [19] that a narrow output distribution is easier to impose on a small model by fine-tuning on the agent's own traces than to elicit from a frontier model by prompting, together with operational properties: small models are easier to version, roll back, host on-premises, and co-locate with their tools.

7.3 Agent-Specific Evidence

General benchmark scores are the wrong instrument for these premises (Section 5.3). This subsection assembles the evidence that bears on them, by the capability an agent exercises, and grades each line (Table 5).

7.3.1 Function Calling and Tool Selection

This is the best-evidenced capability. The Berkeley Function-Calling Leaderboard [96] checks function selection and argument filling with executable checks, in single-turn, parallel, and, from its third version, multi-turn settings; its predecessors [97–99] established that API-call generation is a skill on which general capability transfers only partially. Developer reports place specialized sub-10B models among the leading single-turn entries of 2024: APIGen's 7B model was reported to outperform several GPT-4 models and its 1B model GPT-3.5-Turbo [51]; the xLAM-7B card places it third at 88.24% (the first place belonged to an 8×22B member of the series) [53]; ToolACE reports an 8B model rivalling GPT-4 [52]; and on-device work reports 0.1–7B models competitive on narrower suites [100–103]. Meta reports Llama 3.2 3B at 67.0 on BFCL v2 against 67.1 for Llama 3.1 8B, and the 1B model at 25.7 [88]. For multi-turn use, APIGen-MT trains 1B–70B models on verified multi-turn trajectories and reports that they outperform GPT-4o and Claude 3.5 on τ-bench and BFCL, with smaller models surpassing larger ones in multi-turn settings and more consistent across repeated trials [104]. All of these are developer reports on public benchmarks, and the placements were not checked against archived leaderboard snapshots (C).

Independent evidence qualifies them. Kavathekar, Donakanti, Kumaraguru, and Vaidhyanathan [105] find that small models improve from zero- to few-shot to fine-tuned function calling but "struggle significantly with adhering to the given output format" (B), and Shen, Li, Chen, et al. [106] recover performance only by splitting planning, calling, and summarization across separately fine-tuned small models. Read together, the evidence supports the conditional form of H: per-call competence at this size is something specialization produces, not something an off-the-shelf small model has.

7.3.2 Structured Output: Validity Is Not Correctness

Constrained decoding guarantees that output parses, at an overhead its authors report as small [94, 95] (C). Three results limit what that buys. JSONSchemaBench finds that frameworks differ in schema coverage, efficiency, and output quality, so the guarantee holds for the features a framework supports [107] (B). Grammar-constrained decoding distorts the model's distribution, producing grammatical outputs whose likelihoods are not those the model assigns [108]. And Tam, Wu, Tsai, Lin, Lee, and Chen [109] report that format restrictions degrade reasoning (B), a result disputed by the developers of a constrained-decoding library on methodological grounds [110], so its size is uncertain. What constrained decoding does not do is make the content correct, and the gap between well-formed and correct output, a valid call to the wrong tool or with a wrong argument value, is where small models still lose ground. This is why H4 requires a correctness check (H4b) and not only a validity check (H4a).

7.3.3 End-to-End Agent Success

Here the evidence is weakest and partly contradictory. When the realistic multi-step benchmarks were introduced, success was low for every model: the best GPT-4 agent reached 14.41% on WebArena against 78.24% for humans [111], GPT-4 with plugins 15% on GAIA against 92% [112], and the best model about 2% on SWE-bench [113] (B). The one cross-class comparison of that period, AgentBench, found a large gap between commercial models and open models of 70B or less, attributed to long-horizon reasoning, decision-making, and instruction following [114] (B); it compared open with closed models of 2023, so model size is confounded with openness and training. Against it, the APIGen-MT developers report specialized small models ahead of frontier models on τ-bench [104] (C). We found no dated, independent, like-for-like comparison of current specialized sub-10B agents and frontier agents, so whether a size gap persists for specialized small models is open (D).

The failure mode these benchmarks expose is the one per-call metrics hide. τ-bench reports that even strong models succeed on under half of its tasks and degrade as the number of repeated trials grows [115], and τ^2-bench reports further drops when the agent must coordinate with an active user [116]. A trajectory of n calls at per-call reliability p completes at p^n absent recovery, so the difference between 0.98 and 0.95 per call is the difference between 82% and 60% over ten steps. A correctness check that triggers escalation (H4b) is what breaks this compounding; a validity check alone lets valid but wrong outputs pass and compound.

7.3.4 Routing, Escalation, and Cost per Completed Task

Cascades and routers report large savings on single-turn query benchmarks: FrugalGPT matches the best single model with up to 98% cost reduction [43], Hybrid LLM sends up to 40% fewer calls to the large model with no quality drop [44], AutoMix reports over 50% cost reduction at comparable quality [45], and RouteLLM over 2× at matched quality with transfer to unseen model pairs [46] (all C). RouterBench provides a common dataset of 405K inference outcomes for comparing routers [47]. None of these reports, inside an agent loop, how often the router is confidently wrong, how escalation rates move under distribution shift, or what the seam costs when a trajectory switches models.

Table 5. Agent-specific evidence bearing on hypothesis H, by capability, with the premise or condition each line bears on. Grades follow Section 5.1.2; "None" marks a quantity for which the search of Section 1.5 located no study. Per-call evidence is favorable but mostly developer-reported, independent evidence qualifies it, and the completed-task comparison H requires is absent.

Capability	What the evidence shows	Source class	Bears on	Grade
Single-turn function calling	Specialized sub-10B models among the leading entries in 2024	Developer reports [51–53, 88]	P1, H1	C
Multi-turn function calling	Specialized small models reported ahead of frontier models on BFCL multi-turn and τ-bench	Developer report [104]	P1	C
Format adherence without fine-tuning	Small models often fail the call format; fine-tuning helps most	Independent study [105]	P1	B
Specialization method	LoRA fine-tuning strongest in-distribution for tool calling at 0.6–32B	Independent study [93]	P1	B
Structured-output validity	Constrained decoding guarantees parseable output, model-independently	Method papers [94, 95]	H4a	C
Limits of constrained decoding	Coverage differs by framework; decoding distorts the distribution; reasoning effect disputed	[107–110]	H4	B
Multi-step reliability	Success degrades over repeated trials of the same task	Independent benchmarks [115, 116]	H4b	B
End-to-end success	Low for all models at introduction; open $\leq$70B models trailed commercial APIs in 2023	Independent benchmarks [111–114]	P1	B
Current size gap end to end	Contested: no independent like-for-like comparison of specialized sub-10B and frontier agents	–	P1	D
Estimated replaceable calls	40–70% of calls in three agents judged replaceable by specialized SLMs	Position paper [19]	H2	D
Router quality	Cascades and routers cut cost at matched quality on single-turn benchmarks	Author-reported [43–46]	P2	C
Router calibration, escalation rate	Not reported inside agent loops	–	P2, H2	None
Cost per correct outcome	Lightweight models cost-optimal on basic quantitative tasks, large ones on knowledge-intensive tasks (single-turn); agent cost and accuracy for frontier models	Independent [55, 56]	P2	B
Cost per completed task, SLM-first vs. all-LLM	Not reported	–	H	None

Cost per correct outcome has begun to be measured, but not for the comparison H requires. Cost-of-pass, the expected monetary cost of a correct solution, has been measured across model classes on single-turn tasks [56] (B); the Holistic Agent Leaderboard reports agent accuracy and cost across 9 frontier-class models and 9 benchmarks, and finds, for example, that higher reasoning effort reduced accuracy in most runs [55] (B); and Kapoor, Stroebl, Siegel, Nadgir, and Narayanan [54] argue that agents should be evaluated on joint cost and accuracy. We found no study that reports cost per completed task for an SLM-first agent against an all-LLM agent on the same workload. That is the measurement on which H is confirmed or refuted (Section 9.3).

7.3.5 What This Evidence Supports

The evidence supports a narrower claim than the one usually made. Developers report (C), and independent work qualifies (B), that a *specialized* sub-10B model can perform the individual delegated operation at a standard competitive with much larger models; an off-the-shelf small model often cannot. Validity can be guaranteed independently of model size for supported schemas; correctness cannot. Whether a specialized small-model agent completes realistic multi-step tasks at a comparable rate is contested, and whether it does so at lower cost per completed task has not been measured. The case for H is therefore strongest where a correctness check exists and the call type recurs often enough to repay specialization, and unproven elsewhere.

7.4 Where the Hypothesis Does Not Hold

For the following workload classes the conditions of H fail and a frontier model remains the right default.

Open-ended planning

When the agent must decide *what* to do, the call fails H1: there is no fixed schema and the space of acceptable plans is large. AgentBench locates the open-model gap in long-horizon reasoning and decision-making [114], and a planning error propagates to every later step.

Synthesis over long or heterogeneous context

H3 fails for large document sets, long trajectory histories, and multi-source retrieval requiring cross-document reasoning. Long context in a small model is a memory and prefill problem before it is a capability problem, and we found no evidence that small models close the gap on long-context synthesis (D).

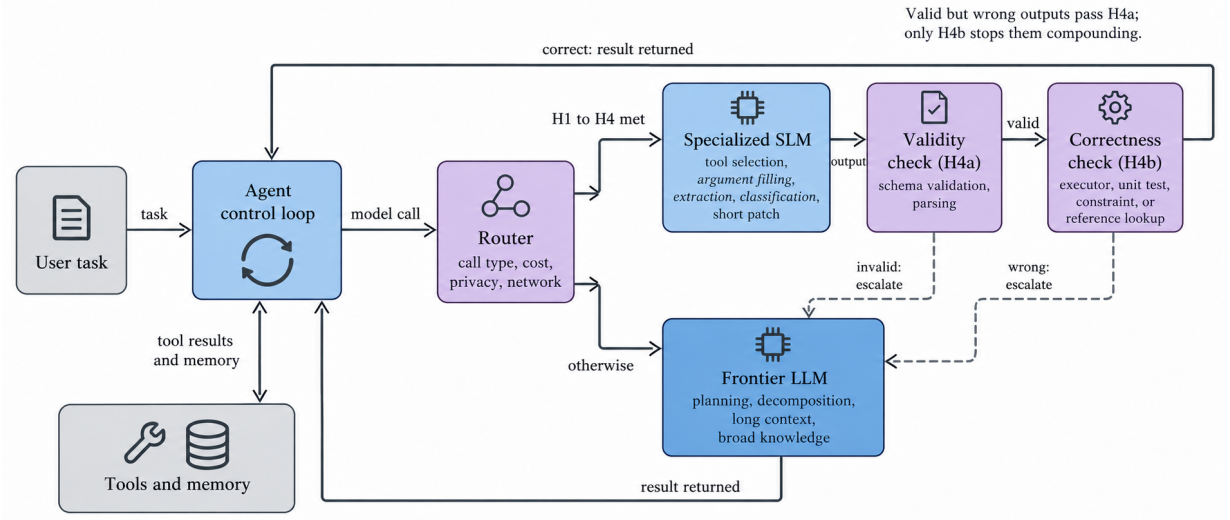


Figure 6. Heterogeneous SLM–LLM agent. A per-call router sends call types meeting H1–H4 to specialised small models by default; their outputs are checked for validity and correctness, and a failed check escalates the call to a generalist LLM, which also handles planning-like and long-context calls. The split between the paths varies by agent. After Belcak, Heinrich, Diao, et al. [19].

Uncheckable output
: H4b fails for open-ended generation, judgment calls, and tasks where verification is as hard as the task; errors then compound silently.

Low-resource-language and safety-sensitive calls
: The multilingual and safety deficits of Section 5.5 are concentrated in small models, and a router that sends such calls to a small model on size grounds degrades them silently.

Sparse or unstable call distributions
: H2 fails when a call type is rare, when traffic drifts faster than specialization can track (raising F), or when cached, batched, or fine-tuned hosted alternatives make c_L low; a hosted model is then cheaper in total.

Cold start
: Before deployment there are no logged traces for the conversion pipeline. The opening move is a frontier model with instrumentation, and migration afterwards.

Planning and uncheckable generation are not transitional: they are categories of work for which narrow competence and constrained output are the wrong properties, and the heterogeneous design below exists because of them.

7.5 Heterogeneous SLM–LLM Systems

For call types meeting H1–H4 the default inverts: SLMs handle them, and a generalist model handles planning, open-ended decomposition, long context, broad world knowledge, and calls whose small-model output fails a check [19, 21]. Figure 6 sketches the arrangement; the share of calls on each path depends on the agent and is not established in general.

7.5.1 Routing Designs

Three designs recur: cascades that try the small model first and escalate on a quality signal [43, 45], predictive routers that decide before generation [44, 46], and decomposing routers that assign sub-questions to a local or a cloud model [48]. H prescribes a cascade whose quality signal is the H4 check. Speculative decoding [49, 50, 117, 118] speeds up the large model without delegating answers, and an escalated call can itself be served speculatively. Appendix L.3 describes each.

7.5.2 Production Instances

Apple pairs an on-device model with a server model on Private Cloud Compute and routes to the server when the on-device model is expected to fall short [16]; its 2026 models were built in collaboration with Google [18, 91]. The design is motivated by privacy and latency as much as by cost, and Apple reports no cost per completed task, so it shows that the heterogeneous pattern is deployable at scale, not that it is cheaper. Google's Gemini Nano occupies the same on-device role on Android [3].

7.6 Conversion and Adoption

For an agent that already runs on a frontier model, Belcak, Heinrich, Diao, et al. [19] propose a six-step conversion: log the agent's calls, strip personal data, cluster them into recurring types, choose a candidate SLM per cluster, specialize it with LoRA or quantized low-rank adaptation (QLoRA), and repeat as traffic drifts. The clustering step is where H1, H3, and H4 can be checked and where the pilot estimates for H2 are made; clusters that fail stay with the generalist, and each converted cluster should be evaluated on cost per completed task against the unconverted agent. They also identify three barriers to adoption: sunk investment in centralized inference, evaluation on generalist leaderboards, and lower attention to SLM releases; Section 7.3 shows the second to be partly a real evidential gap. Appendix L gives the pipeline and barriers in full.

8 Discussion

8.1 What the Evidence Supports

Four conclusions follow from the preceding sections, each at the grade stated.

Capability per parameter has risen, and training data is the main lever. The strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not report, an across-source comparison whose direction alone we assert (D); a single-harness benchmark of 72 small models finds that reasoning tracks training method and data quality more than scale [57] (B).

To compress an existing small model, quantize first. Post-training quantization preserves fidelity better than post-training pruning at 0.5–3.8B (A), 4-bit weights are 2.8–3.6× smaller than FP16, and the decode speedup that follows is real only in single-stream, bandwidth-bound use, as Meta's measured 2.6× on a phone illustrates (Section 6.2). A single accuracy-retention figure per technique is not supported (Section 4.1).

The vendor evaluation record cannot support cross-model comparison. For the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five (B). Comparison tables built from vendor figures, including those in secondary sources, inherit this; single-harness third-party evaluations are the basis on which comparison should rest.

Right-sizing is a conditional design rule, not a general one. For a call type meeting H1–H4, a specialized small model is plausibly the right default: it is cheaper per call, faster once the network hop is counted, available offline, and its errors are caught by the check. That conclusion is graded D, because the quantity that decides it, cost per completed task against cached, batched, hosted, and frontier alternatives, has not been measured (Section 7.3.4), and Section 7.4 names the workloads where it fails.

8.2 Implications for Agentic Systems

Three implications follow for agent builders. Routing must be capability-aware, not size-aware: a "small model unless hard" router degrades non-English and safety-sensitive calls silently (Section 5.5). The economical unit is often a fine-tuned small model per recurring call type rather than one general SLM, which is what the conversion pipeline produces. And the check is part of the design, not an implementation detail: without a correctness check (H4b), an SLM-first system inherits the compounding of per-call errors that the cascade was meant to stop.

8.3 Domain Applications

The domains where SLM deployment is most advanced are each a workflow of bounded, repeated language operations under regulatory or cost constraints, and Appendix M reviews them with the state of the evidence for each. Healthcare is the best evidenced: Meerkat-7B reports 70.6% on MedQA [119] and compressed clinical encoders lose under 5% in macro-AUROC [76] (C), but exam benchmarks are not deployment evidence, and on-premises hosting removes one data-transfer question without satisfying HIPAA or the GDPR. Legal NLP has public benchmarks, including LegalBench [120], LexGLUE [121], and CUAD [122], but we found no evaluation of current sub-10B decoder models across them under one protocol, so its case for small on-premises models is structural rather than evidential. Code has mature small models evaluated on public benchmarks [123–125], and multilingual coverage trades breadth against depth at this size [89]. In every vertical, deployment interest runs ahead of the evidence.

8.4 Access and Sustainability

Running a capable 3B model on inexpensive hardware, with no per-query charge and, within the scope conditions of Section 6.1, no transfer of prompt content, widens access for organizations facing cloud cost, data-sovereignty rules, or unreliable connectivity. The environmental case is narrower than usually stated. Per-query compute scales with active parameters, but per-query energy also depends on utilization, and a well-batched server can be more

efficient per token than an idle device; aggregate effects depend further on teacher training, device manufacture, grid intensity, update distribution, and rebound. We therefore make no aggregate energy claim (Appendix P).

8.5 Limitations

No original measurement. No benchmark was executed for this survey; the one grade-A claim rests on cited controlled studies, and Section 9.3 is written to be acted on.

Inventory provenance. The inventory began from secondary listings, and 205 records were first compiled with the assistance of a language model. Every included record now links a developer source checked on 8 October 2026, but coverage of models no listing surfaced, especially from smaller developers, is not guaranteed, and the change of seed source in 2026 affects year-to-year comparisons.

Literature search. The technique literature was not screened under a recorded protocol, so statements that no study exists are bounded by the search described in Section 1.5.

Definitional conventions. The twenty-billion threshold, the C1/C2 split, and the sub-10B cohort are conventions; Section 1.2.1 gives the counts under other thresholds.

Scope of H's evidence. Every agent-specific result concerns models below 10B, and H relies on C1 more than C2: for server-side calls a small-active-parameter MoE has small-model per-call cost, so it is a comparator in the test of H rather than part of H.

Coverage. The benchmarks audited and most of the compression literature are English-centric, the audit covers 2023–2025 releases, and the snapshot ages; the update policy of Appendix B.5 bounds rather than removes this.

8.6 Ethical Considerations

Alignment is harder to preserve at small scale, several research-grade SLMs ship without safety training, and fine-tuning can erode what exists [90]; deployments that give small models tool access should treat output filtering and monitoring as mandatory. Compression can degrade performance unevenly across languages and dialects [77], and pruning more than quantization [74], so high-stakes applications need per-group performance floors. Against these risks, on-device SLMs remove one egress channel, work without connectivity, and need no subscription, which matters most in under-resourced healthcare and education settings.

9 Challenges and Future Directions

9.1 Model-Level Challenges

Hallucination is more pronounced at small scale, where fewer facts fit in the parameters, and retrieval augmentation is the main mitigation [126, 127]. Multi-step reasoning is where the gap to large models is widest; test-time compute, symbolic tools, and reasoning distillation narrow it. Hardware and model architecture are still designed separately, although NPU efficiency depends on matching attention and feed-forward shapes to the accelerator. Compression can amplify bias [74, 77]. Appendix N treats each.

9.2 Open Problems for Agentic SLM Systems

Several problems stand between hypothesis H and routine practice. **Correctness checking:** H4b requires a check that catches valid but wrong outputs at a cost well below a large-model call; executors with observable effects and unit tests exist for code and tool calls, but most agent call types lack them. **Difficulty estimation without an oracle:** routers use win prediction [46], predicted difficulty [44], self-verification [45], or decomposition [48], but calibration under distribution shift is unsolved. **Multi-objective routing:** accuracy, latency, energy, cost, privacy sensitivity, and network availability must be weighed jointly, and size-only routing degrades non-English and safety-sensitive calls (Section 5.5). **Handoff without a seam:** escalating mid-task must not produce a visible discontinuity. **Agent-appropriate, cost-aware evaluation:** function-calling and interaction benchmarks [96, 115, 116] and cost-aware agent leaderboards [55] exist, but report specialized sub-10B models sparsely and do not compare SLM-first with all-LLM agents. **Fleet management:** many fine-tuned specialists raise questions of versioning, drift detection, and retraining cadence that a single model does not.

9.3 Empirical-Verification Agenda

Six studies would most reduce the uncertainty documented here. Studies 1–4 use one model set chosen by a stated rule: the instruction-tuned checkpoint of the most downloaded sub-10B variant on the Hugging Face Hub at the time of the study for each of the cohort families (Phi, Qwen, Gemma, Llama) and for the 2025–2026 families (for example Qwen3, Gemma 3n, SmolLM3, Granite 4, LFM2), so that the results bear on both the audited and the current generation. Each fixes the harness version, prompts, extraction rules, and decoding settings in advance, runs sampled tasks with at least three seeds, and reports confidence intervals.

1. **Harness-consistent reruns (RQ3).**
   Run the model set on the eight benchmarks of the coverage ledger under one `lm-evaluation-harness` configuration, separately for pre-trained and instruction-tuned checkpoints, so that every vendor figure in the ledger has an independent counterpart.
2. **Quantization degradation (RQ2).**
   Measure FP16, INT8, and three INT4 formats (GGUF Q4_K_M, GPTQ, AWQ) on the same benchmarks with matched baselines, to establish per-model degradation and whether it varies with size.
3. **Edge measurement (RQ2).**
   Measure decode and prefill throughput, time-to-first-token, resident memory, and energy per token on a phone, a single-board computer, a Jetson-class module, and a laptop, reporting batch size, context length, and runtime version.
4. **Agent-specific evaluation (RQ4).**
   Run the model set on BFCL multi-turn splits and τ-bench, with and without constrained decoding, reporting pass^k consistency and classifying failures as invalid (caught by H4a) or valid but wrong (caught only by H4b).
5. **Cost per completed task (RQ5, the test of H; Figure 7).**
   Fix a workload with a checkable success criterion, identify the call types meeting H1, H3, and H4 by inspection, and record pilot estimates of N, F, a, and e for H2 before the comparison. Run six configurations end to end, each with the same checks and the same retry budget: (i) a frontier model throughout; (ii) the same with prompt caching and batch pricing where the provider offers them; (iii) a hosted general-purpose small model or small-active-parameter MoE without specialization; (iv) a hosted small model fine-tuned through the provider's service; (v) a specialized SLM with escalation to the frontier model on a failed check, which is the configuration H prescribes; and (vi) the specialized SLM without escalation, to measure what escalation buys. Report per configuration task success, calls and retries per completed task, escalation rate, the rate of valid but wrong outputs that pass the checks, latency, and monetary and energy cost per completed task, with F reported separately. The decision rule is a pre-registered one-sided non-inferiority test of (v) against (i) on task success at margin δ, with task-clustered bootstrap intervals, together with a comparison of cost per completed task against every other configuration. For independent arms at a success rate near 0.5, a one-sided test at $\alpha = 0.05$ with power 0.8 needs about $n \approx 3.09/\delta^2$ tasks per arm: about 7,700 for $\delta = 2$ percentage points, 1,240 for 5, and 310 for 10; a paired design on the same tasks needs fewer. A τ-bench-scale workload of a few hundred tasks can therefore decide only a margin of about ten points, and a tighter margin needs a synthetic or production workload of thousands of tasks. Under the refutation rule of Section 1.3, H is rejected for a workload class if (v) fails non-inferiority or costs more per completed task than any other configuration, and rejected as a general rule if it fails on a majority of at least three pre-registered workloads with distinct call types.
6. **Call-type prevalence (H2).**
   On logged traces of deployed agents, measure the share of calls, of cost, and of contribution to task success that meets H1–H4, replacing the three estimates of Section 7.1 with measurements.

Studies 1–4 are practical on one modern GPU in weeks; studies 5 and 6 need an agent harness, an API budget, and, for study 6, access to production traces. Study 5 is the one that bears directly on the paper's central hypothesis.

9.4 Further Directions

Beyond verification, Appendix O sets out multilingual evaluation, on-device continual learning, ensembles of specialized SLMs against a single LLM, multimodal evaluation, linear-time hybrid architectures, and federated fine-tuning.

10 Conclusion

This survey fixed a two-condition definition that separates a model's per-token cost from its storage cost, cataloged 179 SLMs and, separately, 30 small-active-parameter mixtures of experts released from 2023 to September 2026 with every record's developer source checked, reviewed compression along separate axes, and quantified why the vendor evaluation record cannot support cross-model comparison: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five. Single-harness third-party evaluations, not vendor tables, are the basis on which comparison should rest.

On the agentic question, we stated the case for small models as a conditional hypothesis. For call types that are bounded, recurrent enough to clear a break-even volume that counts retries, checks, and escalations, short-context, and checkable for correctness as well as validity, a specialized small model should be the default, with escalation to

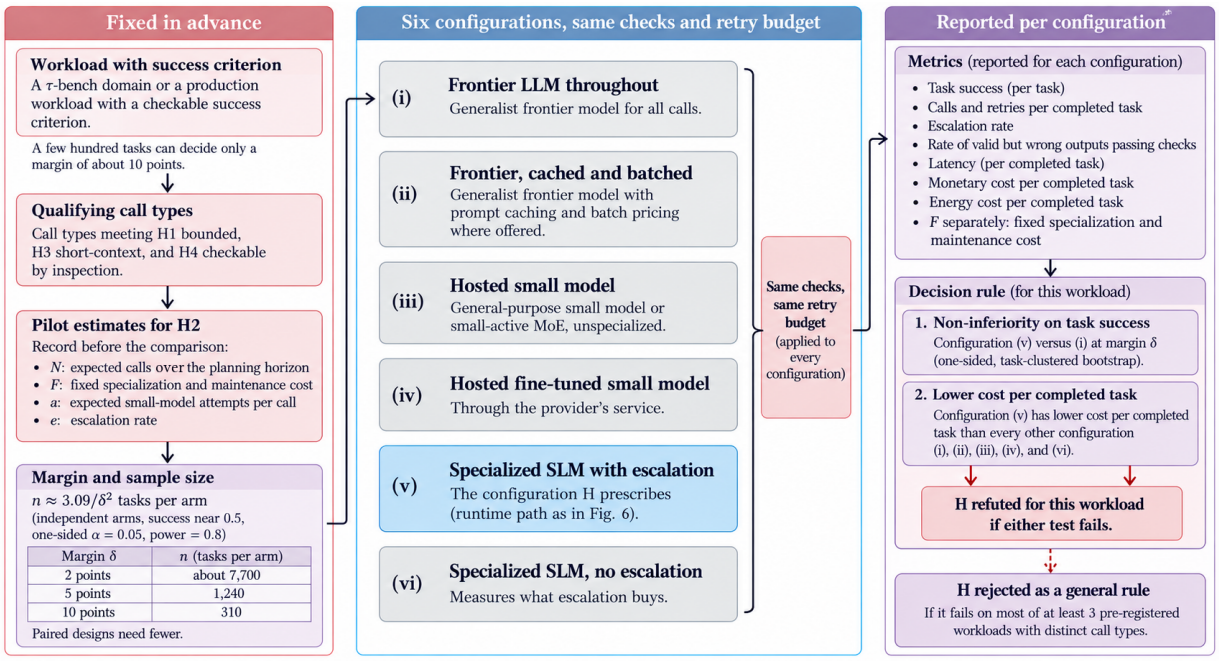


Figure 7. The pre-registered test of hypothesis H (study 5). Call types, pilot estimates for H2, the margin, and the sample size are fixed before running; six configurations are run end to end under the same checks and retry budget; the decision rule combines a non-inferiority test on task success with a comparison of cost per completed task. The sample sizes assume independent arms at a success rate near 0.5, one-sided $\alpha = 0.05$, and power 0.8. No results are implied: the study has not been run.

a frontier model when a check fails. The evidence supports this unevenly. Developers report specialized sub-10B models competitive on single- and multi-turn function calling, while independent work shows that off-the-shelf small models often are not; end-to-end comparisons are contested; cost per correct answer has been measured for single-turn tasks and frontier agents; and no study measures cost per completed task for a small-model-first agent against the alternatives, including cached, batched, and fine-tuned hosted models. Where the conditions fail, for planning, long-context synthesis, and uncheckable work, a frontier model remains the right choice.

The pre-registered comparison of Section 9.3, with its comparators, decision rule, and sample size, is the shortest route from what has been reported to what has been established. Until it is run, the position defended here is a design hypothesis with a stated domain: specialized small models doing the bounded, recurrent, checkable work, and a large model for planning, synthesis, and the calls no cheap check can verify.

Declarations

Funding

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU), grant number IMSIU-DDRSP2602.

Use of Generative AI

A large language model assisted in compiling 205 of the candidate model records from public listings in March 2026 (Section 1.5). During revision, Anthropic's Claude was used to assist with editing the manuscript, checking bibliographic metadata against arXiv and Crossref records, writing the scripts that check each record's developer source and apply the inclusion rule, and transcribing the coverage ledger from the documents it cites. Every included record's developer source was checked through the Hugging Face Hub API or against the developer's own page, and the authors reviewed all generated output, verified factual claims and references against their sources, and take full responsibility for the content.

Competing Interests

The authors declare that they have no competing interests.

Ethics Approval

Not applicable: this is a review of publicly available documentation and involves no human participants, human data, or animals.

Table 6. Reporting dimensions for a small language model, the deployment question each answers, and the coverage achieved across the 209-variant inventory. "Derived" means the value follows arithmetically from others rather than being reported.

Dimension	Question it answers	Bounds	Inventory coverage
Total parameters	How large is the download?	C2	209/209
Active parameters per token	What does one token cost?	C1	36/209 stated below total; else = total
Weight memory at a precision	Will the weights fit?	C2	Derived
Peak runtime memory	Will it fit while running?	C2	Not reported by vendors
FLOPs per token	Compute cost per token	C1	Derived (approximate)
KV-cache size per token	Cost of long context	C1, C2	Derived from head config
Context length	Largest usable input	–	135/209
Target hardware	What did the vendor test on?	–	Vendor-stated only

Data and Code Availability

No benchmark was run for this review; all performance figures are reported values from public sources, whose provenance is given in Appendix Q. The following artifacts are provided as supplementary files with this submission and will be deposited in a versioned archival repository under a persistent identifier on publication: the variant inventory `merged_llm_database.json` (version 3.6), one record per variant with its parameter counts, C1/C2 status, inclusion decision and reason, developer source, the result and date of the source check, and a changelog naming every record added, moved, or removed between versions; the source-check log `source_check_log.csv`; the audit coverage ledger `audit_coverage_ledger.csv`; and the scripts that correct and verify the records, apply the inclusion rule, build the ledger, and generate every inventory table and figure (`fix_records_v3_6.py`, `verify_sources.py`, `apply_inclusion_rules.py`, `build_audit_ledger.py`, `generate_model_tables.py`, `generate_figures_v2.py`).

Author Contributions

M.G.R.: Conceptualization, Methodology, Investigation, Data curation, Software, Writing (original draft), Visualization. **H.A.:** Conceptualization, Methodology, Writing (review and editing), Supervision, Project administration. **S.J.A.:** Investigation, Validation, Writing (review and editing). **W.E.A.:** Validation, Writing (review and editing), Funding acquisition. All authors read and approved the final manuscript.

A Reporting Dimensions and Artifact Vocabulary

A.1 Reporting Dimensions

A single parameter count is not enough to determine whether a model will run on a given device. Table 6 lists the dimensions this survey reports where the primary source supplies them, and states which of them each of our claims depends on. Where a source omits a dimension, the entry is marked as not reported rather than estimated.

A.2 Model-Artifact Vocabulary

Because the inventory counts artifacts rather than papers, we also fix the following. A *family* is a named line of models sharing an architecture and a training recipe (Qwen2.5, Gemma 3). A *variant* is one released parameter size within a family. Where a vendor publishes the base and the post-trained checkpoint of that size in one release, the inventory records one variant and marks its post-training status as "both" (63 of the 209); where only one is published, or the two were released separately, the status is recorded as that one. The post-training status is therefore a field of the variant, not a second count. A *checkpoint* is a specific set of weights published under a version identifier; a variant may be revised through several checkpoints, and the inventory records the variant, not each checkpoint. A *release* is a dated publication event covering one or more variants. A *pretrained* (or base) model is the artifact produced by pretraining alone; an *instruction-tuned* model has undergone supervised instruction fine-tuning and usually a preference-optimization stage; a *reasoning-tuned* model has additionally been trained, by distillation of long chain-of-thought traces, reinforcement learning, or both, to emit extended intermediate reasoning before its answer. These three post-training statuses are not interchangeable for evaluation purposes, which is why an evaluated artifact must be identified to the checkpoint even though the inventory counts variants, and conflating them is one of the specific defects in the public record documented in Section 5.2.

Two conventions follow from this vocabulary and are used consistently throughout. Family names are written as their developers write them and with a space before the version, as in Gemma 3, Qwen2.5, Phi-4, and Llama 3.2; the original 2023 LLaMA release keeps its own capitalization, since Meta renamed the line to "Llama" from Llama 2

onward. Checkpoint identifiers are written in the hyphenated form used on the model hub, as in `Gemma-3-4B-it` or `Qwen2.5-7B-Instruct`, and are used wherever the specific artifact rather than the family is meant.

B Inclusion Criteria, Source Verification, Released Artifacts, and Update Policy

B.1 Inclusion Criteria and Appraisal

Inclusion criteria for the technique literature were: (T1) the work concerns models at or below roughly twenty billion parameters, or a compression, adaptation, routing, or deployment technique whose evaluation includes that regime; (T2) it is a peer-reviewed paper, an archived preprint, or a vendor technical report with a stable identifier; and (T3) it reports enough methodological detail to determine what was measured.

Inclusion criteria for model variants were: (M1) a source from the developer (technical report, paper, model card, model repository, or release announcement) that states the parameter count and architecture; (M2) a public artifact or a documented evaluation card; (M3) satisfaction of C1, with C2 status recorded separately; and (M4) a dated release. Fine-tunes and merges of another party's checkpoint were excluded unless their developer released a technical report describing its own training recipe (as for the DeepSeek-R1-Distill models), as were quantized re-uploads, API aliases of a listed checkpoint, and variants differing only in serialization format.

What is computed and what is decided. C1, C2, M1, and M4 are computed from recorded fields by `apply_inclusion_rules.py`: M1 from the stated parameter count and the field `developer_source_found` written by the source check below, and M4 from `release_date_precision`. Three kinds of exclusion are named editorial decisions listed in the script, because they cannot be computed from fields: encoder-decoder architecture (Flan-T5), derivative fine-tunes without a technical report (Nous Hermes 2 Mixtral, Vicuna-7B v1.5, Vicuna-13B, Jan-v1-4B), and an API alias (Mistral Tiny). Records whose size is undisclosed cannot be classified against C1 or C2; all of them, whatever their marketing (for example GPT-4o mini, Amazon Nova Micro and Lite, Claude Haiku, Gemini Flash-Lite, Nex-N2-Mini), are kept in the comparison part of the file with class `undisclosed` and are neither counted nor plotted as SLMs or LLMs. Release dates are recorded at the precision their sources give: day for 174 of the 209 included records, month for 22, and year for 13.

Appraisal of empirical works. Works relied on for quantitative claims were appraised on whether they name the harness, state the evaluation conditions, report a matched baseline, and identify the evaluated checkpoint; the outcome, together with the reporting party's independence, is the evidence grade of Section 5.1.2.

B.2 Source Verification

On 8 October 2026, `verify_sources.py` checked every included and excluded record, and every comparison record of stated size, against the Hugging Face Hub API: for each Hub repository a record links, it recorded whether the repository exists, its owner, creation date, licence tag, and the parameter count of its safetensors weights, and it classified the record's reference as a developer page, a paper, a Hub model page, or a secondary source (a model aggregator or press article). The result is written into each record (`source_check`, `developer_source_found`) and into `source_check_log.csv`; Figure 8 places the check in the build process. Of the 209 included records, 197 link an existing repository owned by the developer and 12 link the developer's own announcement, paper, or code repository.

The check led to the following corrections, applied by `fix_records_v3_6.py` and listed in the file's metadata. Twelve records whose reference was a model aggregator or a press article (the DeepSeek V4 Flash, Ling 3.0 Flash, Laguna, North Mini Code, Inkling Small, Nex-N2-Pro, Dots3-Note Preview, and Granite 4.0 Nano records) were re-sourced to the developer's Hub repository, with the original reference kept as `discovered_via`; Ling 3.0 Flash Fin, recorded as API-only, is open-weight under MIT. Ling 3.0 Flash Sante, for which no developer source exists, fails M1 and was excluded. Voxtral Mini, previously excluded as undisclosed with a link to a different model, is the open-weight Voxtral Mini 3B (4.7B including its audio encoder, Apache 2.0) and was included. The BitNet b1.58 record, which described the 2024 paper but linked the 2025 checkpoint, now describes the released 2B4T model. The SmolLM2 records, which linked the SmolLM3 page, the Mistral 7B v0.3 release date (2024, not 2023), and the Granite 4.1 references were corrected. For nine records the ratio of the Hub parameter count to the vendor's nominal size falls outside 0.8–1.25 (for example Gemma 2-2B, nominal 2B, Hub 2.6B; Qwen2.5-Omni-7B, Hub 10.7B including its audio components; BitNet b1.58 2B4T, whose ternary weights are stored packed); the recorded size is the vendor's statement, the Hub count is kept in the log, and none of the differences changes a record's C1 or C2 status.

B.3 Count History

Earlier drafts reported 126 variants. Version 3.3 of the inventory added 27 releases from May to September 2026 (153). Version 3.4 placed records by the rule rather than by how they had been filed: 64 models meeting C1 had

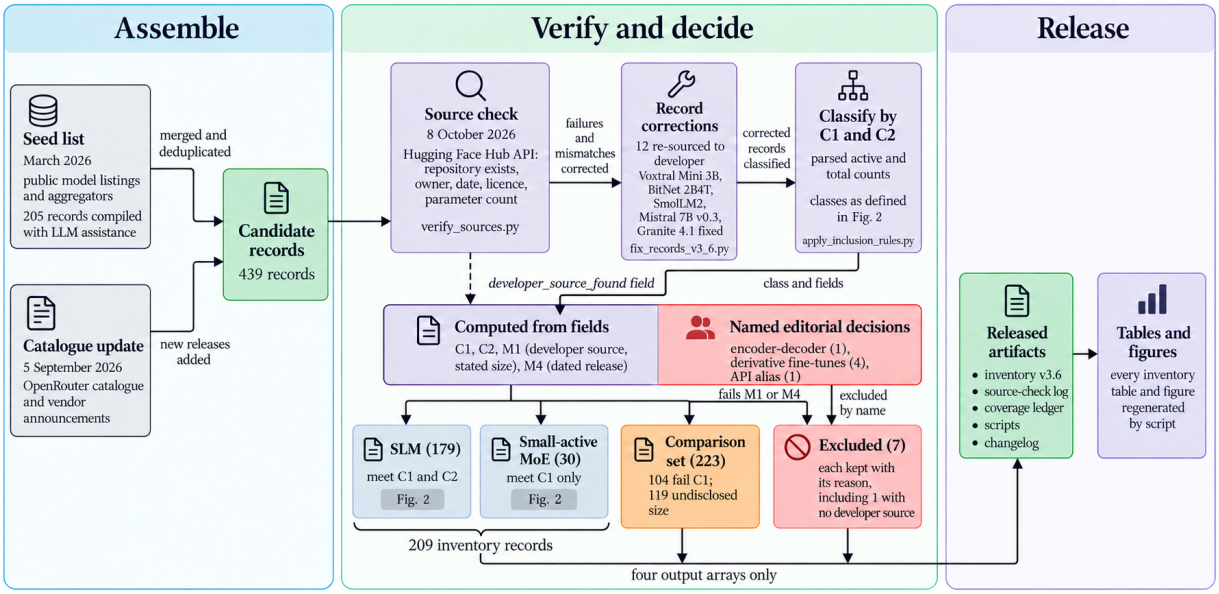


Figure 8. How the inventory was assembled, verified, and filtered. Secondary listings only seeded the candidate list; every record was then checked against its developer source and corrected where needed. C1, C2, M1, and M4 are computed from recorded fields; the encoder-decoder, derivative, and alias exclusions are named editorial decisions. All four arrays, the source-check log, the coverage ledger, the scripts, and the changelog are released.

been filed outside the SLM inventory and two 24B dense models inside it (215). Version 3.5 corrected a parser that had read the total in place of the active count for five legacy MoE records and removed eleven records by the inclusion criteria (209: 178 SLMs and 31 small-active-parameter MoEs). Version 3.6 applied the source verification above: Voxtral Mini 3B entered as an SLM and Ling 3.0 Flash Sante left, and the four records of undisclosed size that version 3.5 had excluded as "vendor-positioned small" were moved to the comparison set with every other undisclosed record (209: 179 SLMs and 30 small-active-parameter MoEs; 223 comparison records, of which 104 are of disclosed size and fail C1 and 119 are undisclosed; 7 excluded). The changelog in the released file names every record added, moved, or removed at each step.

B.4 Released Artifacts

The following are released with the paper: the inventory `merged_llm_database.json` (version 3.6), with one record per variant carrying its parameter counts, C1/C2 status, class, inclusion decision and reason, developer source, source-check result and date, and the excluded and comparison records alongside; the source-check log `source_check_log.csv`; the audit coverage ledger `audit_coverage_ledger.csv` (Appendix I.2); the scripts that correct and verify records, apply the inclusion rule, build the ledger, and generate every inventory table and figure; and the changelog. They are provided as supplementary files with the submission and will be deposited in a versioned archival repository under a persistent identifier, so that the exact snapshot underlying this paper, and any later one, can be retrieved.

B.5 Snapshot and Update Policy

This paper reports model releases through 4 September 2026, as recorded in version 3.6 of the inventory (8 October 2026). Releases appearing between submission and acceptance are added to the released inventory and recorded in the changelog, and are discussed in the text only where they change a conclusion; tables and figures are regenerated from the inventory at the last revision before acceptance. Works appearing during review that bear on RQ2–RQ5 are added to the synthesis, as was done for one study published after the snapshot [93]. The released artifact is the authority on what the inventory contained at any given date.

C Key SLM Families

C.1 Microsoft Phi Series

The Phi series represents Microsoft's systematic exploration of data-quality-driven model development. Phi-1 (1.3B) [30] demonstrated strong code generation performance through training on synthetic "textbook-quality" data. Phi-2 (2.7B) [128] extended this to general language understanding. Phi-3-mini (3.8B) [10] formalized the approach with 3.3 trillion training tokens, reporting 68.8% on MMLU (5-shot) and 82.5% on GSM8K (8-shot, chain-of-thought)

for the instruction-tuned checkpoint (C). Phi-3.5-mini [129] added 128K context length support with improved multilingual capabilities. Phi-4 (14B) [130] introduced a fundamentally enhanced training pipeline incorporating ~40% synthetic data, reporting 84.8% on MMLU and 80.4% on MATH under OpenAI's simple-evals settings, and reporting that it exceeds its teacher GPT-4o on several STEM benchmarks (C). Phi-4-mini (3.8B) [131] updated the architecture with GQA (24 query, 8 KV heads), a 200K-token vocabulary, and training on 5 trillion tokens, reporting 88.6% on GSM8K (8-shot, chain-of-thought; C). Phi-4-reasoning [132] is Phi-4 after supervised fine-tuning on curated prompts with o3-mini reasoning demonstrations; Phi-4-reasoning-plus adds a short phase of outcome-based reinforcement learning. Both are reported to outperform DeepSeek-R1-Distill-Llama-70B and to approach the full DeepSeek-R1 on AIME 2025 (C).

C.2 Google Gemma Series

Google's Gemma family [11] derives from the Gemini model architecture and training infrastructure. Gemma 1 offered 2B and 7B variants trained on 3T and 6T tokens respectively, using Multi-Query Attention (MQA) for the 2B model and standard Multi-Head Attention for the 7B. Gemma 2 [133] introduced significant architectural improvements: local-global interleaved attention layers, logit soft-capping, and knowledge distillation for the 2B and 9B variants (from a larger teacher model). The pre-trained Gemma 2-9B reports 71.3% on MMLU (5-shot; C). Gemma 3 [31] further advanced the architecture with a 5:1 local-to-global attention interleaving ratio (reducing the sliding window from 4096 to 1024 tokens) and replaced the logit soft-capping of Gemma 2 with QK-norm. Gemma 3 distills all model sizes from larger teachers and introduces a 1B variant whose instruction-tuned checkpoint reports 48.0% on MATH, close to the 49.4% of the instruction-tuned Gemma 2-9B with 9× fewer parameters (C).

C.3 Alibaba Qwen Series

The Qwen2 [134] and Qwen2.5 [12] families from Alibaba are trained on up to 18 trillion tokens, far beyond Chinchilla-optimal ratios. Qwen2.5 offers models at 0.5B, 1.5B, 3B, 7B, 14B, 32B, and 72B parameters, all sharing a consistent GQA-based architecture with rotary position embeddings (RoPE) and SwiGLU activations. The pre-trained 3B model reports 65.6% on MMLU (5-shot), 79.1% on GSM8K, and 42.6% on MATH (both 4-shot) (C); its developers credit the STEM and code share of the training data for its mathematics and coding results.

C.4 Other Significant SLM Families

TinyLlama (1.1B) [135] adopted the Llama 2 architecture and tokenizer and trained on about 1 trillion tokens of SlimPajama and StarCoder data for roughly three epochs, showing that extended training of very small models continues to pay. Mistral 7B [136] combined grouped-query attention, introduced by Ainslie, Lee-Thorp, Jong, Zemlyanskiy, Lebrón, and Sanghai [60], with sliding-window attention in a 7.3B-parameter model, and reports outperforming Llama 2 13B while being 1.8× smaller. Meta's Llama 3.2 [32] released 1B and 3B models for edge and mobile devices, derived from Llama 3.1 8B by pruning and distillation, trained on up to 9 trillion tokens with 128K context. SmolLM2 (135M, 360M, 1.7B) [59] from Hugging Face trained on 11 trillion tokens of carefully curated data. H2O-Danube (1.8B) [137] introduced sliding window attention in a compact model trained on 1 trillion tokens. OLMo 2 (1B, 7B, 13B) [138] from AI2 was released as a fully open model (data, code, weights, and training logs), achieving competitive performance through QK-norm and carefully staged training. StableLM-2 (1.6B) [139] was trained on about 2 trillion tokens (counting repeated epochs) of multilingual data. The BLOOM family (560M, 1.1B, 1.7B, 3B, 7.1B) [140] remains notable as a multilingual model trained on 46 languages using ALiBi positional encoding, though its 350B-token training budget is modest by current standards.

D Training, Compression, and Deployment Background

D.1 Knowledge Distillation and Model Compression

Knowledge distillation, first formalized by Hinton, Vinyals, and Dean [33], transfers knowledge from a large "teacher" model to a smaller "student" by training the student to match the teacher's soft probability distributions (logits) rather than hard labels. This approach preserves the teacher's learned representations of inter-class similarities, information that is lost in one-hot labels, and has proven remarkably effective in the LLM era. Modern distillation strategies for language models encompass several complementary approaches.

Logit-based distillation trains the student to minimize the KL divergence between its output distribution and the teacher's softened distribution, parameterized by a temperature τ. Google's Gemma 2 [133] and Gemma 3 [31] employ this approach extensively: all Gemma 3 model sizes, including the 27B variant, are trained using knowledge distillation from a larger teacher. The distillation loss is combined with a standard next-token prediction loss, with the relative weighting serving as a key hyperparameter. Feature-based distillation [141] additionally aligns intermediate representations between teacher and student, capturing structural knowledge that may not be fully

expressed in output logits. Microsoft's Orca 2 [142] demonstrated a variant where a smaller model is taught to use different reasoning strategies depending on task complexity, outperforming models with 5–10× more parameters on reasoning benchmarks.

Synthetic data generation represents an increasingly dominant form of implicit distillation. Rather than direct logit matching, a powerful LLM (e.g., GPT-4, Claude 3.5) generates high-quality training examples that are then used to train smaller models. The Phi series pioneered this approach with the "Textbooks Are All You Need" methodology [30], generating synthetic textbook-style content with carefully controlled complexity and diversity. Phi-4's training data is reported to be about 40% synthetic, generated across 50 dataset types including multi-agent conversations and structured reasoning chains [130] (C); its developers attribute much of its STEM performance to this data, an attribution that has not been tested in a controlled comparison against web-corpus training at equal compute. Post-training quantization, pruning, and adaptation methods are reviewed in Appendix G; together with distillation during training they form the usual SLM optimization pipeline.

D.2 Efficient Fine-Tuning Methods

Full fine-tuning updates all model parameters using task-specific data, which for a 7B-parameter model can require tens of gigabytes of accelerator memory once gradients and optimizer states are included. This cost has made parameter-efficient fine-tuning (PEFT) central to the practical adaptation of SLMs, particularly when the target deployment environment is a single GPU or an edge-oriented workstation.

LoRA [36] addresses this constraint by freezing the pretrained backbone and learning low-rank update matrices for selected linear layers. For a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is parameterized as $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters while preserving the ability to specialize a model for downstream tasks. QLoRA [37] extends this idea by quantizing the frozen base model to 4-bit NormalFloat (NF4) precision and training the adapters in higher precision, allowing models that would ordinarily require multi-GPU setups to be fine-tuned on a single high-memory accelerator.

Beyond LoRA-style adapters, the PEFT landscape includes prompt-based and prefix-based adaptation strategies that shift task information into learned soft prompts or attention prefixes rather than updating the backbone itself. These approaches can be attractive when multiple lightweight task specializations must be maintained simultaneously. However, they tend to underperform LoRA-family methods on complex reasoning tasks, where direct adaptation of attention and feed-forward projections provides a stronger inductive bias for task transfer.

The choice between prompting and fine-tuning therefore depends on task specificity, data availability, and deployment constraints. In-context learning offers zero-training adaptation and is well suited to broad tasks when labeled data are scarce, but it consumes context length and raises inference latency. By contrast, PEFT methods incur an upfront training cost yet reduce per-query overhead and typically yield larger gains on narrow or reasoning-heavy tasks. Prompting and QLoRA results are rarely reported under matched conditions, so they cannot be compared across models from the published record; measuring them is one of the verification studies proposed in Section 9.3.

D.3 Edge AI and On-Device Inference

The edge AI inference stack has matured rapidly, transforming SLM deployment from a research exercise to a production-ready capability. The llama.cpp framework [38] is the most widely used runtime for CPU and mixed CPU-GPU inference, and the one against which the others in Table 13 position themselves, supporting the GGUF quantization format with dozens of precision variants (Q2_K through Q8_0). Its ecosystem includes llamafile (single-executable packaging), Ollama (user-friendly serving), and extensive platform support spanning x86, ARM, Apple Silicon, and NVIDIA GPUs. Benchmarking on single-board computers shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models [143].

Meta's ExecuTorch [39] is an on-device runtime with a small base runtime and backends for CoreML, Qualcomm QNN, ARM, MediaTek, and Vulkan; Meta used it for the on-device measurements of its Llama 3.2 1B and 3B models [88]. MLC-LLM [40] deploys through ML compilation (TVM) to iOS, Android, web, and embedded GPUs. ONNX Runtime [144] provides cross-platform deployment with optimization passes for diverse hardware. WebLLM [145] runs inference in the browser with WebGPU and reports retaining up to 80% of native performance (C). Google's LiteRT (formerly TensorFlow Lite) targets mobile CPUs, GPUs, and NPUs [146]; its vendor-reported speedups on Qualcomm NPUs are measured against CPU execution on the vendor's chosen models and are not comparable with the other figures here.

The fundamental constraint in edge deployment is memory bandwidth rather than compute. Flagship phones and mainstream laptops offer on the order of 50–100 GB/s of memory bandwidth, unified-memory laptop designs considerably more, and single-board computers well under 30 GB/s, versus 2–3 TB/s for datacenter GPUs [147]. Single-stream autoregressive decoding reads the full weight set once per generated token, so in that regime it is

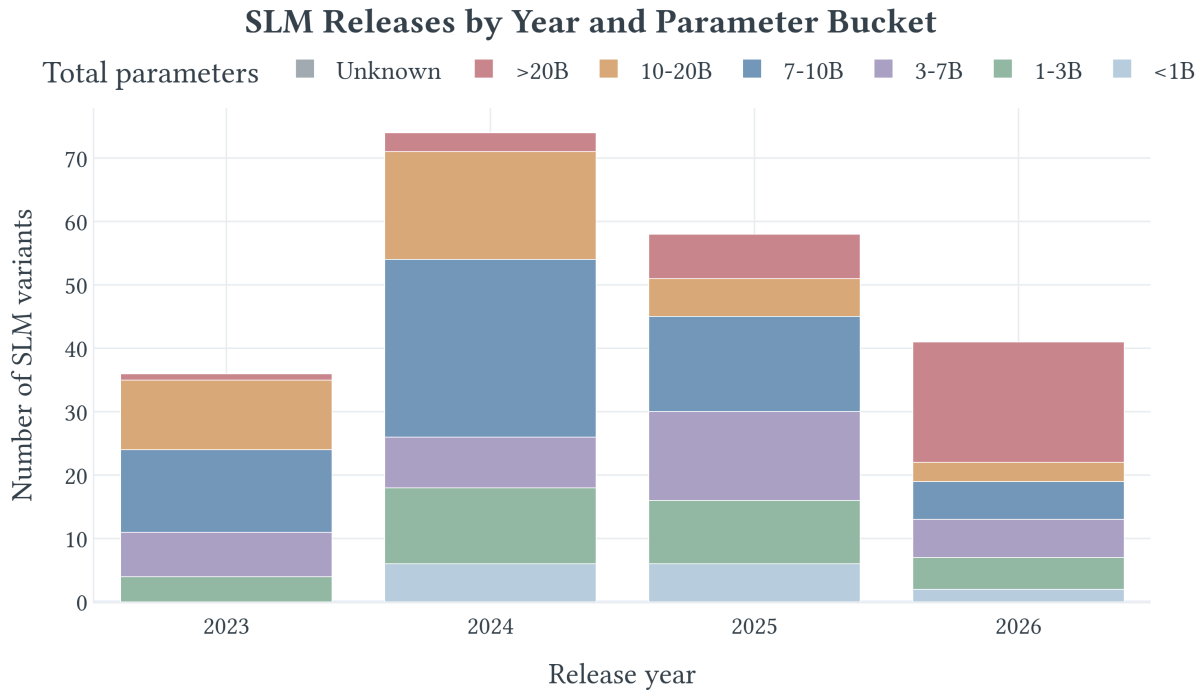


Figure 9. Inventory variants by release year and total-parameter band (2026 to 4 September). The 7–10B band is the largest in 2023, 2024, and 2025; the sub-1B and 1–3B bands appear in numbers from 2024; in 2026 the largest band is the one above 20B total, which consists of small-active-parameter MoEs.

bandwidth-bound and quantization from 16 to 4 bits converts a storage saving into a decode-time saving roughly in proportion to the bandwidth freed. The proportionality is regime-dependent rather than general: it weakens under batching as arithmetic intensity rises, largely disappears during the compute-bound prefill phase, and is reduced further on hardware without native low-precision kernels. Section 6.2 sets out the conditions in full. Because single-stream interactive use is the dominant on-device regime, INT4 quantization is nevertheless a performance enabler there and not only a storage optimization.

E Inventory Figures

The figures below are generated from version 3.6 of the released inventory (releases through 4 September 2026) and are referred to from Section 3 of the main text. Unless stated otherwise, they cover the 209 inventory records, that is, the 179 SLMs and the 30 small-active-parameter MoEs. Figure 9 gives releases by size band, Figure 10 the size span of the principal families, Figure 11 context length by year, Figure 12 the dense and MoE split, and Figure 13 supported modalities.

F Taxonomy Facets in Detail

This section expands Section 3 of the main text.

F.1 Architecture

Dense transformers. Most SLMs use a dense decoder in which every parameter is active for every token, from SmolLM2-135M [59] to 7–14B models such as Mistral 7B [136] and Phi-4. SmolLM3-3B [148] uses grouped-query attention with no positional encoding in a share of its layers and a three-stage curriculum over 11.2T tokens; Qwen3's dense line [14] spans 0.6B–32B with a switch between thinking and non-thinking modes. Three refinements recur. Grouped-query attention [60], adopted by Mistral 7B and most later SLMs, divides the KV cache by the ratio of query to KV heads. Long context is reached by raising the rotary base or rescaling positions: Llama 3.2 uses a base of 500,000 to reach 128K tokens, Gemma 3 uses 1,000,000 on its global-attention layers, Phi-4 uses 250,000 with a 16K context, and Phi-3-mini's 128K checkpoint uses LongRoPE rescaling. Gated feed-forward layers are standard, SwiGLU in most families and GeGLU in Gemma.

Mixtures of experts. MoE layers [61, 62] activate a subset of expert feed-forward blocks per token through a learned router, so total and active parameters differ. Mixtral 8×7B [149] (46.7B total, 12.9B active) is the canonical

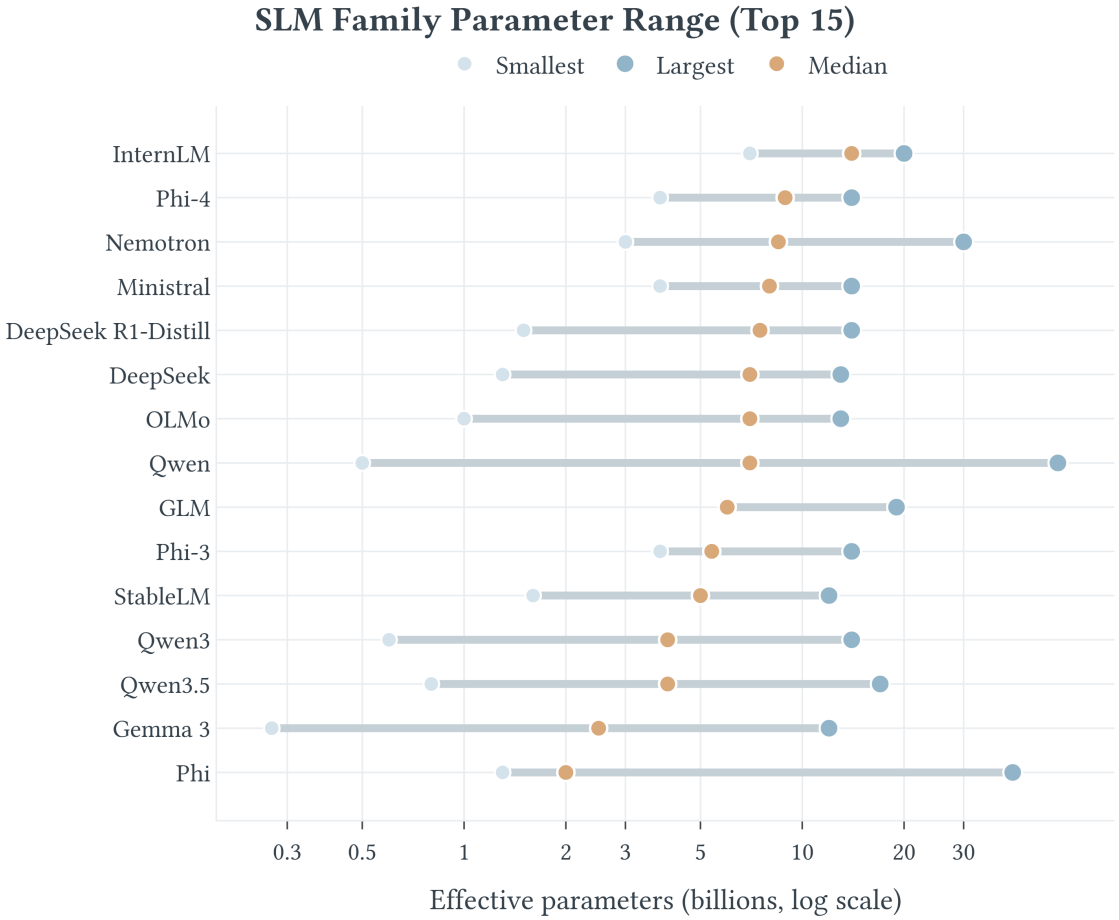


Figure 10. Parameter span (minimum, median, maximum, in billions, log scale) of the principal families in the inventory.

example; DeepSeek-MoE [150] uses finer-grained and shared experts (about 2.8B active of 16B); Qwen3-30B-A3B [14] activates 3B of 30B; and Llama 4 Scout [151] activates 17B of 109B. All of these are small-active-parameter MoEs under C1 and C2, while Granite 4.0 Tiny (about 7B total, 1B active) [63] is an SLM. Apple's third-generation on-device model is a 20B-total sparse model that activates 1–4B parameters and keeps its full weights in flash, swapping experts into DRAM on demand [18]; its total count meets C2 at the bound, and its resident memory is set by the swapping policy rather than by the total.

Hybrid state-space models. Mamba [64] introduced hardware-aware selective state-space layers that process sequences in linear time. Jamba [65] interleaves Mamba and attention blocks (one attention layer in eight) and reports a KV cache eight times smaller than Mixtral's at 256K context. Falcon-H1 [13] spans 0.5B–34B with parallel Mamba-2 and attention blocks; IBM's Granite 4.0 [63] interleaves Mamba-2 and attention at roughly 9:1 with a fine-grained MoE and reports over 70% memory reduction for long-context and multi-session inference relative to transformer-only models (C); NVIDIA's Nemotron Nano 2 [66] replaces most attention layers with Mamba-2 in a 9–12B model and reports up to 6× the throughput of Qwen3-8B on long reasoning traces (C); and the Qwen3.5 small series uses Gated DeltaNet layers with one full-attention layer in four [67]. Linear-time layers matter most where SLMs are deployed: long, streaming, memory-bound contexts on edge hardware.

F.2 Training Strategy

Trained from scratch. Phi showed that "textbook-quality" synthetic and curated data yields large gains at fixed size [30, 152]: Phi-1 (1.3B) was trained on under 8B tokens yet reached competitive Python results. Gemma 3 [31] scales its token budget with size, from 2T tokens for the 1B variant to 14T for the 27B, under a curriculum that increases the share of mathematics, code, and multilingual data. Recurring practices are classifier-based quality filtering, synthetic data from larger teachers [78, 153], curricula that move toward harder data late in training, and continued pre-training for long context.

Distilled. Distillation [33] transfers capability from a teacher, through its internal representations (white-box) or its outputs and generated text (black-box). Orca [78] distilled GPT-4 explanation traces into a 13B student. The

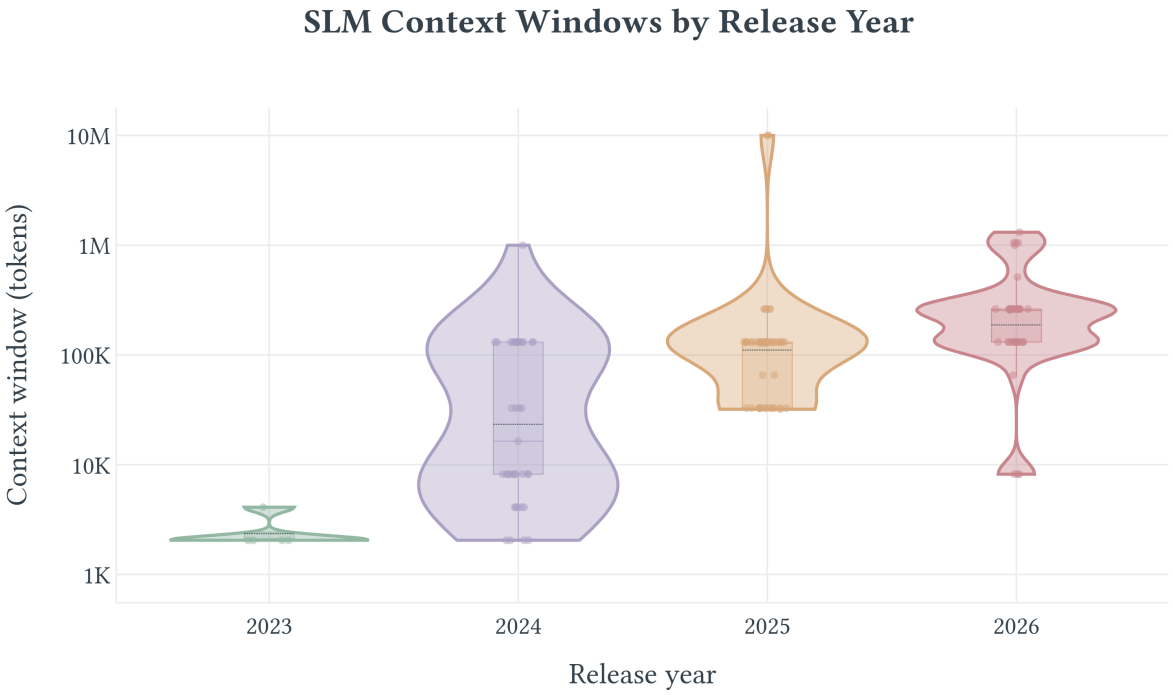


Figure 11. Supported context window of inventory variants by release year (log scale), for the records that state one: 5 of 36 in 2023, 39 of 74 in 2024, 52 of 58 in 2025, and 39 of 41 in 2026. Each violin is a kernel-density estimate; the inner box marks the interquartile range, the dashed line the mean, and the dots individual models. Because 2023 coverage is thin and 2026 includes small-active-parameter MoEs, the trend is indicative.

DeepSeek-R1 distillations [79] fine-tune Qwen2.5 and Llama students from 1.5B to 70B on long chain-of-thought traces from DeepSeek-R1, and Phi-4-mini-reasoning [92] combines distillation from a reasoning teacher with reinforcement learning at 3.8B. Gemma 3 and Gemma 4 [15, 31] distill every size from a larger teacher.

Pruned and distilled. NVIDIA's Minitron pipeline [69] prunes Nemotron-4 15B in depth and width, guided by activation-based importance, and retrains the pruned 8B and 4B students by distillation from the unpruned model, needing up to 40× fewer training tokens per derived model than training from scratch (a 1.8× compute saving for the family); a follow-up applies the recipe to Llama 3.1 8B and Mistral NeMo 12B [75]. Meta derived Llama 3.2 1B and 3B by pruning Llama 3.1 8B and distilling from the 8B and 70B models [32].

F.3 Modality

Text-only. Most families ship text-only variants, and text-only models dominate the inventory (Figure 13).

Vision–language. PaliGemma (3B) [154] combines a SigLIP image encoder with a Gemma decoder and reports strong transfer after fine-tuning. Phi-3-vision (4.2B) [10] adds an image encoder to Phi-3-mini for document and chart understanding. Gemma 3 [31] attaches a SigLIP-based encoder, operating on 896 × 896 inputs with Pan-and-Scan cropping, to its 4B, 12B, and 27B variants; the 1B variant is text-only with a 32K context. Qwen2.5-VL [155] at 3B and 7B supports documents, charts, and video.

Audio and multimodal. Gemma 3n [156] nests a 2B-effective sub-model inside a 4B-effective one (MatFormer) and keeps per-layer embeddings off the accelerator, so that its 5B and 8B raw models run with the resident memory of roughly 2B and 4B models. Gemma 4 [15] spans 2.3B-effective to 31B with dense and MoE variants and a thinking mode. Voxtral Mini 3B (4.7B including its audio encoder) adds speech understanding and transcription to a 3B decoder under an open licence [157], and Apple's on-device models are multimodal [16, 18].

F.4 Application Domains

Most SLMs are general-purpose models evaluated on broad benchmarks. Developer-trained domain models target narrow tasks:

Healthcare

Meerkat-7B [119], fine-tuned on chain-of-thought traces synthesized from medical textbooks, reports 70.6% on MedQA, which its authors describe as the first 7B result above the roughly 60% USMLE pass threshold (C).

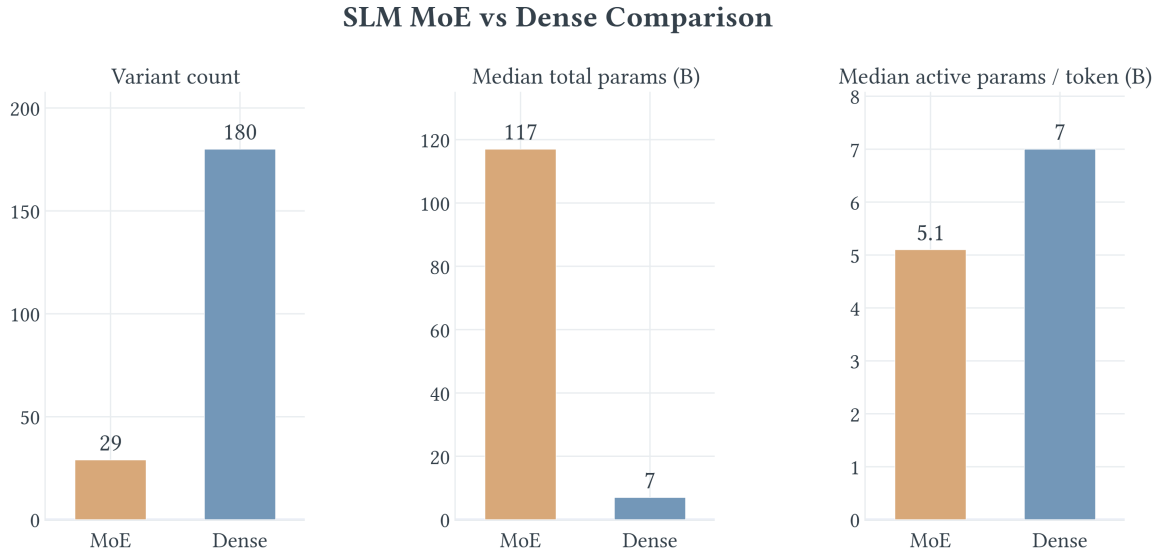


Figure 12. Mixture-of-experts versus dense variants in the inventory: variant count (left), median total parameters (centre), and median *active* parameters per token (right). MoE models carry a much larger total parameter count at a per-token cost comparable to dense models, which is the distinction between conditions C1 and C2.

OptimCLM [76] compresses clinical BERT encoders with under 5% macro-AUROC loss on outcome prediction (C).

Code

Code Llama [123], StarCoder2 [124], and DeepSeek-Coder [125] are code-specialized families evaluated on HumanEval and MBPP (Appendix M.3).

Legal

Small models fine-tuned on statutes, case law, and contracts target clause classification, obligation extraction, and regulatory triage. On-premises deployment can help meet privilege and data-residency obligations, subject to the scope conditions of Section 6.1; public evaluation of current small models in this domain is sparse (Appendix M.2).

Multilingual

Aya 23 [89] allocates the capacity of an 8B decoder to 23 languages and reports outperforming the 13B Aya-101 [158] on the languages both cover (C); Aya-101 is an mT5-based encoder-decoder, outside C1. Qwen2.5-7B [12] documents coverage of Chinese, Japanese, Korean, Arabic, and the major European languages.

G Compression Methods in Detail

This section expands the summary of Section 4 of the main text.

G.1 Post-Training Quantization

Post-training quantization needs only a small calibration set (typically 128–512 samples) and no gradient computation, which is why it is the dominant paradigm. GPTQ [34] quantizes layer by layer, using an approximation of the inverse Hessian of the layer-wise reconstruction loss to choose rounding decisions. AWQ [35] observes that about 1% of weights, those on large-magnitude activation channels, account for a disproportionate share of the quantization error, and applies per-channel scaling to protect them; its authors report higher accuracy than GPTQ at matched bit-width. At 8 bits the obstacle is activations rather than weights: LLM.int8() [70] keeps outlier dimensions in 16-bit, and SmoothQuant [71] migrates the difficulty from activations to weights to enable W8A8 integer kernels; both report accuracy essentially unchanged from FP16. SqueezeLLM [159] uses sensitivity-based non-uniform quantization with a dense-and-sparse decomposition. GGUF [38] packages weights, vocabulary, and metadata in one file, with block-wise "K-quant" schemes from Q2_K to Q8_0; mixed schemes such as Q4_K_M keep the most sensitive tensors at higher precision.

Two independent benchmarks bear on small models specifically. ThinkSLM [57] evaluates 72 small models, including quantized and pruned derivatives, on 17 reasoning benchmarks with three repeats, and finds that

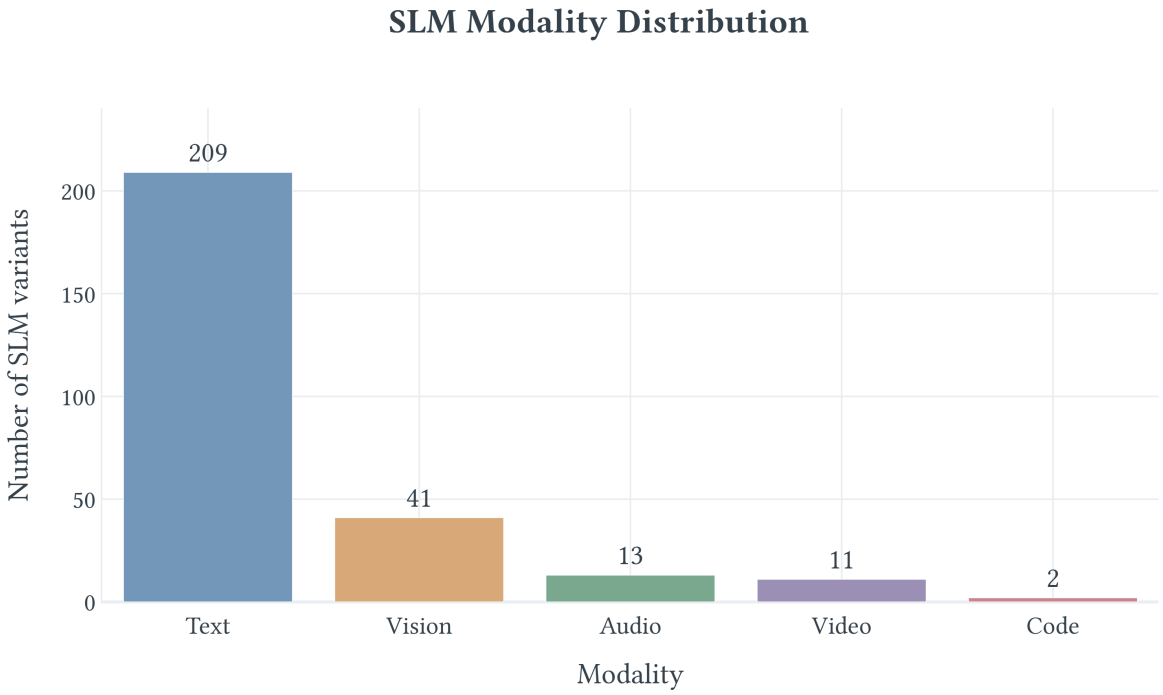


Figure 13. Input modalities supported by inventory variants (a variant may support several). Text dominates, vision is common, and audio and video remain a small minority.

quantization preserves reasoning ability while pruning disrupts it. SLMQuant [72] evaluates quantization methods on two sub-1B models (SmolLM-135M and Qwen2.5-0.5B) on knowledge, commonsense, and language-modeling tasks, and finds that methods tuned on large models transfer imperfectly and that sensitivity depends on architecture: under W4A8 quantization the larger of its two models scored lower than the smaller, which the authors attribute to architectural differences rather than size. Neither study establishes that 4-bit loss grows monotonically as model size falls, and we make no such claim.

G.2 Quantization-Aware and Low-Bit Training

Quantization-aware training simulates quantization in the training loop so that weights become robust to it. It recovers part of the accuracy that post-training quantization loses at 2–3 bits, with the margin widening as bit-width falls, but needs training data and compute, which is why it appears mainly in vendor pipelines such as Apple's 2-bit on-device quantization [16]. Its limit case is training in the low-bit format from the start: BitNet b1.58 [68] restricts weights to $\{-1, 0, 1\}$, and the released 2B4T model was trained in that format on four trillion tokens.

G.3 Pruning

SparseGPT [73] extends the GPTQ machinery to weight removal and prunes OPT-175B and BLOOM-176B to 50–60% unstructured sparsity in one shot with a negligible increase in perplexity. Wanda [80] removes the weights with the smallest product of magnitude and input-activation norm, with no weight update, and its authors report parity with SparseGPT on LLaMA and Llama 2; that parity does not hold below 4B [74]. Structured pruning removes heads, channels, or layers and yields models that run on dense kernels: LLM-Pruner [81] removes coupled structures selected by gradient information and recovers quality with a short LoRA tuning; ShortGPT [82] deletes the layers that change their input least, and we found no study establishing how far this tolerance holds below 7B (D). Sheared-LLaMA [83] prunes Llama 2 7B to 1.3B and 2.7B and reports outperforming same-size open models at about 3% of the compute of training them from scratch (C).

H Knowledge Distillation Variants

H.1 White-Box vs. Black-Box KD

The survey by Yang, Lu, Zhu, et al. [160] classifies LLM distillation into two paradigms:

White-box knowledge distillation (KD) has access to the teacher's intermediate representations. TinyBERT [161] pioneered layer-by-layer feature alignment, matching student hidden states, attention matrices, and embedding

layers to teacher counterparts via MSE loss. In the decoder-only LLM setting, this requires pairing student and teacher layers, which is non-trivial when sizes differ substantially, and is computationally expensive but yields superior retention of teacher capabilities.

Black-box KD uses only the teacher's generated outputs, either token probabilities or free-form text, as the supervision signal. Orca [78] exemplifies text-output black-box KD: GPT-4 generates explanation traces and step-by-step reasoning, which a 13B LLaMA-based student learns to imitate. DeepSeek-R1 [79] extends this to long chain-of-thought reasoning distillation, fine-tuning 1.5B–70B Qwen and Llama students on the reasoning traces of its large model trained by reinforcement learning; its authors report that the reasoning patterns of the large model can be transferred to smaller ones in this way (C).

H.2 Chain-of-Thought Distillation

A particularly productive form of black-box KD is chain of thought (CoT) distillation [162]: the teacher model generates explicit reasoning steps, and the student learns to produce or internalize these steps. Feng, Li, Chenglin, Chen, Yu, and Zhang [163] demonstrate counterfactual distillation, which augments CoT traces with contrastive incorrect reasoning, as a way to sharpen the student's reasoning boundaries. Liao, He, Xu, Zhang, Liu, and Zhao [164] propose Neural-Symbolic Collaborative Distillation, using a neuro-symbolic teacher to provide structured reasoning supervision that is more transferable to small students than purely neural CoT.

H.3 Data Augmentation in KD

The survey by Xu, Li, Tao, et al. [165] highlights data augmentation (DA) as a central paradigm within KD frameworks. When the student lacks sufficient coverage of the teacher's knowledge space, the teacher can generate synthetic training instances targeting specific skill gaps, the mechanism underlying the Phi family's success [30]. Ballout, Krumnack, Heidemann, and Kühnberger [166] use a roughly 3B teacher's most influential input tokens, identified by attribution methods such as saliency maps, as rationales for the student, and report improvements over standard fine-tuning and over prior distillation methods on four datasets.

H.4 Self-Distillation

An emerging approach eliminates the external teacher entirely. Fu, Yu, Han, et al. [167] propose Dynamic Self-Distillation, in which the model uses its own predictions from previous mini-batches as soft targets during fine-tuning, providing a form of implicit regularization that reduces overfitting on small fine-tuning datasets, a critical challenge for SLMs.

I Audit Protocol Detail

This section expands the protocol summary of Section 5.1 of the main text.

I.1 Benchmark Tasks and Metrics

The benchmarks that recur across the reviewed release documents span seven capability areas: MMLU [168] for broad knowledge; ARC-Easy and ARC-Challenge [169] and HellaSwag [170] for commonsense and language reasoning; GSM8K [171] and MATH [172] for mathematical reasoning; HumanEval [173] and MBPP [174] for code generation; TriviaQA [175] and SQuAD 2.0 [176] for question answering; MNLI [177] for natural language inference; and SST-2 [178] for sentiment classification. Each benchmark carries a standard metric: accuracy for multiple choice, exact match or F1 for question answering, pass@1 for code, exact match for GSM8K, and symbolic equivalence for MATH. Instruction-following (MT-Bench-style [179]) and summarization are reported too sparsely and too heterogeneously to compare at all, and the conventions behind them are described in Appendix Q.5. Section 5.3 takes up separately the question of whether any of these benchmarks bears on agentic suitability.

Five conditions determine what a benchmark number means, and a source that omits any of them has not reported a comparable result. *Benchmark version:* MMLU exists in the original and in MMLU-Pro, ARC in Easy and Challenge splits, GSM8K in original and GSM8K-Platinum revisions, and HumanEval in the original and HumanEval+ forms, and these are not interchangeable. *Prompt template:* whether the model's own chat template is applied, whether options are lettered or restated, and whether a system prompt is present. *Answer extraction:* whether a multiple-choice answer is scored by comparing per-option log-likelihoods or by parsing a generated letter, which can differ by several points on the same model and benchmark, and for chain-of-thought tasks, which span of the generation is taken as the answer. *Decoding settings:* greedy versus sampled, temperature, and for pass@k the number of samples. *Harness:* the framework and its version, most often `lm-evaluation-harness` [180] or HELM [84], each with its own defaults for all of the above.

Table 7. Coverage ledger for the instruction-tuned checkpoints of the 14 cohort models from the four largest vendors: the protocol under which each model's primary release document reports each benchmark. Numbers are shot counts; "c" marks chain-of-thought; "SE" OpenAI simple-evals defaults; "u" a value reported without a stated protocol; "–" not reported. Sources and locations are listed in `audit_coverage_ledger.csv`; transcribed on 8 October 2026.

Model	MMLU	MMLU-Pro	ARC-C	HellaSwag	GSM8K	MATH	HumanEval	MBPP
Phi-3-mini	5	–	10	5	8c	–	0	3
Phi-3.5-mini	5	0c	10	5	8c	0c	0	3
Phi-4-mini	5	0c	10	5	8c	0c	–	–
Phi-4	SE	–	–	–	–	SE	SE	–
Qwen2.5-0.5B	–	u	–	–	u	u	u	u
Qwen2.5-1.5B	–	u	–	–	u	u	u	u
Qwen2.5-3B	–	u	–	–	u	u	u	u
Qwen2.5-7B	–	u	–	–	u	u	u	u
Llama 3.2-1B	5	–	0	0	8c	0c	–	–
Llama 3.2-3B	5	–	0	0	8c	0c	–	–
Gemma 2-2B	u	–	–	–	–	–	–	u
Gemma 2-9B	u	–	–	–	–	–	–	u
Gemma 3-1B	0	0	–	–	0c	0	0	3
Gemma 3-4B	0	0	–	–	0c	0	0	3
Reported (of 14)	10	8	5	5	11	11	9	10

The coverage of these five conditions across the reviewed sources is what Section 5.2 reports on, and it is poor. Which benchmarks a vendor reports, in which version, under how many shots, and with which extraction rule varies enough to prevent a common table.

I.2 Coverage Ledger

Table 7 reproduces the coverage ledger summarized in Section 5.2. Its scope is the instruction-tuned checkpoints of the 14 cohort models from Microsoft, Alibaba, Meta, and Google, against the eight benchmarks that recur in their headline reporting; the other four cohort models (TinyLlama, SmolLM2, StableLM 2, Mistral 7B), each from a different developer, were not transcribed. Each cell was transcribed from the model's primary release document or official model card, listed with its table in the released CSV. Only coverage and stated protocol are recorded, not scores. Of the 112 cells, 69 carry an instruction-tuned value and 45 of those state a protocol; no benchmark is reported for all 14 models; and the largest group of models sharing one stated protocol on any benchmark is five (MMLU at 5 shots; GSM8K at 8 shots with chain-of-thought). Of the reconciliation rules written before collection, rule (a) (exclude a value that cannot be cross-checked against a second source) would exclude nearly every vendor cell by construction, so the finding is stated without it: protocol mismatch alone (rule (b)) and separate pre-trained and instruction-tuned tabulation (rule (c)) are sufficient to prevent a common table.

I.3 Hardware and Quantization Scope

The review reads reported behavior against six reference configurations spanning the cloud-to-edge spectrum (Table 10) and three precisions: FP16 (reference), INT8, and INT4, with GGUF Q4_K_M, GPTQ group-128, and AWQ group-128 treated as distinct INT4 subclasses when the source distinguishes them. Inference-framework attribution (vLLM [181], llama.cpp [38], ONNX Runtime [144], ExecuTorch [39], MLC-LLM [40]) follows the stack used by the source of each figure; cross-framework comparisons are made only when compared figures share a framework or are explicitly flagged.

I.4 Limitations of the Evidence Base

Because the figures are compiled from heterogeneous public sources rather than a single controlled pipeline, four limitations bound the strength of every comparison below. **Prompt-format and harness variation:** different sources use different templates and answer-extraction heuristics, which can shift reported accuracy by several points under the same nominal benchmark. **Self-reporting bias:** vendor-reported scores are systematically more favorable than third-party reproductions; where third-party figures exist, comparisons preferentially use them and flag the divergence. **Condition-coverage sparsity:** not every model has reported scores for every task, tier, or precision. Missing values are never imputed, interpolated, or carried across from a sibling model size or from a base checkpoint to its instruction-tuned variant; a tuple with no qualifying source is recorded as absent, and absence is itself reported, since the pattern of what vendors decline to publish is part of the finding of Section 5.2. No

statistic in this paper is computed over a set from which absent tuples were silently dropped. **Snapshot staleness:** the ecosystem evolves monthly, and figures reflect the state of public reporting at the collection window. These limitations motivate the empirical-verification agenda of Section 9.3.

I.5 Quantization Impact

The reported picture for the sub-10B regime is summarized in Table 3 of the main text. INT8 post-training quantization is reported as effectively lossless by the authors of the main methods [70, 71] (C). At 4 bits, an independent benchmark of 72 small models finds reasoning largely preserved [57], and an independent benchmark of two sub-1B models finds that methods tuned on large models transfer imperfectly, with sensitivity depending on architecture [72] (B); neither establishes that the loss grows monotonically as size falls. AWQ is reported to retain more accuracy than GPTQ at matched bit-width [35] (C), and mixed schemes such as GGUF Q4_K_M keep the most sensitive tensors at higher precision [38]. In the controlled comparisons at 0.5–3.8B, post-training quantization preserved fidelity better than post-training pruning [57, 74] (A); Section 4.2 gives the scope.

Per-model, per-benchmark degradation figures are not reported consistently enough across this cohort to tabulate: sources differ in harness, calibration set, and group size, and few publish a matched FP16 baseline alongside the quantized result. Measuring them under a single toolchain is one of the verification studies proposed in Section 9.3.

The memory consequence, by contrast, is firmly established, because it follows from the storage format rather than from measurement. Table 8 gives weight footprints estimated from parameter count and effective bits per weight. The FP16 column is exactly two bytes per parameter. The four-bit columns correspond to 4.4–5.8 effective bits per parameter: four-bit weights plus per-group scales and zero-points, with the embedding and output layers typically kept at higher precision, which is a larger share of a smaller model. The reduction therefore falls short of the nominal 4×, at 2.8 to 3.6× across the table, and is smallest for the smallest models. Qwen2.5-7B drops from 15.2 GB to 4.2–4.5 GB, and Phi-4-mini (3.8B) from 7.6 GB to 2.2–2.4 GB, which is what moves each of them across a hardware tier boundary. The exact figure for a given artifact is its file size, which should be read from the artifact rather than from this table.

Table 8. Model-weight memory footprint by precision (GB). FP16 is two bytes per parameter; the four-bit columns are estimates at 4.4–5.8 effective bits per parameter, including the group scales and higher-precision tensors each scheme retains, which is why the reduction (2.8–3.6×) is below the nominal 4×. Activation and KV-cache memory are additional and depend on context length and batch size.

Model	FP16	GPTQ-4bit	AWQ-4bit	Q4_K_M
Qwen2.5-7B	15.2	4.2	4.4	4.5
Phi-4-mini	7.6	2.2	2.3	2.4
Llama 3.2-3B	6.4	1.9	2.0	2.1
SmolLM2-1.7B	3.4	1.1	1.1	1.2
TinyLlama-1.1B	2.2	0.7	0.8	0.8

J Cohort and Hardware Reference Tables

Table 9 gives the architectural specifications of the 18-model cohort of Section 5; Tables 10 and 11 give the reference hardware of Sections 5 and 6.

Table 12 summarizes the four primary facets of the taxonomy of Section 3 with representative models.

Table 13 compares the deployment frameworks of Section 6.1.1.

Table 14 gives the secondary facets of the taxonomy.

K Platforms and Device Tiers in Detail

K.1 Apple Intelligence (AFM)

Apple's Foundation Models [16, 184] are the most fully documented on-device SLM deployment. The 2025 report [16] describes a ~3B on-device model, optimized for Apple silicon through KV-cache sharing (dividing the model so the second block reuses the first block's cache, a reported 37.5% memory saving) and 2-bit quantization-aware training, paired with a Parallel-Track Mixture-of-Experts server model on Private Cloud Compute. The on-device model is reported competitive with Qwen3-4B and Gemma-3-4B on its target tasks, and the server model with Llama 4 Scout (C). The third-generation family [18], five models built in collaboration with Google, adds a 20B-total on-device model that activates 1–4B parameters per token and keeps its full weights in flash, swapping experts into DRAM as needed; Private Cloud Compute was extended to NVIDIA GPUs in Google Cloud. This followed a January 2026 joint statement that Apple's next foundation models would be based on Gemini models and cloud

Table 9. Architectural specifications of reviewed models (as reported by each model's official release materials).

Model	Params	Layers	Hidden	Attention	Context	Vocab	Train Tokens
TinyLlama-1.1B	1.1B	22	2,048	GQA-32/4	2K	32K	3.0T
SmolLM2-1.7B	1.7B	24	2,048	MHA-32h	8K	49K	11.0T
StableLM 2-1.6B	1.6B	24	2,048	MHA-32h	4K	100K	2.0T
Qwen2.5-0.5B	0.5B	24	896	GQA-14/2	32K	152K	18.0T
Qwen2.5-1.5B	1.5B	28	1,536	GQA-12/2	32K	152K	18.0T
Qwen2.5-3B	3.0B	36	2,048	GQA-16/2	32K	152K	18.0T
Qwen2.5-7B	7.6B	28	3,584	GQA-28/4	128K	152K	18.0T
Llama 3.2-1B	1.2B	16	2,048	GQA-32/8	128K	128K	9.0T
Llama 3.2-3B	3.2B	28	3,072	GQA-24/8	128K	128K	9.0T
Gemma 2-2B	2.6B	26	2,304	GQA-8/4	8K	256K	2.0T
Gemma 2-9B	9.2B	42	3,584	GQA-16/8	8K	256K	8.0T
Gemma 3-1B	1.0B	26	1,152	GQA-4/1, LG	32K	262K	2.0T
Gemma 3-4B	4.3B	34	2,560	GQA-LG	128K	262K	4.0T
Mistral 7B	7.3B	32	4,096	GQA-32/8	8K	32K	n/a
Phi-3-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.3T
Phi-3.5-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.4T
Phi-4-mini	3.8B	32	3,072	GQA-24/8	128K	200K	5.0T
Phi-4	14.0B	40	5,120	GQA	16K	100K	~10T

Note: MHA = Multi-Head Attention; GQA = Grouped-Query Attention (Q/KV heads); GQA-LG = GQA with local–global interleaving. Every model in this cohort is dense, so total and active parameters coincide and a single parameter column suffices; no small-active-parameter MoE model (Section 1.2.2) is in the cohort, because none has public reporting complete enough to include.

On training-token counts. These are the least reliable column in the table, and their status differs by row. Marked *per-variant* where the source states a figure for that specific size: Gemma 2-2B (2.0T), Gemma 2-9B (8.0T), Gemma 3-1B (2.0T), Gemma 3-4B (4.0T), Phi-3-mini (3.3T), Phi-3.5-mini (3.4T), Phi-4-mini (5.0T), TinyLlama (3.0T), SmolLM2-1.7B (11.0T), StableLM 2 (2.0T). Marked *series-level* where the source reports one figure for the whole family and the individual sizes are not separately stated: Qwen2.5 (18.0T across 0.5B–7B) and Llama 3.2 (9.0T across 1B and 3B). Phi-4's ~10T is an approximation given as such by its source. Mistral 7B's training-token count is not disclosed. We report series-level figures in the size rows they cover rather than omitting them, but they should not be read as independent per-model measurements, and no claim in this paper relies on comparing a series-level figure against a per-variant one.

On context length. The value is the maximum the primary source documents, which for several models is reached through position interpolation at inference rather than trained natively: Qwen2.5-7B is natively 32K and extended to 128K, and Phi-3-mini's 128K is a separate long-context checkpoint from the 4K base.

technology [91]. Google's Gemini Nano, the on-device member of the Gemini family [3], occupies the same design point on Android.

K.2 Samsung Gauss

Samsung announced Gauss in November 2023 as a family of generative models in three parts, Gauss Language, Gauss Code, and Gauss Image, used first for internal productivity and planned for its products [185]. Samsung has published no parameter count, architecture, or on-device specification for it, so Gauss fails criterion M1, is not in the inventory, and we make no claim about its deployment. It is listed here because it is frequently cited as an on-device SLM, and the published record does not support that description.

K.3 General Mobile Deployment Considerations

SLMs targeting mobile deployment must navigate three constraints. (1) *Memory*: Consumer smartphones typically provide 6–12 GB RAM, of which the OS uses 2–4 GB, leaving 4–8 GB for applications. A 3B model at Q4 requires ~2 GB, comfortable for most flagship devices. (2) *Thermal*: Sustained inference on mobile NPUs is throttled by thermals; realistic sustained throughput is lower than peak throughput benchmarks suggest. (3) *Battery*: published per-device battery figures are scarce and not reported under a common protocol. The fundamental constraint in edge decoding is memory bandwidth rather than compute: flagship phones and mainstream laptops offer on the order of 50–100 GB/s (single-board computers well under 30 GB/s, unified-memory laptop designs considerably more) versus 2–3 TB/s for datacenter GPUs [147], making INT4 quantization not merely a storage optimization but a performance enabler for single-stream generation.

Table 10. Reference hardware configurations used to organize the deployment discussion of Section 6. They define the tiers against which reported figures are read; no per-model, per-configuration result is tabulated, because the sources do not report one consistently (Section 6.2).

Configuration	Hardware	RAM/VRAM	TDP	Use Case
Cloud GPU	NVIDIA A100 80GB	80 GB HBM2e	300W	Training + serving
High-end Consumer	NVIDIA RTX 4090	24 GB GDDR6X	450W	Local inference
CPU/iGPU Laptop	AMD Ryzen 7 7840HS	32 GB DDR5	35–54W	Laptop deployment
Edge GPU Module	NVIDIA Jetson Orin Nano	8 GB LPDDR5	7–15W (25W Super)	Edge AI / robotics
Edge SBC	Raspberry Pi 5	8 GB LPDDR4X	12W	Embedded IoT
Mobile SoC	Snapdragon 8 Gen 3	12 GB LPDDR5X	~8W sustained	Smartphone

Table 11. Detailed specifications of the devices referenced in this section.

Specification	Raspberry Pi 5	Jetson Orin Nano	Galaxy S24+	Laptop
CPU	BCM2712 4×A76 @ 2.4GHz	6×A78AE @ 1.5GHz	Snapdragon 8 Gen 3	Ryzen 7 7840HS
GPU	VideoCore VII	1024 CUDA (Ampere)	Adreno 750	Radeon 780M
NPU	None	None	Hexagon	Ryzen AI
RAM	8 GB LPDDR4X	8 GB LPDDR5 (unified)	12 GB LPDDR5X	32 GB DDR5-5600
Memory bandwidth	~17 GB/s	68 GB/s	76.8 GB/s	89.6 GB/s
Storage	microSD / NVMe	NVMe	UFS 4.0	NVMe SSD
TDP	12W	7–15W (25W Super)	~8W sustained	35–54W
Inference engine	llama.cpp	MLC LLM	ExecuTorch	llama.cpp

K.4 Edge and IoT Deployment

Below smartphone-class hardware lies a diverse ecosystem of edge devices: Raspberry Pi single-board computers, NVIDIA Jetson modules, and microcontrollers. SLMs targeting this tier typically require:

Sub-1B or extreme quantization
: A Raspberry Pi 5 (8 GB RAM, Cortex-A76 CPU, about 17 GB/s of memory bandwidth) can hold 4-bit models up to a few billion parameters, and its decode rate is bounded by that bandwidth. A benchmark of runtimes on single-board computers reports that llamafile reaches up to 4× the throughput of Ollama at 30–40% lower power for equivalent models [143] (B).

Jetson Orin
: NVIDIA's Jetson Orin modules, with TensorRT-LLM [183] among their runtimes, hold quantized 3–7B models in 8–64 GB of unified memory, with the larger-memory NX and AGX parts suited to the upper end of that range, for industrial and robotics use.

Microcontrollers (MCU)
: ARM Cortex-M class microcontrollers carry roughly 512 KB–2 MB of SRAM. Even a 10M-parameter model needs about 5 MB at 4 bits, and a 100M-parameter one about 50 MB, so a language model of the size this survey covers cannot be resident in microcontroller SRAM, and would at best stream weights from external memory at a large cost in speed. Keyword detection and sensor analytics on these parts are served by task-specific networks far smaller than any model in the inventory; we found no documented SLM deployment at this tier, and it is outside the scope of this paper.

Table 11 details the specifications of the four representative devices referenced throughout this section, from the Raspberry Pi 5 up to a laptop-class CPU.

L Conversion Pipeline and Barriers to Adoption

L.1 Converting an LLM Agent to an SLM Agent

For teams with an existing LLM-only agent, Belcak, Heinrich, Diao, et al. [19] propose a six-step conversion pipeline, which we summarize here because it operationalizes the position; the steps are theirs. **S1, instrumentation:** log every non-interface model call (inputs, outputs, latency) from the running agent, with encryption and access control. **S2, curation:** strip personally identifiable and protected health information and application secrets from the logs. **S3, clustering:** apply unsupervised clustering to the curated calls to surface the recurring request types, which are the specialization targets. **S4, selection:** for each cluster, choose a candidate SLM by matching required capability against benchmarks, license, and deployment constraints. **S5, specialization:** fine-tune the chosen SLM

Table 12. Summary of the four primary facets of the SLM taxonomy with representative models. Assignment is multi-label: a model may carry several categories on one facet, and the same model may therefore appear more than once in a block.

Facet	Category	Key Characteristics	Representative Models
Architecture	Dense Transformer	All params active per token; GQA, RoPE, SwiGLU	Phi-4-mini, Qwen3, SmolLM3, Gemma 3
	MoE	Sparse activation; total ≫ active params	Granite 4.0 Tiny (SLM); Mixtral 8×7B, Qwen3-30B-A3B (small-active MoE)
	Hybrid SSM-Transformer	Linear-time; interleaved SSM/attention blocks	Falcon-H1, Granite 4.0, Nemotron Nano 2
Training Strategy	From scratch	Curated/synthetic data; full pre-training	Phi, Qwen2.5/Qwen3, SmolLM3
	Distillation	Knowledge transfer from larger teacher	Gemma 3/4, DeepSeek-R1-Distill, Phi-4-mini-reasoning
	Pruning-based	Parameter removal from large model + distillation	Minitron, Sheared-LLaMA
Modality	Text-only	Tokenized text I/O; majority of production SLMs	Mistral 7B, SmolLM3, Qwen3
	Vision-Language	Image + text; CLIP/SigLIP encoder added	Phi-3-vision, Gemma 3 (4B and above), PaliGemma
	Audio/Multimodal	Text + vision + audio in one sub-10B model	Gemma 3n, Gemma 4, Apple AFM
Domain	General-purpose	Multi-domain training; evaluated on broad benchmarks	Phi, Gemma, Llama, Qwen, Mistral
	Healthcare	Clinical NLP; on-premises for data residency	Meerkat 7B, OptimCLM
	Code	Code corpora; FIM training; HumanEval/MBPP focus	Code Llama, StarCoder2, DeepSeek-Coder
	Legal/Multilingual	Domain corpora; regulatory and cross-lingual tasks	Aya 23 8B, Qwen2.5 multilingual

Table 13. Key SLM deployment frameworks, their primary targets, and characteristics. Throughput is not tabulated, because project-reported figures are measured on different hardware and settings and are not comparable across rows (Section 6.2).

Framework	Primary Target	Quantization	Notes
llama.cpp [38]	CPU, Metal, CUDA, ROCm	GGUF (Q2–Q8)	Widely used reference runtime; minimal dependencies; ecosystem includes Ollama, llamafile
ONNX Runtime [144]	GPU, CPU, NPU, WASM	INT4/INT8	Cross-platform; DirectML, CoreML, CUDA backends
MLX [182]	Apple Silicon	4-bit	Unified memory; no host–device weight transfers
TensorRT-LLM [183]	NVIDIA GPUs	INT4/INT8/FP16	Continuous batching; kernel fusion
ExecuTorch [39]	Mobile (iOS, Android)	INT4	Small base runtime; CoreML, QNN, ARM, MediaTek, Vulkan backends; used for Meta's Llama 3.2 on-device measurements
MLC-LLM [40]	iOS, Android, web, embedded GPU	4-bit	TVM compilation; WebGPU support
WebLLM [145]	Web browsers	4-bit WASM	WebGPU acceleration; OpenAI-compatible API

on the cluster's curated data, typically with LoRA or QLoRA. **S6, iteration:** re-run the loop periodically as the traffic distribution drifts. Two additions follow from H. Step S3 is where the conditions are assessed: each cluster's output schema (H1), input length (H3), and the availability of validity and correctness checks (H4a, H4b) can be checked by inspection, and its volume can be compared with a pilot estimate of the break-even N^* (H2), recorded before the comparison; clusters that fail are left with the generalist. And each converted cluster should be evaluated on cost per completed task against the unconverted agent, which is the measurement the literature lacks.

L.2 Barriers to Adoption

Belcak, Heinrich, Diao, et al. [19] identify three reasons the shift has lagged the technical case. First, large centralized-inference investments create sunk-cost inertia and an operational bias toward one big model behind one API (B1). Second, SLMs are still evaluated on generalist leaderboards (MMLU, broad chat benchmarks) that are misaligned with agentic utility, so their fitness for delegated sub-tasks is under-measured (B2). Section 5 shows that the generalist record is also too inconsistent across vendors to support cross-model comparison, so it cannot be used to choose among small models even for general capability. Third, SLM releases receive far less attention than frontier launches, so awareness lags capability (B3). The first and third are not technical obstacles. The second is different, and Section 7.3 shows it to be partly a real evidential gap rather than only a misdirected emphasis: agent-specific evaluation exists and is maturing [55, 96, 114–116], and developer reports support per-call competence for specialized small models, but independent multi-step evaluations of current sub-10B models are sparse, and no study compares cost per completed task for small-model-first and large-model agents. Until that changes, the strength of the case for hypothesis H will remain uneven in the way Table 5 records.

L.3 Learned Routing and Cascades

Three designs recur. *Cascades* try the cheap model first and escalate on a quality signal: FrugalGPT [43] learns which sequence of LLM APIs to query, and AutoMix [45] escalates when the small model's own few-shot self-verification judges its answer unreliable, which is the pattern H prescribes when the checker of H4 is the model itself. *Predictive routers* decide before generation: Hybrid LLM [44] routes by predicted query difficulty with a quality level tunable at test time, and RouteLLM [46] trains a lightweight router on human-preference data to predict whether a weak

Table 14. Secondary facets. These do not describe what a model is but whether it can be adopted, and they are the facets practitioners report finding hardest to recover from model cards. Access and license are recorded per variant in the released inventory, though for 96 of the 209 records the license is given only as a pointer to the source; the other facets are assigned in this paper to representative models.

Facet	Values	Why it gates adoption
Licensing	Permissive (Apache-2.0, MIT); custom vendor licence with use restrictions; research-only; unspecified; proprietary/API-only	Determines commercial use, redistribution of fine-tunes, and whether an on-premises deployment is legal at all
Data transparency	Full (corpus released); partial (composition described, data not released); disclosed sources only; undisclosed	Determines whether contamination, licensing of training data, and domain fit can be assessed
Safety alignment	Preference-optimized with a published safety evaluation; instruction-tuned without one; base model only	A base checkpoint wired into an agent with tool access has no refusal behavior to rely on
Reasoning mode	None; always-on extended reasoning; controllable budget; separate reasoning checkpoint	Determines token cost per call and whether latency is predictable, which is decisive inside an agent loop
Deployment target	Documented inference engine(s) and hardware class the vendor ships an artifact for	An unquantized checkpoint with no GGUF, ONNX, or ExecuTorch export is not an edge model whatever its size

model's answer would be judged as good as a strong model's, reporting transfer of the trained router to unseen strong/weak model pairs. *Decomposing routers* push the decision below the query level: Division-of-Thoughts [48] decomposes a task into sub-questions and assigns each to a local SLM or a cloud LLM according to estimated difficulty, for on-device agents. The open problems are difficulty estimation without an oracle, routing objectives that jointly weigh accuracy, latency, privacy sensitivity, and network availability, and handoff that does not produce a visible seam in output quality [186].

A different form of SLM–LLM collaboration leaves the large model in charge of every output and uses the small one only to make it faster. In speculative decoding a small draft model proposes several tokens and the large model verifies them in one forward pass, accepting a prefix, so that the output distribution is that of the large model [49, 50]; Medusa [117] and EAGLE [118] replace the separate draft model with lightweight heads on the large model itself. This is not delegation in the sense of H, since no call's answer comes from the small model, but it is the most widely deployed way in which small models already reduce the cost of large-model inference, and it is complementary to routing: an escalated call can itself be served speculatively.

M Domain-Specific Applications in Detail

This section expands the summary of Section 8.3 of the main text.

M.1 Healthcare and Clinical NLP

Healthcare natural language processing (NLP) presents demanding requirements: high factual accuracy, interpretability, compatibility with clinical workflows, and operation within a regulatory regime that constrains where identifiable patient data may be processed. That regime is the pull toward on-premises models, and it needs stating precisely, because it is frequently overstated.

On-premises deployment is not compliance. Neither HIPAA nor the GDPR is satisfied by the location of a model's weights. HIPAA compliance is a property of a covered entity's administrative, physical, and technical safeguards taken together: access control, audit logging, integrity controls, transmission security, workforce training, breach notification, and business-associate agreements where a third party is involved. The GDPR requires a lawful basis for processing, purpose limitation, data minimization, a data protection impact assessment for high-risk processing, and, for automated decisions with legal or similarly significant effects, the safeguards of Article 22. A locally hosted model can fail every one of these. What on-premises deployment does is remove one specific difficulty, the transfer of identifiable data to a processor outside the entity's control, which under HIPAA otherwise requires a business-associate agreement and under the GDPR raises transfer and sub-processor questions. That is a real and often decisive simplification, and it is the reason the pull exists. It is not a compliance property of the model, and no claim in this paper should be read as asserting one. The same scope conditions as Section 6.1 apply: an on-premises model whose application logs prompts to a cloud service has reintroduced the transfer it was chosen to avoid.

The Meerkat family [119] demonstrates the potential of domain-specific SLM fine-tuning. Built on a 7B base with instruction fine-tuning on curated clinical QA data and chain-of-thought traces synthesized from medical textbooks, Meerkat-7B reports 70.6% on MedQA (USMLE-style), which its authors describe as the first 7B result above the

USMLE passing threshold, ahead of earlier open medical models and GPT-3.5 across six medical benchmarks though below GPT-4 (C). The larger Meerkat-70B is reported to exceed GPT-4 by 1.3 points on average and to diagnose 21 of 38 complex clinical cases, against 21.8 for GPT-4 and 13.8 for human respondents (C). Meerkat-7B runs on a single GPU; Meerkat-70B needs about 140 GB at 16-bit precision.

OptimCLM [76] distils an ensemble of domain-adapted clinical BERT encoders into the smaller BERT-PKD and TinyBERT, then applies unstructured pruning and 8-bit quantization, reporting up to 22.88× compression with under 5% loss in macro-AUROC on length-of-stay, mortality, diagnosis, and procedure prediction from MIMIC-III admission notes. It shows that the compression stages compose for clinical classifiers that can run on hospital workstations; it is not evidence about generative SLMs.

The broader SLM healthcare ecosystem includes domain-pretrained biomedical models in the 2–3B range, clinical note generation models, and radiology-specific models. A key trend is the use of retrieval-augmented generation (RAG) [127] to anchor SLM outputs to verified clinical knowledge bases, reducing hallucination rates in patient-facing applications.

M.1.1 The State of the Evidence in This Domain

Healthcare is the best-evidenced of the verticals surveyed here, and it is still weaker evidence than it appears. Three qualifications apply to everything above.

Licensing-exam benchmarks are not deployment evidence. MedQA and its siblings are multiple-choice examinations designed to test candidate physicians, and performance on them establishes that a model can answer exam questions. It does not establish accuracy on the tasks clinical systems actually perform: coding an encounter, drafting a discharge summary, extracting a medication list from a note, or flagging a contraindication. The Meerkat results are grade-C, author-reported on public benchmarks with a documented method, which is a good standard for the model literature and not the standard a clinical deployment decision requires.

Independent prospective evaluation is largely absent. We found no prospective study evaluating a sub-10B model in a live clinical workflow with outcome measurement, and no head-to-head comparison of an on-premises small model against a cloud frontier model on the same institution's data under the same protocol. Retrospective benchmark performance and deployment performance are known to diverge in clinical NLP, and nothing in the current SLM literature closes that gap.

Compression results are single-study. The OptimCLM finding [76] that compressed clinical encoders lose under 5% in macro-AUROC on outcome prediction is one pipeline, on one dataset, for classification rather than generation. It has not been independently reproduced, and no comparable study exists for a compressed generative SLM on clinical tasks such as coding an encounter or drafting a discharge summary.

The honest summary is that healthcare provides the strongest published case for domain-specific small models and that the case rests on exam benchmarks and single studies rather than on deployment evidence. Establishing per-task clinical benchmarks with independent evaluation, and reporting per-group performance floors of the kind Section N.4 argues for, is an open need in this vertical specifically.

M.2 Legal NLP

Legal document processing, which covers contract review, compliance monitoring, regulatory change analysis, and case law search, involves long and precisely structured documents where hallucination is particularly dangerous. SLMs address legal NLP deployment constraints in two ways: (1) by enabling on-premises deployment to satisfy attorney-client privilege and data security requirements; and (2) by enabling cost-effective processing of the massive volumes of routine legal work that would be prohibitively expensive with frontier LLMs.

The argument for domain-adapted SLMs (3B–7B, fine-tuned on contract corpora) in this vertical is structural rather than empirical: contract review, compliance triage, and regulatory monitoring consist of bounded, highly repetitive classification and extraction operations over documents that frequently cannot leave the client's infrastructure, which is precisely the profile the conversion pipeline of Section L.1 targets. Narrow fine-tuning on a specific regulatory regime concentrates in-domain knowledge that a generalist model must retrieve or infer.

We state the evidential position for this vertical explicitly, because it is weaker than for the other three and it would be easy to read the structural argument above as though it were a result. Public legal benchmarks exist: LegalBench [120] collects 162 legal-reasoning tasks, LexGLUE [121] groups legal language-understanding tasks, and CUAD [122] provides expert-annotated contract clauses. What is missing is their use for the current generation of small decoder models: we found no evaluation of 2024–2026 sub-10B models across them under a common protocol, and deployment comparisons rest on vendor pilot reports that are not independently verifiable, often because the corpora are client-confidential. Two structural features of the domain make this worse: an error in contract or compliance work can be both consequential and hard to detect, which is exactly the condition under which hypothesis H's verifiability requirement (Section 1.3) fails; and long, cross-referential documents strain the

short-context condition. Legal work is therefore a domain where the economic case for small models is strong and the capability evidence is absent, and we present the section as such rather than as a claim of parity. Establishing open legal-domain benchmarks with independent evaluation is a prerequisite for saying anything stronger.

M.3 Code Generation

Code-focused SLMs have matured into a category distinct from general SLMs, with specialized pre-training corpora (GitHub, Stack Overflow, documentation), code-aware tokenization, and evaluation on function-level benchmarks (HumanEval, MBPP, SWE-Bench).

Code Llama [123]
Meta's family of code models fine-tuned on Llama 2 (7B, 13B, 34B). Code Llama 7B achieves 33.5% HumanEval pass@1.

StarCoder2 [124]
BigCode's 3B/7B/15B models trained on The Stack v2 (600+ programming languages). StarCoder2-7B reports 35.4% HumanEval pass@1, and StarCoder2-15B 46.3%.

DeepSeek-Coder [125]
A 1.3B–33B family trained from scratch on 2T tokens. The 6.7B base model reports 49.4% on Python HumanEval, and its instruction-tuned version 78.6%, above the 76.2% the same report gives for GPT-3.5-Turbo.

Codestral [187]
Mistral's 22B code model trained on 80+ programming languages with fill-in-the-middle (FIM) training for IDE integration. At 22B dense it is just outside the C1/C2 bound and is not in the inventory; it is listed as the upper reference for this category.

M.4 Education and Multilingual NLP

Multilingual SLMs address the linguistic coverage gap in AI assistants: the majority of LLM pre-training data is English, leaving speakers of lower-resource languages with markedly worse performance. Aya 23 [89] allocates the capacity of an 8B decoder to 23 languages and reports outperforming its predecessor Aya-101 [158], a 13B mT5-based encoder-decoder covering 101 languages (outside C1), on the languages both cover (C); the pair illustrates the breadth-versus-depth trade-off at this size. Qwen2.5 [12] and Gemma 3 [31] document large non-English pretraining mixtures and report multilingual evaluations (C). For educational use, which includes on-device tutoring and low-connectivity tools, the requirements beyond language coverage are low hallucination rates and age-appropriate safety filtering, neither of which is reported comparably across small models.

N Model-Level Challenges in Detail

N.1 Hallucination Mitigation in Small Models

Hallucination, the confident generation of factually incorrect content, is an intrinsic failure mode of autoregressive language models, and it is more pronounced in smaller models [126]. The core issue is information capacity: a 3B model's parameters can encode far fewer facts than a 175B model, and when queried about knowledge at the boundary of its training distribution, small models are more likely to confabulate plausible-sounding but incorrect responses.

Mitigation strategies include: (1) *RAG* [127], which anchors SLM generation to retrieved documents rather than parametric memory; (2) *uncertainty quantification* through sampling and ensemble methods; (3) *RLHF with truthfulness rewards* to penalize confident incorrect responses; and (4) *calibration post-processing* to align model confidence to actual accuracy.

N.2 Reasoning Limitations Under Parameter Constraints

Language-modeling loss falls as a power law in parameter count [5], and in the reported results reviewed here the gap between small and large models is widest on multi-step reasoning, particularly in mathematics, formal logic, and scientific inference (D). At the 1B–7B scale, SLMs struggle with problems that require holding many intermediate states simultaneously or with deep deductive chains.

Emerging directions include: (1) *Test-time scaling*: allocating additional inference compute (more generation tokens, repeated sampling, tree-of-thought search) to substitute for model capacity; the DeepSeek-R1 authors [79] report that distilling the long chains of thought of their large reasoning model into 1.5B–70B students transfers much of its mathematical-reasoning ability (C). (2) *Neuro-symbolic integration*: augmenting SLMs with symbolic solvers (SAT solvers, theorem provers, calculators) to handle formal reasoning externally. (3) *Speculative decoding*

[49, 50] using a small draft SLM to propose tokens verified by a larger model, which speeds up the larger model rather than substituting for it (Section L.3).

N.3 Hardware Co-Design and NPU Optimization

The efficiency of SLM inference depends as much on hardware as on model architecture. Modern consumer hardware increasingly includes dedicated AI accelerators: Apple ANE, Qualcomm Hexagon NPU [188], Samsung Exynos NPU, and Intel NPU. These accelerators are optimized for specific operation types (INT4/INT8 matrix multiplication) and data layouts.

A critical open problem is *hardware-model co-design*: developing SLM architectures whose attention patterns, FFN widths, and layer counts align with the data bus widths and tile sizes of target NPUs. Future SLMs trained with hardware constraints as first-class objectives, rather than retrofitting quantization after the fact, will likely yield significant efficiency improvements.

N.4 Ethical Considerations: Bias Amplification in Compressed Models

Model compression introduces an underexplored ethical risk: *bias amplification*. Pruning and quantization may disproportionately affect model performance on underrepresented demographic groups, dialects, and languages, as the model's remaining capacity may preferentially serve the majority distributions present in training data [77]. Zhou, Kurz, and Zhao [74] find that quantization preserves multilingual perplexity better than pruning, suggesting that pruning disproportionately degrades non-English performance.

Research priorities include: (1) *fairness-aware compression* that includes demographic parity metrics as part of the compression objective; (2) *representation auditing* of compressed models across language, dialect, gender, and race; and (3) *minimum performance floors* for safety-critical applications, ensuring compressed models meet baseline accuracy thresholds for all user populations.

N.5 Sustainable AI: SLMs as a Green AI Alternative

The energy consumption of training and serving frontier LLMs has become a significant environmental and economic concern [189, 190]. Training GPT-3 emitted an estimated 552 metric tons of CO_2 equivalent [7], and training a model two orders of magnitude smaller from scratch needs correspondingly less compute. The defensible claim about inference is conditional: *for a task a small model completes correctly, on hardware it keeps busy, serving it with the small model uses less inference energy* (D). Whether a shift to small models reduces total system energy depends on teacher-training energy, the compression pipeline, device embodied carbon, regional grid intensity, update distribution, utilization, and the rebound effect, and no published accounting answers it. Appendix P sets out the accounting boundary.

N.6 Democratization and Sustainability

Running a 3B model with strong reported grade-school-mathematics accuracy on an inexpensive single-board computer is a qualitative change in access. Organizations facing cloud-API cost, data-sovereignty rules, or unreliable connectivity can run capable NLP locally with no per-query charge and, subject to the scope conditions of Section 6.1, without transferring prompt content to a processor outside their control. The environmental side of the argument is conditional in the way set out above and in Appendix P: we make a per-task inference claim and no aggregate one.

O Open Research Directions

Beyond direct empirical verification, several promising research directions emerge from this review:

Multilingual SLM evaluation
: Extending the consolidated performance evidence to non-English benchmarks would reveal whether the English-centric performance hierarchy reported in the literature holds across languages, particularly for models with multilingual training data (Qwen2.5, Gemma 3, BLOOM).

On-device continual learning
: Developing methods for incremental adaptation of deployed SLMs to new data without full retraining would enable personalization and domain adaptation at the edge.

SLM ensembles vs. single LLM
: Investigating whether ensembles of specialized SLMs (one for math, one for code, one for general knowledge) with a lightweight router can match single-LLM performance at lower aggregate cost.

Multimodal SLMs
Evaluating vision-language SLMs such as Phi-3-vision, Gemma 3 with vision, and Gemma 3n [156] on multimodal benchmarks would extend the comparison to the rapidly growing multimodal inference domain.

Hybrid architectures
The emergence of non-transformer SLM architectures (SambaY [191], Gated Delta Networks [192]) suggests that the transformer's dominance may not extend to the edge, where linear-time inference is particularly valuable.

Federated fine-tuning
Developing privacy-preserving fine-tuning protocols that aggregate gradient updates across edge devices would enable collaborative model improvement without centralized data collection.

P What the Environmental Claim Covers

Environmental claims for small models are routinely made at a level of generality the accounting does not support. A statement that many small-model instances on consumer hardware use less total energy than one multi-GPU server, for example, cannot be sourced or derived: its truth depends on utilization on both sides, and a server at high batch utilization is considerably more energy-efficient per token than a consumer device performing occasional single-stream inference.

A complete comparison of a small-model deployment against a large-model one has to include the training energy of the small model; the training energy of any teacher used to distill it, which can dominate and which vendors do not report; the energy of the distillation, quantization, or pruning pipeline; the embodied carbon of the devices inference runs on, amortized over their service life and the share of it spent on inference; the marginal grid intensity where each side runs, which differs by more than a factor of ten between regions; and the energy of model updates, which for on-device models means redistributing weights to every device.

Almost none of these terms is reported. The available measurements are per-query and partial: a 2023 estimate put a frontier query at about ten times the energy of a web search [8], while a 2025 production measurement covering accelerators, host systems, idle capacity, and data-centre overhead reports 0.24 Wh for the median text prompt of one large assistant [9]. Per-token compute scales with active parameters, so for a query both can answer a small model performs one to two orders of magnitude less compute than a frontier model; whether that becomes an energy saving depends on the utilization of the hardware each runs on (D). The defensible claim is therefore conditional: *for a task a small model completes correctly, on hardware it keeps busy, serving it with the small model uses less inference energy.* Whether a shift to small models reduces total system energy is a question no published accounting answers, and the rebound effect, that cheaper inference induces more of it, cuts against it.

Q Data Provenance and Supplementary Protocols

This appendix supports the reproducibility of the *review itself.* It records where the quantitative material in Sections 5 and 6 comes from, and documents the evaluation and compression conventions the reviewed sources report, so that a reader can judge any individual figure against the conditions behind it. Device specifications are in Table 11. The appendix reports no original experimental runs.

Q.1 Provenance of the Consolidated Figures

Provenance is recorded at two levels. *Model-level* provenance fixes, for each of the 18 models in the cohort, the primary release document from which its architectural specification and the vendor-reported scores quoted in Section 5.2 derive (Table 15). *Source-class* provenance records, for each table retained in the paper, the category of source its contents are drawn from (Section Q.2). Benchmark definitions and metric variants follow the originating task papers cited in Section I.1.

Q.2 Provenance Status by Table

Every quantitative table retained in this paper falls into one of three categories, and the category determines how much weight it can carry.

Sourced from primary release documents. The architectural specifications in Table 9 are taken from the release document listed for each model in Table 15. Layer counts, hidden sizes, attention configurations, context lengths, vocabulary sizes and training-token counts are stated directly in those documents and are stable across printings.

Computed from stated quantities. The weight footprints in Table 8 follow arithmetically from parameter count and an effective number of bits per weight (two bytes per parameter for FP16, 4.4–5.8 effective bits for the four-bit schemes, as stated in its caption) and are estimates, not measurements. The device specifications in Table 10 and Table 11 come from manufacturer datasheets.

Table 15. Model-level provenance for the reviewed cohort: the primary release document from which each model's architectural specification and headline reported scores derive.

Model(s)	Developer	Primary source	Source type
TinyLlama-1.1B	StatNLP / SUTD	Zhang, Zeng, Wang, and Lu [135]	Technical report
SmolLM2-1.7B	Hugging Face	Allal, Lozhkov, Bakouch, et al. [59]	Technical report
StableLM 2-1.6B	Stability AI	Stability AI [139]	Technical report
Qwen2.5-0.5B/1.5B/3B/7B	Alibaba	Qwen Team, Alibaba Group [12]	Technical report
Llama 3.2-1B/3B	Meta	Llama Team, AI @ Meta [4]	Technical report
Gemma 2-2B/9B	Google DeepMind	Gemma Team, Google DeepMind [133]	Technical report
Gemma 3-1B/4B	Google DeepMind	Gemma Team, Google DeepMind [31]	Technical report
Mistral 7B	Mistral AI	Jiang, Sablayrolles, Mensch, et al. [136]	Technical report
Phi-3-mini	Microsoft	Abdin, Aneja, Awadalla, et al. [10]	Technical report
Phi-3.5-mini	Microsoft	Microsoft [129]	Technical report
Phi-4-mini	Microsoft	Microsoft, Abouelenin, Ashfaq, et al. [131]	Technical report
Phi-4	Microsoft	Abdin, Aneja, Behl, et al. [130]	Technical report

Structural properties of a technique. Table 2 states what each compression technique reduces by construction. Its entries are consequences of how the technique works, not measurements: the memory factor of n-bit quantization follows from the bit-width, and the parameter factor of s-fraction pruning follows from the sparsity level. Where a cell reports a figure from a specific study, that study is cited in the cell.

Inventory-derived. The counts in the text, Table 6, the inventory appendix, and every generated figure are computed from version 3.6 of the released inventory by the released scripts.

Transcribed coverage. The coverage ledger (Table 7) records whether, and under which stated protocol, each model's primary release document or official model card reports each benchmark; the document and table for every cell are listed in `audit_coverage_ledger.csv`. For the Phi models these are the official model cards [87], and for Llama 3.2 the official model card [88].

Qualitative synthesis of the reviewed literature. The quality-cost statements in Table 3, the framework comparison in Table 13, the survey comparison in Table 1, the taxonomy summary in Table 12, and the agent-evidence summary in Table 5 summarize the works cited in their respective sections. They are positional rather than metric, and each row of the two graded tables carries the class of evidence standing behind it (Section 5.1.2).

Not reported by design. No table in this paper gives a single compression ratio or accuracy-retention percentage per technique, for the reasons in Section 4.1, and no table reports a cross-model accuracy comparison, because, as Section 5.2 and Table 7 show, such a table cannot be assembled from vendor reporting without mixing incompatible protocols and separately reported checkpoints. A partial table from single-harness third-party evaluations is the defensible alternative.

Q.3 Prompt Formats in the Source Literature

This subsection describes the conventions the reviewed sources use; it does not describe a protocol applied in this review, which ran no evaluations. The sources commonly preserve each model's own instruction wrapper while following a broadly similar task-payload structure. **Multiple-choice question answering** presents the question followed by the enumerated options (A–D) and an "Answer:" cue, with the model expected to emit a single option letter. **Mathematical reasoning** prepends an instruction to solve the problem step by step and appends a "Let's think step by step" cue, so that a chain-of-thought precedes the final numeric answer. **Extractive question answering** supplies the context passage, then the question, then an instruction to answer from the context and to respond "unanswerable" when the context does not contain the answer. **Abstractive summarization** supplies the source article preceded by an instruction to produce a two- to three-sentence summary. Where a source reports few-shot results, it typically prepends a fixed set of exemplars of its own choosing, which differs from source to source. Because sources differ in the exemplars, in the exact wording of these wrappers, and in answer-extraction heuristics, figures from different sources are not comparable, which is part of the finding of Section 5.2.

Q.4 Quantization and Compression Pipeline

The compression workflows reported across the reviewed sources are summarized here. The reviewed literature reports five recurring precision or compression settings:

FP16 baseline
: Standard half-precision inference used as the reference condition.

GGUF Q4_K_M
: Mixed-precision 4-bit quantization via `llama.cpp`, assigning higher precision to the most sensitive tensors.

AWQ 4-bit
: Activation-aware weight quantization with group size 128 and calibration on held-out text sequences.

GPTQ 4-bit
: Layer-wise post-training quantization using second-order curvature approximations.

bitsandbytes NF4
: 4-bit NormalFloat quantization with double quantization, matching the QLoRA training regime.

Some sources also report SparseGPT [73]-induced 50% unstructured sparsity applied before 4-bit quantization. We do not tabulate those figures, because support for sparse inference remains hardware dependent and the setting is not uniformly reported across the reviewed sources.

Q.5 Human-Evaluation Conventions in the Source Literature

For subjective tasks such as summarization quality and instruction following, the reviewed sources that report human judgments typically rate outputs along three axes (**relevance**, **coherence**, and **factual accuracy**) on a 1–5 Likert scale, with model identity hidden from annotators and inter-annotator reliability reported via a Krippendorff-style coefficient. MT-Bench-style results substitute a strong LLM judge for human raters. Because these protocols differ across sources in rubric wording and judge model, and because per-model coverage is sparse, instruction-following and summarization are not tabulated in this review and are discussed only qualitatively.

R Broader Small-Model Ecosystem Context

This appendix situates the reviewed SLM cohort within the wider model ecosystem, which matters because the 2025–2026 release cycle moved compact open models, open-weight reasoning models, and frontier API systems in parallel rather than in sequence. The summary below records the ecosystem context needed to interpret the cohort, followed by the full generated variant inventory.

R.1 Landscape Snapshot

Three ecosystem trends are especially relevant to the present study:

Capability compression
: Reasoning, long-context handling, and multimodality have moved into the 1B–8B range, narrowing the gap between deployable SLMs and cloud-scale LLMs.

Open-weight expansion
: Families such as Qwen, Phi, Gemma, SmolLM, DeepSeek distills, Granite, and MiniCPM have expanded the design space for self-hosted deployment (Figure 16).

Deployment diversification
: Inference targets now span cloud GPUs, consumer GPUs, laptops, smartphones, browsers, and embedded edge boards, making model choice inseparable from hardware choice.

The small-active-parameter turn
: The most distinctive feature of the mid-2026 wave is how much of it activates 3–13B parameters against totals of 30B to 284B: Nemotron 3.5 Lightning (30B/3B), Hy-MT2-30B-A3B, North Mini Code (30B/3B), Laguna XS 2.1 (33B/3B), Ling 3.0 Flash (124B/5.1B) and its domain variants, and DeepSeek V4 Flash (284B/13B). These are the models that make the C1/C2 distinction load-bearing rather than pedantic, since each is cheap per token and none is resident on a phone.

The May–September 2026 additions to the inventory illustrate both ends of the range. At the small end, IBM shipped Granite 4.1 and then Granite 4.2 at 3B and 8B with a thinking switch, Liquid AI extended LFM2.5 from 230M to an 8.3B/1.5B on-device MoE, and Tencent released the Hy-MT2 translation family at 1.8B and 7B. Frontier API releases continued in parallel over the same months; they are recorded, with their sources, in the LLM part of the released inventory, and none of them bears on a claim in this paper.

Figure 14 gives a size-proportional overview of the inventory by developer and family, and Figure 15 tracks the entry of new developers into the open-SLM space.

R.2 Representative Family Reference

Table 16 lists the representative families tracked in the inventory, with their developers, size ranges, access, and the role each plays in the landscape.

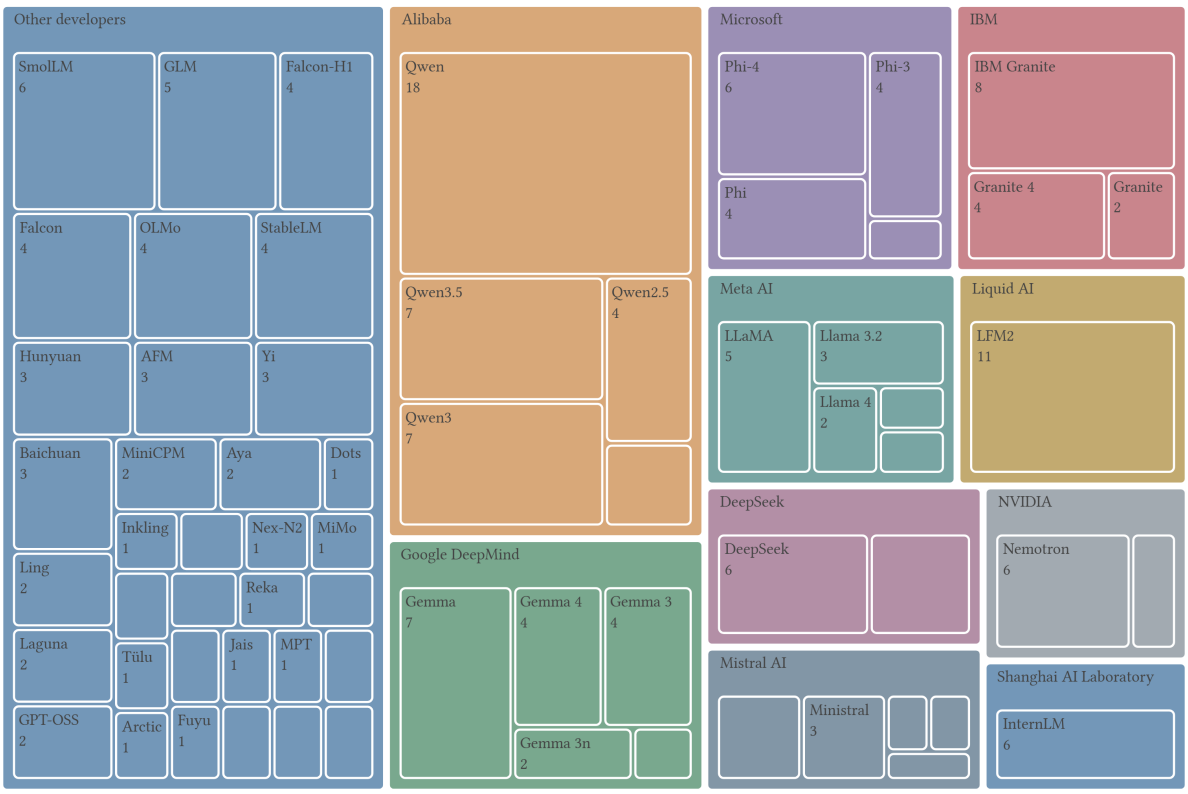


Figure 14. Documented SLM variants grouped by developer and family; area is proportional to the number of variants. Generated from `merged_llm_database.json` (version 3.6; releases through 4 September 2026).

R.3 Classification Conventions

The inventory applies conditions C1 and C2 of Section 1.2.1 from the parameter counts each record carries:

- A record enters the inventory when its *active* parameter count is at or below roughly 20B (C1); dense models, for which active equals total, therefore enter at or below 20B total. C2 (total at or below 20B) then separates SLMs from small-active-parameter MoEs; both are listed, and only the former carry the paper's storage, resident-memory, and on-device claims.
- Where a source states no active-parameter count for a model it describes as a mixture of experts, the record is marked undisclosed rather than having active set equal to total. Nested legacy records are read from their active field.
- Per-layer-embedding "effective" sizes (Gemma 3n, Gemma 4 E2B/E4B) are not sparse activation; C1 is evaluated on the total and the effective size is kept as a note.
- M1 is satisfied when a developer source states size and architecture; the source check of Appendix B.2 records which source that is. Records of undisclosed size, whatever their marketing, are kept in the comparison part of the file as `undisclosed`.
- Encoder-decoder models, derivative fine-tunes without a technical report, and API aliases are excluded by named editorial decision and kept under `models.excluded` with their reason.

The count history from 126 to the present 209 is given in Appendix B.3, and the released changelog names every record added, moved, or removed at each step.

R.4 Generated Variant Inventory

The following inventory is generated directly from `merged_llm_database.json` using `scripts/generate_model_tables.py`. Version 3.6 of the database covers 209 inventory records (179 SLMs and 30 small-active-

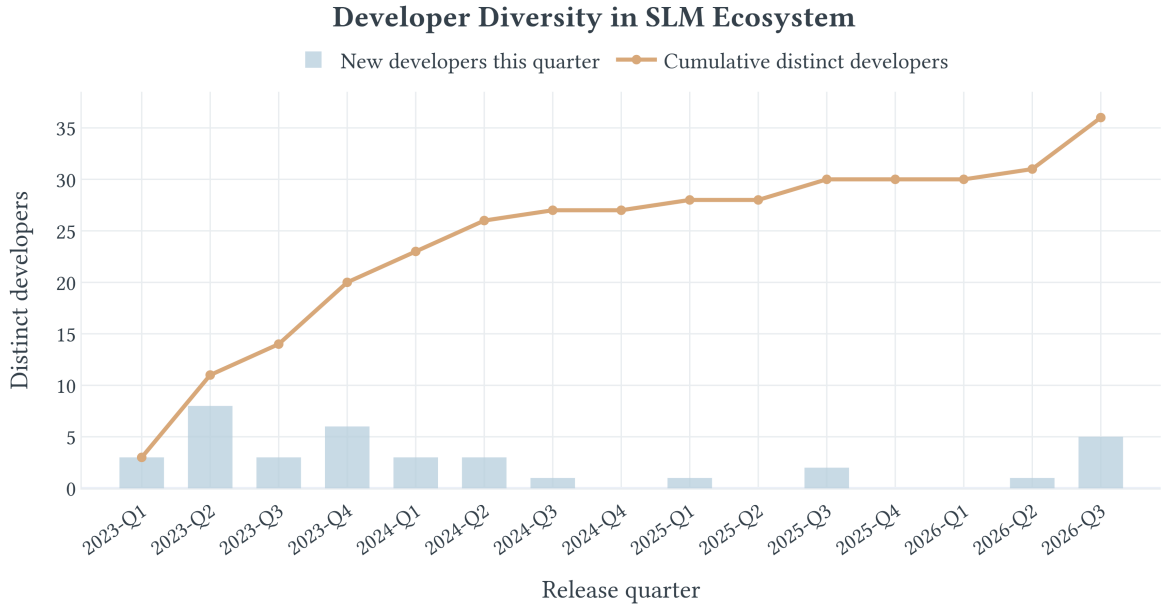


Figure 15. Cumulative distinct developers releasing documented SLM variants (line) and new developers per quarter (bars).

Table 16. Representative model families tracked in the inventory.

Family	Developer	Representative Versions	Access	Typical Role
Qwen2.5 / Qwen3	Alibaba	0.5B–14B dense, MoE variants	Open weights	Strong multilingual, math, and code performance
Phi-3 / Phi-4	Microsoft	3.8B mini variants, 14B reasoning	Open weights	Data-curated reasoning-focused SLMs
Gemma 2 / 3 / 3n	Google DeepMind	1B–12B dense, mobile-first multimodal variants	Open weights	Distilled compact models
Llama 3.2 / 4	Meta	1B–3B edge models, larger MoE frontier models	Open weights	Broad deployment ecosystem from edge to server
SmolLM2 / SmolLM3	Hugging Face	135M–3B	Open weights	Lightweight, fully open research and edge baselines
DeepSeek distills	DeepSeek	1.5B–14B distilled reasoning models	Open weights	Reasoning transfer into self-hosted sizes
Granite 3 / 4	IBM	2–8B dense; 4.0 hybrid Mamba-2/MoE (350M–32B, 1–9B active)	Open weights	Enterprise-friendly, tool-use and RAG focus
Falcon-H1	TII	0.5–34B parallel attention/Mamba-2 hybrid	Open weights	Long-context edge-to-server
LFM2 / LFM2.5	Liquid AI	230M–8.3B hybrid conv/attention, incl. 8B-A1B MoE	Open weights	CPU-oriented on-device models
MiniCPM	OpenBMB	Compact multimodal variants	Open weights	Edge multimodal use cases
Ling / Nemotron 3.5 / Laguna	inclusionAI / NVIDIA / Poolside	Small-active MoE, 3–13B active	Open weights	SLM-like per-token cost at LLM-like residency
Frontier API families	OpenAI / Anthropic / Google / xAI / others	API frontier models (see released inventory)	API only	High-end proprietary baselines and teacher models

parameter MoEs), 223 comparison records (104 of disclosed size that fail C1 and 119 of undisclosed size), and 7 excluded candidates, 439 records in total, for releases through 4 September 2026. Only the inventory is reproduced here (Table 17), organized by release year and family, with total and, where different, active parameters given in the size column; the comparison records are in the released file.

Each record carries the fields `meets_c1_active_le_20b`, `meets_c2_total_le_20b`, `slm_class`, `inclusion_decision`, `developer_source_found`, and `source_check`, so that a reader can recompute the inventory under a different threshold, restrict it to models satisfying both conditions, or audit each record's source.

Chronology. Inventory: 2026, 2025, 2024, 2023 and earlier.

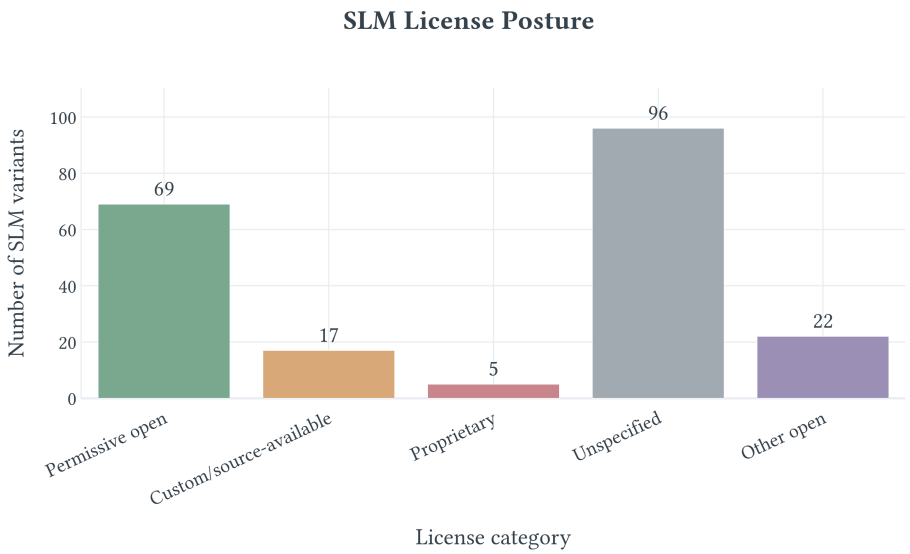


Figure 16. License posture of documented SLM variants. Permissive open licenses (Apache-2.0, MIT) account for the plurality; "unspecified" entries are typically research checkpoints without an attached license file.

Table 17. Generated chronological small language model inventory from `merged_llm_database.json`, grouped by release year and family. Entries whose parameter column reads "*N*B (*M*B act.)" with $N > 20$ are small-active-parameter MoEs (C1 only), not SLMs, and are excluded from every storage and on-device claim.

Model	Access	Params	Context	Modalities	License	Release	Notes
2026							
Qwen3.5 — Alibaba (Qwen3.5 (Mar 2026)) [family page]							
Qwen3.5-0.8B	OW	0.8B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	ultra-lightweight multimodal; on-device
Qwen3.5-2B	OW	2B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	on-device multimodal chat; edge AI
Qwen3.5-4B	OW	4B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	mobile multimodal assistant; agentic coding
Qwen3.5-9B	OW	9B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	strong multimodal SLM; coding
Qwen3.5-35B-A3B	OW	35B (3B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	efficient MoE for edge/server; multimodal reasoning
Qwen3.5-122B-A10B	OW	122B (10B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	strong MoE reasoning; locally runnable flagship
Qwen3.5-397B-A17B	Both	397B (17B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	flagship open-weight multimodal; 201 languages
AFM — Apple (AFM) [family page]							
Apple AFM 3 Core Advanced	API	20B (1-4B act.)	64K	Text+Vision	Proprietary	2026-06-08	on-device assistant; multimodal understanding
Command — Cohere (Command) [family page]							
North Mini Code 1.0	OW	30B (3B act.)	250K	Text	CC-BY-NC (Cohere Labs)	2026-06-17	coding; agentic coding
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek V4 Flash Vision Exp	OW	284B (13B act.)	1M	Text+Vision	See source	2026-08-21	vision; long context
DeepSeek V4 Flash 0731	OW	284B (13B act.)	1280K	Text	See source	2026-07-31	general; long context
Dots — Dots Studio (Dots3-Note Preview (Aug 2026)) [family page]							
Dots3-Note Preview	OW	280B (16B act.)	500K	Text	Apache 2.0	2026-08-14	note taking; general
Gemma 4 — Google DeepMind							
Gemma 4 26B (MoE)	OW	26B (3.8B act.)	128K	Text+Vision	Apache 2.0	2026-04-02	reasoning; multimodal
Gemma 4 E2B	OW	~5B	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	on-device; multimodal
Gemma 4 E4B	OW	~8B	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Gemma 4 12B	OW	12B	256K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Granite 4 — IBM							
Granite 4.2 3B	OW	3B	128K	Text	Apache 2.0	2026-08-25	reasoning; on-device
Granite 4.2 8B	OW	8B	128K	Text	Apache 2.0	2026-08-25	reasoning; agentic
Granite 4.1 3B	OW	3B	128K	Text	Apache 2.0	2026-04-29	enterprise; on-device
Granite 4.1 8B	OW	8B	128K	Text	Apache 2.0	2026-04-29	enterprise; agentic

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
LFM2 — Liquid AI							
LFM2.5-2.6B	Both	2.69B	128K	Text	LFM1.0	2026-08-04	agentic workflows; data extraction
LFM2.5-8B-A1B	OW	8.3B (1.5B act.)	128K	Text	LFM1.0	2026-05-28	on-device; tool calling
LFM2.5-350M	OW	350M	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-1.2B	OW	1.2B	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-Audio-1.5B	OW	1.5B	–	Text+Audio	LFM1.0	2026	on-device speech; transcription
LFM2.5-VL-3B	OW	3B	–	Text+Vision	LFM1.0	2026	on-device VLM; document extraction
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral Small 4 (119B)	Both	119B (6.5B act.)	256K	Text+Vision	Apache 2.0	2026-03-16	hybrid reasoning; coding
NVIDIA Nemotron — NVIDIA (Nemotron 3 Super (Mar 2026)) [family page]							
NemotronLabs VoiceChat 11B	OW	11B	128K	Text+Audio	NVIDIA Custom	2026-08-03	voice interaction; on-device speech
Nemotron 3 Super	OW	120B (12B act.)	1M	Text	NVIDIA Open Model License	2026-03	open hybrid Mamba-Transformer MoE; agentic reasoning
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron 3.5 Lightning 30B-A3B	OW	30B (3B act.)	256K	Text	NVIDIA Open Model License	2026-08-11	agentic execution layer; high-volume tool calling
Nemotron 3.5 Content Safety	OW	4B	128K	Text+Vision	NVIDIA Open Model License	2026-06-04	guardrails; content moderation
Nex-N2 — Nex AGI (Nex-N2-Mini (Jun 2026)) [family page]							
Nex-N2-Pro	OW	397B (17B act.)	256K	Text+Vision	See source	2026-06-08	agentic; coding
Laguna — Poolside (Laguna S 2.1 (Jul 2026)) [family page]							
Laguna S 2.1	OW	118B (8B act.)	1M	Text	See source	2026-07-21	coding agent
Laguna XS 2.1	OW	33B (3B act.)	256K	Text	See source	2026-07-02	coding agent
Hunyuan — Tencent (Hunyuan) [family page]							
Hy-MT2-1.8B	OW	1.8B	8K	Text	Tencent Hunyuan License	2026-08-20	translation; on-device
Hy-MT2-30B-A3B	OW	30B (3B act.)	8K	Text	Tencent Hunyuan License	2026-08-20	translation
Hy-MT2-7B	OW	7B	8K	Text	Tencent Hunyuan License	2026-08-19	translation; Lang: 33 language pairs + 5 dialect/minority pairs
Inkling — Thinking Machines Lab (Inkling Small (Jul 2026)) [family page]							
Inkling Small	OW	276B (12B act.)	1M	Text+Vision	See source	2026-07-30	general; multimodal
MiMo — Xiaomi (MiMo-V2-Pro (Mar 2026)) [family page]							
MiMo-V2-Flash	OW	309B (15B act.)	256K	Text	Undisclosed	2026	high-throughput coding agents; production serving
Ling — inclusionAI (Ling-1T (2026)) [family page]							
Ling 3.0 Flash Fin	OW	124B (5.1B act.)	256K	Text	MIT	2026-08-27	finance
Ling 3.0 Flash	OW	124B (5.1B act.)	256K	Text	See source	2026-07-23	general; agentic

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
2025							
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Omni-7B	OW	7B	–	Text+Audio+Video+Image	See source	2025-03-26	omni-modal; text and audio output
Qwen2.5-VL-7B	OW	7B	–	Text+Video+Image	See source	2025-01-22	active
Qwen3 — Alibaba (Qwen3 (Apr 2025)) [family page]							
Qwen3-4B-Instruct-2507	OW	4B	128K	Text	Apache 2.0	2025-07	improved instruction following; non-thinking mode optimised
Qwen3-0.6B	OW	0.6B	32K	Text	Apache 2.0	2025-04-29	edge inference; IoT
Qwen3-1.7B	OW	1.7B	32K	Text	Apache 2.0	2025-04-29	mobile inference; Q&A
Qwen3-30B-A3B	OW	30B (3B act.)	128K	Text	Apache 2.0	2025-04-29	MoE efficiency; broad capability in tiny footprint
Qwen3-4B-Instruct	OW	4B	128K	Text	Apache 2.0	2025-04-29	on-device chat; coding
Qwen3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04-29	single-GPU inference; reasoning
Qwen3-14B-Instruct	OW	14B	128K	Text	Apache 2.0	2025-04-29	high-quality on-premise chat; coding
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device, 2nd gen)	API	~3B	64K	Text+Vision	Proprietary	2025-06-09	on-device assistant; writing tools
DeepSeek R1-Distill — DeepSeek (R1-Distill (Jan 2025)) [family page]							
DeepSeek-R1-Distill-Qwen-1.5B	OW	1.5B	128K	Text	MIT	2025-01-20	reasoning at ultra-small scale; math
DeepSeek-R1-Distill-Qwen-7B	OW	7B	128K	Text	MIT	2025-01-20	chain-of-thought reasoning; coding
DeepSeek-R1-Distill-Llama-8B	OW	8B	128K	Text	MIT	2025-01-20	reasoning on Llama 3 base; on-premise
DeepSeek-R1-Distill-Qwen-14B	OW	14B	128K	Text	MIT	2025-01-20	strong reasoning 14B; top distill for coding
Gemma 3 — Google DeepMind (Gemma 3 (Mar 2025)) [family page]							
Gemma 3 270M-IT	OW	270M	32K	Text	Google Gemma License	2025-03	ultra-lightweight prototyping; experiments
Gemma 3 1B-IT	OW	1B	32K	Text	Google Gemma License	2025-03	on-device mobile inference; lightweight Q&A
Gemma 3 4B-IT	OW	4B	128K	Text+Vision	Google Gemma License	2025-03	multimodal on-device; image understanding
Gemma 3 12B-IT	OW	12B	128K	Text+Vision	Google Gemma License	2025-03	high-quality assistant; 140+ languages
Gemma 3n — Google DeepMind (Gemma-3n (2025)) [family page]							
Gemma-3n-E2B-IT	OW	~5B	32K	Text+Vision+Audio+Video	Google Gemma License	2025	mobile-first multimodal; on-device speech/vision/text
Gemma-3n-E4B-IT	OW	~8B	32K	Text+Vision+Audio+Video	Google Gemma License	2025	higher-accuracy mobile multimodal; edge devices

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM3-3B	OW	3B	64K	Text	Apache 2.0	2025	dual-mode reasoning; tool calling
Granite — IBM (Granite) [family page]							
Granite 3.2-2B	OW	2B	125K	Text	See source	2025-02-26	Thinking mode; 128K context
Granite 3.2-8B	OW	8B	125K	Text	See source	2025-02-26	Thinking mode (chain-of-thought); 128K context
IBM Granite — IBM (Granite 3.3 (Apr 2025)) [family page]							
Granite 4.0 Nano 350M	OW	350M	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 Nano 1B	OW	1B	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 H-Tiny	OW	7B (1B act.)	128K	Text	Apache 2.0	2025-10-02	low-latency; agentic
Granite 4.0 H-Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; tool use
Granite 4.0 Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; RAG
Granite 4.0 H-Small	OW	32B (9B act.)	128K	Text	Apache 2.0	2025-10-02	enterprise; agentic
Granite 3.3-2B-Instruct	OW	2B	128K	Text	Apache 2.0	2025-04	enterprise NLP; governed AI
Granite 3.3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04	enterprise reasoning; Fill-in-Middle code
LFM2 — Liquid AI							
LFM2-8B-A1.5B (MoE)	OW	8.3B (1.5B act.)	32K	Text	LFM Open License v1.0	2025-08	on-device; agentic
LFM2-2.6B	OW	2.6B	32K	Text	LFM Open License v1.0	2025-08	on-device
LFM2-350M	OW	350M	32K	Text	LFM Open License v1.0	2025-07-10	on-device; low latency
LFM2-700M	OW	700M	32K	Text	LFM Open License v1.0	2025-07-10	on-device
LFM2-1.2B	OW	1.2B	32K	Text	LFM Open License v1.0	2025-07-10	on-device; agentic
Llama 4 — Meta AI (Llama 4 Maverick (Apr 2025)) [family page]							
Llama 4 Maverick	Both	400B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	image understanding; coding
Llama 4 Scout	Both	109B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	extreme long-context; single H100 runnable
BitNet — Microsoft (BitNet) [family page]							
BitNet b1.58 2B4T	OW	2B	–	Text	MIT	2025-04-16	1-bit LLM (ternary weights: -1, 0, 1); Extreme efficiency
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4-mini-flash-reasoning	OW	3.8B	128K	Text	MIT	2025-04	fast reasoning; edge agents
Phi-4-mini-reasoning	OW	3.8B	128K	Text	MIT	2025-04	chain-of-thought reasoning; math
Phi-4-reasoning	OW	14B	128K	Text	MIT	2025-04	complex reasoning; STEM
Phi-4-reasoning-plus	OW	14B	128K	Text	MIT	2025-04	RL-trained reasoning; math olympiad-level
Phi-4-mini-instruct	OW	3.8B	128K	Text	MIT	2025-02	on-device chat; function calling

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral 3 (14B)	OW	14B	–	Text+Vision	See source	2025-12-02	edge deployment; multimodal
Ministral-3B-Instruct-2512	OW	3.8B	256K	Text+Vision	Apache 2.0	2025-12	tiny multimodal; function calling
Voxtral — Mistral AI (Voxtral) [family page]							
Voxtral Mini 3B	OW	4.7B	–	Text+Audio	Apache 2.0	2025-07-15	speech understanding; transcription
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron 3 Nano-30B	OW	30B	–	Text	See source	2025-12-15	hybrid Mamba-Transformer MoE; 1M context
NVIDIA Nemotron Nano 2	OW	9B	128K	Text	See source	2025-08-18	on-device; efficient
GPT-OSS — OpenAI (gpt-oss-120b (2025)) [family page]							
gpt-oss-20b	OW	21B (3.6B act.)	128K	Text	Apache 2.0	2025	edge reasoning; 16GB device inference
gpt-oss-120b	OW	117B (5.1B act.)	128K	Text	Apache 2.0	2025	open-weight reasoning; near o4-mini parity
MiniCPM — OpenBMB (MiniCPM-V 4.5 (2025)) [family page]							
MiniCPM-V 4.5	OW	~8B	32K	Text+Vision	Apache 2.0	2025	mobile vision-language; on-device VQA
MiniCPM-o 3.0	OW	~9B	32K	Text+Vision+Audio	Apache 2.0	2025	end-to-end multimodal on-device; live streaming
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM3-8B	OW	8B	32K	Text	See source	2025-01-09	Reasoning via deep thinking; Knowledge-rich training pipeline
Falcon-H1 — Technology Innovation Institute							
Falcon-H1-0.5B	OW	0.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-1.5B	OW	1.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-3B	OW	3B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
Falcon-H1-7B	OW	7B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
2024							
Yi — 01.AI (Yi) [family page]							
Yi-1.5-6B	OW	6B	–	Text	See source	2024-05-13	legacy
Yi-1.5-9B	OW	9B	–	Text	See source	2024-05-13	legacy
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Coder-1.5B	OW	1.5B	–	Text	See source	2024-11-12	compact coding model
Qwen2.5-14B	OW	14B	–	Text	See source	2024-09-19	active
Qwen2-VL-7B	OW	7B	–	Text+Video+Image	See source	2024-08-28	vision-language; video understanding
Qwen2-Audio-7B	OW	7B	–	Text+Audio	See source	2024-08-09	audio understanding; speech recognition
Qwen2-0.5B	OW	0.5B	–	Text	See source	2024-06-07	active
Qwen2-1.5B	OW	1.5B	–	Text	See source	2024-06-07	active

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Qwen2-7B	OW	7B	–	Text	See source	2024-06-07	active
Qwen2-57B-A14B	OW	57B	–	Text	See source	2024-06-07	active
Qwen1.5-MoE-A2.7B	OW	14.3B	–	Text	See source	2024-03-28	MoE architecture; efficient
Qwen1.5-0.5B	OW	0.5B	–	Text	See source	2024-02-05	legacy
Qwen1.5-7B	OW	7B	–	Text	See source	2024-02-05	legacy
Qwen1.5-14B	OW	14B	–	Text	See source	2024-02-05	legacy
Qwen2.5 — Alibaba (Qwen2.5 (Sep 2024)) [family page]							
Qwen2.5-0.5B-Instruct	OW	0.5B	32K	Text	Apache 2.0	2024-09-19	extreme edge/IoT; classification
Qwen2.5-1.5B-Instruct	OW	1.5B	32K	Text	Apache 2.0	2024-09-19	edge NLP; lightweight assistants
Qwen2.5-3B-Instruct	OW	3B	32K	Text	Apache 2.0	2024-09-19	mobile chat; summarization
Qwen2.5-7B-Instruct	OW	7B	128K	Text	Apache 2.0	2024-09-19	general assistant; coding
Qwen2.5-Coder — Alibaba (Qwen2.5-Coder (Nov 2024)) [family page]							
Qwen2.5-Coder-7B-Instruct	OW	7B	128K	Text+Code	Apache 2.0	2024-11	code generation; debugging
Qwen2.5-Coder-14B-Instruct	OW	14B	128K	Text+Code	Apache 2.0	2024-11	advanced code assistant; agentic code tasks
OLMo — Allen Institute for AI (Ai2) (OLMo) [family page]							
OLMo 2-7B	OW	7B	–	Text	See source	2024-11-26	fully open; training code included
OLMo 2-13B	OW	13B	–	Text	See source	2024-11-26	active
OLMo-1B	OW	1B	2K	Text	See source	2024-02-01	Fully open (data, code, weights, logs); Dolma training data
OLMo-7B	OW	7B	–	Text	See source	2024-02-01	fully open; training data/code/logs released
Tülu — Allen Institute for AI (Ai2) (Tülu) [family page]							
Tülu 3-8B	OW	8B	–	Text	See source	2024-11-22	instruction following; DPO
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device)	API	~3B	4K	Text	Proprietary	2024-06-10	On-device Apple Intelligence; Privacy-preserving
OpenELM — Apple (OpenELM) [family page]							
OpenELM	OW	3B	–	Text	See source	2024-04-24	layer-wise scaling; on-device
Aya — Cohere (Aya) [family page]							
Aya Expanse-8B	OW	8B	8K	Text	See source	2024-10-22	Improved multilingual performance; Data arbitrage and synthetic data strategies
Aya 23-8B	OW	8B	8K	Text	See source	2024-05-23	23 language support; Multilingual instruction following
Cohere Command — Cohere (Command A (Mar 2025)) [family page]							
Command R7B	Both	7B	128K	Text	Apache 2.0	2024-12	RAG; multilingual enterprise tasks

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
TinyLlama — Community (TinyLlama-1.1B (2024)) [family page]							
TinyLlama-1.1B-Chat	OW	1.1B	2K	Text	Apache 2.0	2024	ultra-light on-device chat; prototyping
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-VL-7B	OW	7B	4K	Text+Vision	See source	2024-03-13	Vision-language model; Hybrid vision encoder
DeepSeek-Math-7B	OW	7B	4K	Text	See source	2024-02-05	Math-specialized; Group Relative Policy Optimization (GRPO)
Gemma — Google DeepMind (Gemma) [family page]							
Gemma 2-2B	OW	2B	8K	Text	See source	2024-07-31	Knowledge distillation from larger model; Sliding window attention
Gemma 2-9B	OW	9B	–	Text	See source	2024-06-27	improved efficiency; knowledge distillation
PaliGemma	OW	3B	8K	Text+Vision	See source	2024-05-14	Vision-language model; SigLIP vision encoder + Gemma 2B
RecurrentGemma-2B	OW	2B	8K	Text	See source	2024-04-09	Griffin architecture with linear recurrence; Efficient inference
CodeGemma-7B	OW	7B	–	Text	See source	2024-04-09	code generation; code completion
Gemma-2B	OW	2B	–	Text	See source	2024-02-21	legacy
Gemma-7B	OW	7B	–	Text	See source	2024-02-21	built from Gemini technology; efficient
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM2-135M	OW	135M	2K	Text	Apache 2.0	2024-10-31	ultra-micro inference; browser/WASM
SmolLM2-360M-Instruct	OW	360M	8K	Text	Apache 2.0	2024-10-31	on-device/browser inference
SmolLM2-1.7B-Instruct	OW	1.7B	8K	Text	Apache 2.0	2024-10-31	efficient on-device chat; coding assist
SmolLM-135M	OW	135M	2K	Text	See source	2024-07-17	Ultra-small LM; On-device capable
SmolLM	OW	1.7B	–	Text	See source	2024-07-16	tiny but capable; on-device
LLaMA — Meta AI (LLaMA) [family page]							
Llama-3-8B	OW	8B	–	Text	See source	2024-04-18	improved tokenizer; grouped query attention
Llama 3.1 — Meta AI (Llama 3.1 (Jul 2024)) [family page]							
Llama 3.1-8B-Instruct	OW	8B	128K	Text	Llama 3.1 Community License	2024-07-23	most fine-tuned SLM base globally; RAG
Llama 3.2 — Meta AI (Llama 3.2 (Sep 2024)) [family page]							
Llama 3.2-1B-Instruct	OW	1B	128K	Text	Llama 3.2 Community License	2024-09-25	ultra-small on-device; Android/iOS apps
Llama 3.2-3B-Instruct	OW	3B	128K	Text	Llama 3.2 Community License	2024-09-25	mobile AI; default on-device Qualcomm
Llama 3.2-11B-Vision-Instruct	OW	11B	128K	Text+Vision	Llama 3.2 Community License	2024-09-25	vision-language tasks; document understanding

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Phi — Microsoft (Phi) [family page]							
Phi-3.5-MoE	OW	41.9B	–	Text	See source	2024-08-20	MoE architecture; 16 experts
Phi-3 — Microsoft (Phi-3.5 (Aug 2024)) [family page]							
Phi-3.5-mini-instruct	OW	3.8B	128K	Text	MIT	2024-08	multilingual instruction following
Phi-3-mini-128k-instruct	OW	3.8B	128K	Text	MIT	2024-04-23	on-device chat; instruction following
Phi-3-small-128k-instruct	OW	7B	128K	Text	MIT	2024-04-23	balanced reasoning & coding
Phi-3-medium-128k-instruct	OW	14B	128K	Text	MIT	2024-04-23	on-premise deployment; complex reasoning
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4	OW	14B	16K	Text	MIT	2024-12-12	STEM reasoning; math
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral-8B-Instruct-2410	OW	8B	128K	Text	Apache 2.0	2024-10	edge-friendly assistant; function calling
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral NeMo 12B	OW	12B	128K	Text	Apache 2.0	2024-07	enterprise RAG; long-document summarization
Mistral 7B v0.3	OW	7.3B	32K	Text	Apache 2.0	2024-05-22	general chat; RAG
Pixtral — Mistral AI (Pixtral) [family page]							
Pixtral-12B	OW	12B	–	Text+Image	See source	2024-09-17	vision-language; variable image resolution
Nemotron — NVIDIA (Nemotron) [family page]							
Minitron-8B	OW	8B	8K	Text	See source	2024-07-17	Width and depth pruning; Knowledge distillation
Nemotron-4-15B	OW	15B	8K	Text	See source	2024-03-14	High-quality training data; Strong downstream task performance
Reka — Reka AI (Reka) [family page]							
Reka Edge	API	7B	–	Text+Vision	Proprietary	2024-02-12	compact; multimodal
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM2.5-7B	OW	7B	–	Text	See source	2024-07-03	1M context; tool use
InternLM2.5-20B	OW	20B	1M	Text	See source	2024-07-03	1M context with LMDeploy; Strong reasoning
InternLM2-20B	OW	20B	–	Text	See source	2024-01-17	200K context; tool use
Arctic — Snowflake (Arctic) [family page]							
Snowflake Arctic	OW	480B	–	Text	See source	2024-04-24	enterprise-grade; 128 experts
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM 2 12B	OW	12B	4K	Text	See source	2024-02-20	Multilingual (7 languages); Strong code capability
StableLM 2 1.6B	OW	1.6B	4K	Text	Stability AI License	2024-01-19	lightweight creative writing; basic chat

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon 3	OW	10B	–	Text+Audio+Video+Image	See source	2024-12-12	multimodal; runs on laptops
Falcon Mamba-7B	OW	7B	–	Text	See source	2024-08-12	SSM architecture; constant memory usage
Falcon 2-11B	OW	11B	–	Text	See source	2024-05-14	multimodal variant; vision-to-language
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
GLM-4-9B	OW	9B	–	Text	See source	2024-06-06	128K-1M context; tool use
CogVLM2-19B	OW	19B	8K	Text+Video+Image	See source	2024-05-24	Visual understanding; Video understanding
2023 and earlier							
Yi — 01.AI (Yi) [family page]							
Yi-6B	OW	6B	–	Text	See source	2023-11-02	bilingual; strong reasoning
Fuyu — Adept AI (Fuyu) [family page]							
Fuyu-8B	OW	8B	–	Text+Vision	See source	2023-10-17	Simplified multimodal architecture; No separate vision encoder
Qwen — Alibaba (Qwen) [family page]							
Qwen-Audio	OW	7B	–	Text+Audio	See source	2023-11-30	Audio understanding; Multi-task audio processing
Qwen-14B	OW	14B	–	Text	See source	2023-09-25	legacy
Qwen-VL	OW	9.6B	–	Text+Image	See source	2023-08-24	vision-language; image understanding
Qwen-7B	OW	7B	–	Text	See source	2023-08-03	bilingual; tool use
Baichuan — Baichuan Inc. (Baichuan) [family page]							
Baichuan-2-13B	OW	13B	–	Text	See source	2023-09-06	legacy
Baichuan-13B	OW	13B	–	Text	See source	2023-07-11	legacy
Baichuan-7B	OW	7B	–	Text	See source	2023-06-15	bilingual; strong Chinese performance
Cerebras-GPT — Cerebras Systems (Cerebras-GPT) [family page]							
Cerebras-GPT-13B	OW	13B	2K	Text	See source	2023-03-28	Chinchilla compute-optimal training; Trained on Cerebras CS-2 hardware
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-Coder-1.3B	OW	1.3B	–	Text	See source	2023-11-02	compact code model; fill-in-the-middle
DeepSeek-Coder-6.7B	OW	6.7B	–	Text	See source	2023-11-02	code generation; fill-in-the-middle
Pythia — EleutherAI (Pythia) [family page]							
Pythia-12B	OW	12B	2K	Text	See source	2023-04-03	154 intermediate checkpoints; Reproducible training
Gemini — Google DeepMind (Gemini) [family page]							
Gemini 1.0 Nano	API	3.25B	–	Text	Proprietary	2023-12-06	on-device; efficient

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
StarCoder — Hugging Face / BigCode (StarCoder) [family page]							
StarCoder	OW	15.5B	–	Text	See source	2023-05-04	code generation; 80+ programming languages
Jais — Inception / Core42 / Cerebras (Jais) [family page]							
Jais-13B	OW	13B	2K	Text	See source	2023-08-30	Best Arabic LLM at launch; Bilingual Arabic-English
Code Llama — Meta AI (Code Llama) [family page]							
Code Llama-7B	OW	7B	–	Text	See source	2023-08-24	code generation; infilling
LLaMA — Meta AI (LLaMA) [family page]							
Llama-2-7B	OW	7B	–	Text	See source	2023-07-18	open weights; commercial use allowed
Llama-2-13B	OW	13B	–	Text	See source	2023-07-18	legacy
LLaMA-7B	OW	7B	–	Text	Research only	2023-02-24	efficient training; strong few-shot performance
LLaMA-13B	OW	13B	–	Text	Research only	2023-02-24	legacy
Phi — Microsoft (Phi) [family page]							
Phi-2	OW	2.7B	–	Text	See source	2023-12-13	language comprehension; reasoning
Phi-1.5	OW	1.3B	–	Text	See source	2023-09-11	reasoning; language understanding
Phi-1	OW	1.3B	–	Text	See source	2023-06-20	Python coding; textbook-quality data
Mixtral — Mistral AI (Mixtral 8x22B (Apr 2024)) [family page]							
Mixtral-8x7B	OW	46.7B	–	Text	See source	2023-12-11	MoE architecture; 8 experts
MPT — MosaicML (Databricks) (MPT) [family page]							
MPT-7B	OW	7B	–	Text	See source	2023-05-05	ALiBi positional encoding; long context
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM-20B	OW	20B	–	Text	See source	2023-09-20	legacy
InternLM-7B	OW	7B	–	Text	See source	2023-07-06	bilingual; long context
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM-Zephyr-3B	OW	3B	–	Text	See source	2023-11-29	chat assistant; compact
StableLM-Alpha	OW	7B	–	Text	See source	2023-04-19	open-source; conversation
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon-7B	OW	7B	–	Text	See source	2023-06-05	RefinedWeb dataset; high-quality pretraining
RedPajama — Together AI (RedPajama) [family page]							
RedPajama-INCITE-7B	OW	7B	2K	Text	See source	2023-05-05	Fully open training data; Reproducible
SOLAR — Upstage (SOLAR) [family page]							
SOLAR-10.7B	OW	10.7B	4K	Text	See source	2023-12-13	Depth Up-Scaling (DUS); Merged and scaled from Llama 2 architecture
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
ChatGLM3-6B	OW	6B	–	Text	See source	2023-10-27	tool use; code interpreter

continued on next page

Table 17. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
ChatGLM2-6B	OW	6B	–	Text	See source	2023-06-25	legacy
ChatGLM-6B	OW	6B	–	Text	See source	2023-03-14	bilingual; quantized for consumer GPUs

R.5 Monitoring Sources and Maintenance

Table 18 lists the sources monitored to keep the inventory current between versions.

Table 18. High-value monitoring sources retained from the inventory files.

Source	Use
Hugging Face model hub and leaderboards	New releases, model cards, and benchmark submissions
Artificial Analysis	Context windows, pricing, and comparative capability snapshots
LLM Stats	Consolidated model metadata and API comparisons
LMSYS / Chatbot Arena	Human preference rankings
BentoML and vendor engineering blogs	Practical deployment write-ups and model updates
Official vendor changelogs	Release confirmation for OpenAI, Anthropic, Google, Meta, Mistral, Alibaba, and DeepSeek

Recommended maintenance queries from the inventory drafts included combinations such as "best small language model [year]", "new open-weight reasoning model", and vendor-specific searches around context window, model card, and release notes. In practice, the highest-signal workflow is to combine vendor documentation with model cards and a benchmark tracker rather than relying on any single static list.

References

1. OpenAI. GPT-4 Technical Report. arXiv:2303.08774. 2023. arXiv: 2303.08774. URL: https://arxiv.org/abs/2303.08774.
2. Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. Anthropic technical report. 2024. URL: https://www.anthropic.com/news/claude-3-family.
3. Gemini Team, Google. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805. 2023. arXiv: 2312.11805. URL: https://arxiv.org/abs/2312.11805.
4. Llama Team, AI @ Meta. The Llama 3 Herd of Models. arXiv:2407.21783. 2024. URL: https://arxiv.org/abs/2407.21783.
5. Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, Gray S, Radford A, Wu J, and Amodei D. Scaling Laws for Neural Language Models. arXiv:2001.08361. 2020. arXiv: 2001.08361. URL: https://arxiv.org/abs/2001.08361.
6. Hoffmann J, Borgeaud S, Mensch A, Buchatskaya E, Cai T, Rutherford E, Las Casas D de, Hendricks LA, Welbl J, Clark A, Hennigan T, Noland E, Millican K, Driessche G van den, Damoc B, Guy A, Osindero S, Simonyan K, Elsen E, Rae JW, Vinyals O, and Sifre L. Training Compute-Optimal Large Language Models. In: *Advances in Neural Information Processing Systems*. Vol. 35. New Orleans, Louisiana, USA: Curran Associates, Inc., 2022. DOI: 10.52202/068431-2176. arXiv: 2203.15556. URL: https://arxiv.org/abs/2203.15556.
7. Patterson D, Gonzalez J, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, and Dean J. Carbon Emissions and Large Neural Network Training. arXiv:2104.10350. 2021. arXiv: 2104.10350. URL: https://arxiv.org/abs/2104.10350.
8. Vries A de. The Growing Energy Footprint of Artificial Intelligence. Joule 2023;7:2191–4.
9. Elsworth C, Huang K, Patterson D, Schneider I, Sedivy R, Goodman S, Townsend B, Ranganathan P, Dean J, Vahdat A, Gomes B, and Manyika J. Measuring the environmental impact of delivering AI at Google Scale. arXiv:2508.15734. 2025. arXiv: 2508.15734. URL: https://arxiv.org/abs/2508.15734.
10. Abdin M, Aneja J, Awadalla H, Awadallah A, Awan AA, Bach N, Bahree A, Bakhtiari A, Bao J, Behl H, Benhaim A, Bilenko M, Bjorck J, Bubeck S, Cai M, Cai Q, Chaudhary V, Chen D, Chen D, Chen W, Chen YC, Chen YL, Cheng H, Chopra P, Dai X, Dixon M, Eldan R, Fragoso V, Gao J, Gao M, et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219. 2024. arXiv: 2404.14219. URL: https://arxiv.org/abs/2404.14219.
11. Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295. 2024. arXiv: 2403.08295. URL: https://arxiv.org/abs/2403.08295.
12. Qwen Team, Alibaba Group. Qwen2.5 Technical Report. arXiv:2412.15115. 2024. arXiv: 2412.15115. URL: https://arxiv.org/abs/2412.15115.

13. Technology Innovation Institute. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448. 2025. URL: https://arxiv.org/abs/2507.22448.

14. Yang A, Li A, Yang B, Zhang B, Hui B, Zheng B, Yu B, Gao C, Huang C, Lv C, Zheng C, Liu D, Zhou F, Huang F, Hu F, Ge H, Wei H, Lin H, Tang J, Yang J, Tu J, Zhang J, Yang J, Yang J, Zhou J, Zhou J, Lin J, Dang K, Bao K, Yang K, et al. Qwen3 Technical Report. arXiv:2505.09388. 2025. arXiv: 2505.09388. URL: https://arxiv.org/abs/2505.09388.

15. Gemma Team, Google DeepMind. Gemma 4 Technical Report. arXiv:2607.02770. Google DeepMind; models released 2 April 2026 (E2B–31B, dense and MoE). 2026. arXiv: 2607.02770. URL: https://arxiv.org/abs/2607.02770.

16. Apple. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv:2507.13575. 2025. arXiv: 2507.13575. URL: https://arxiv.org/abs/2507.13575.

17. Alizadeh K, Mirzadeh I, Belenko D, Khatamifard K, Cho M, Mundo CCD, Rastegari M, and Farajtabar M. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024. arXiv: 2312.11514. URL: https://arxiv.org/abs/2312.11514.

18. Apple Machine Learning Research. Introducing the Third Generation of Apple's Foundation Models. https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models. 2026.

19. Belcak P, Heinrich G, Diao S, Fu Y, Dong X, Muralidharan S, Lin YC, and Molchanov P. Small Language Models are the Future of Agentic AI. arXiv:2506.02153. Version 3, 22 September 2026. 2025. arXiv: 2506.02153. URL: https://arxiv.org/abs/2506.02153.

20. Chen S, Liu Y, Han W, Zhang W, and Liu T. A Survey on LLM-Based Multi-Agent System: Recent Advances and New Frontiers in Application. arXiv:2412.17481. 2024. arXiv: 2412.17481. URL: https://arxiv.org/abs/2412.17481.

21. Sharma R and Mehta M. Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade offs. arXiv:2510.03847. 2025. arXiv: 2510.03847. URL: https://arxiv.org/abs/2510.03847.

22. Lu Z, Li X, Cai D, Yi R, Liu F, Zhang X, Lane ND, and Xu M. Small Language Models: Survey, Measurements, and Insights. arXiv:2409.15790. 2024. arXiv: 2409.15790. URL: https://arxiv.org/abs/2409.15790.

23. Nguyen CV, Shen X, Aponte R, Xia Y, Basu S, Hu Z, Chen J, Parmar M, Kunapuli S, Barrow J, Wu J, Singh A, Wang Y, Gu J, Dernoncourt F, Ahmed NK, Lipka N, Zhang R, Chen X, Yu T, Kim S, Deilamsalehy H, Park N, Rimer M, Zhang Z, Yang H, Rossi RA, and Nguyen TH. A Survey of Small Language Models. arXiv:2410.20011. 2024. arXiv: 2410.20011. URL: https://arxiv.org/abs/2410.20011.

24. Wang F, Zhang Z, Zhang X, Wu Z, Mo T, Lu Q, Wang W, Li R, Xu J, Tang X, He Q, Ma Y, Huang M, and Wang S. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. ACM Transactions on Intelligent Systems and Technology 2025.

25. Gupta A, Thomas B, Asnani H, Madduru PR, Feroze S, Subramanian S, Elango V, and Gungor M. Small Language Models (SLMs) Can Still Pack a Punch: A survey (updated 2026). arXiv:2501.05465. 2025. arXiv: 2501.05465. URL: https://arxiv.org/abs/2501.05465.

26. Tian C, Cai D, Zhao W, and Lane ND. Beyond Scaling: Agents Are Heading to the Edge. arXiv:2605.18535. 2026. arXiv: 2605.18535. URL: https://arxiv.org/abs/2605.18535.

27. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, and Polosukhin I. Attention is All You Need. In: *Advances in Neural Information Processing Systems*. Vol. 30. Long Beach, California, USA: Curran Associates, Inc., 2017:5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

28. Devlin J, Chang MW, Lee K, and Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019:4171–86. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

29. Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, et al. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems 2020;33:1877–901.

30. Gunasekar S, Zhang Y, Aneja J, Mendes CCT, Giorno AD, Gopi S, Javaheripi M, Kauffmann P, Rosa G de, Saarikivi O, Salim A, Shah S, Behl HS, Wang X, Bubeck S, Eldan R, Kalai AT, Lee YT, and Li Y. Textbooks Are All You Need. arXiv:2306.11644. 2023. arXiv: 2306.11644. URL: https://arxiv.org/abs/2306.11644.

31. Gemma Team, Google DeepMind. Gemma 3 Technical Report. arXiv:2503.19786. 2025. URL: https://arxiv.org/abs/2503.19786.

32. Meta AI. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/. Published 25 September 2024. 2024.

33. Hinton G, Vinyals O, and Dean J. Distilling the Knowledge in a Neural Network. arXiv:1503.02531. 2015. arXiv: 1503.02531. URL: https://arxiv.org/abs/1503.02531.

34. Frantar E, Ashkboos S, Hoefler T, and Alistarh D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In: *11th International Conference on Learning Representations (ICLR)*. Kigali, Rwanda: OpenReview.net, 2023. URL: https://openreview.net/forum?id=tcbBPnfwxS.

35. Lin J, Tang J, Tang H, Yang S, Chen WM, Wang WC, Xiao G, Dang X, Gan C, and Han S. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In: *Proceedings of Machine Learning and Systems*. Vol. 6. Santa Clara, California, USA: MLSys, 2024. URL: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.

36. Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, and Chen W. LoRA: Low-Rank Adaptation of Large Language Models. In: *10th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.

37. Dettmers T, Pagnoni A, Holtzman A, and Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs. In: *Advances in Neural Information Processing Systems*. Vol. 36. New Orleans, Louisiana, USA: Curran Associates, Inc., 2023. DOI: 10.52202/075280-0441. arXiv: 2305.14314. URL: https://arxiv.org/abs/2305.14314.

38. Gerganov G. llama.cpp. GitHub repository: https://github.com/ggerganov/llama.cpp. 2023.

39. Meta AI. ExecuTorch: End-to-End Solution for Enabling On-Device AI. PyTorch Blog. 2024. URL: https://pytorch.org/executorch/.

40. MLC Team. MLC LLM: Universal LLM Deployment Engine. GitHub repository: https://github.com/mlc-ai/mlc-llm. 2024.

41. Chen L and Varoquaux G. What is the Role of Small Models in the LLM Era: A Survey. Computational Linguistics 2026.

42. Chen Y, Zhao J, and Han H. A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv:2505.07460. 2025. arXiv: 2505.07460. URL: https://arxiv.org/abs/2505.07460.

43. Chen L, Zaharia M, and Zou J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. 2023. arXiv: 2305.05176. URL: https://arxiv.org/abs/2305.05176.

44. Ding D, Mallick A, Wang C, Sim R, Mukherjee S, Ruhle V, Lakshmanan LVS, and Awadallah AH. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. arXiv:2404.14618. 2024. arXiv: 2404.14618. URL: https://arxiv.org/abs/2404.14618.

45. Aggarwal P, Madaan A, Anand A, Potharaju SP, Mishra S, Zhou P, Gupta A, Rajagopal D, Kappaganthu K, Yang Y, Upadhyay S, Faruqui M, and Mausam. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963. 2023. arXiv: 2310.12963. URL: https://arxiv.org/abs/2310.12963.

46. Ong I, Almahairi A, Wu V, Chiang WL, Wu T, Gonzalez JE, Kadous MW, and Stoica I. RouteLLM: Learning to Route LLMs with Preference Data. In: *International Conference on Learning Representations (ICLR)*. 2025. arXiv: 2406.18665. URL: https://arxiv.org/abs/2406.18665.

47. Hu QJ, Bieker J, Li X, Jiang N, Keigwin B, Ranganath G, Keutzer K, and Upadhyay SK. RouterBench: A Benchmark for Multi-LLM Routing System. arXiv:2403.12031. 2024. arXiv: 2403.12031. URL: https://arxiv.org/abs/2403.12031.

48. Shao C, Hu X, Lin Y, and Xu F. Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. In: *Proceedings of the ACM Web Conference 2025*. 2025. DOI: 10.1145/3696410.3714765. arXiv: 2502.04392. URL: https://arxiv.org/abs/2502.04392.

49. Leviathan Y, Kalman M, and Matias Y. Fast Inference from Transformers via Speculative Decoding. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. Honolulu, Hawaii, USA: PMLR, 2023:19274–86. URL: https://proceedings.mlr.press/v202/leviathan23a.html.

50. Chen C, Borgeaud S, Irving G, Lespiau JB, Sifre L, and Jumper J. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318. 2023. arXiv: 2302.01318. URL: https://arxiv.org/abs/2302.01318.

51. Liu Z, Hoang T, Zhang J, Zhu M, Lan T, Kokane S, Tan J, Yao W, Liu Z, Feng Y, Murthy R, Yang L, Savarese S, Niebles JC, Wang H, Heinecke S, and Xiong C. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. arXiv:2406.18518. 2024. arXiv: 2406.18518. URL: https://arxiv.org/abs/2406.18518.

52. Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, Wang S, Gan W, Liu Z, Yu Y, Wang Z, Wang Y, Ning W, Hou Y, Wang B, Wu C, Wang X, Liu Y, Wang Y, Tang D, Tu D, Shang L, Jiang X, Tang R, Lian D, Liu Q, and Chen E. ToolACE: Winning the Points of LLM Function Calling. arXiv:2409.00920. 2024. arXiv: 2409.00920. URL: https://arxiv.org/abs/2409.00920.

53. Zhang J, Lan T, Zhu M, Liu Z, Hoang T, Kokane S, Yao W, Tan J, Prabhakar A, Chen H, Liu Z, Feng Y, Awalgaonkar T, Murthy R, Hu E, Chen Z, Xu R, Niebles JC, Heinecke S, Wang H, Savarese S, and Xiong C. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*. 2025. DOI: 10.18653/v1/2025.naacl-long.578. arXiv: 2409.03215. URL: https://arxiv.org/abs/2409.03215.

54. Kapoor S, Stroebl B, Siegel ZS, Nadgir N, and Narayanan A. AI Agents That Matter. arXiv:2407.01502. 2024. arXiv: 2407.01502. URL: https://arxiv.org/abs/2407.01502.

55. Kapoor S, Stroebl B, Kirgis P, Nadgir N, Siegel ZS, Wei B, Xue T, Chen Z, Chen F, Utpala S, Ndzomga F, Oruganty D, Luskin S, Liu K, Yu B, Arora A, Hahm D, Trivedi H, Sun H, Lee J, Jin T, Mai Y, Zhou Y, Zhu Y, Bommasani R, Kang D, Song D, Henderson P, Su Y, Liang P, and Narayanan A. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. arXiv:2510.11977. 2025. arXiv: 2510.11977. URL: https://arxiv.org/abs/2510.11977.

56. Erol MH, El B, Suzgun M, Yuksekgonul M, and Zou J. Cost-of-Pass: An Economic Framework for Evaluating Language Models. arXiv:2504.13359. 2025. arXiv: 2504.13359. URL: https://arxiv.org/abs/2504.13359.

57. Srivastava G, Cao S, and Wang X. Towards Reasoning Ability of Small Language Models. arXiv:2502.11569. 2025. arXiv: 2502.11569. URL: https://arxiv.org/abs/2502.11569.

58. Feng S, Fang G, Ma X, and Wang X. Efficient Reasoning Models: A Survey. arXiv:2504.10903. 2025. arXiv: 2504.10903. URL: https://arxiv.org/abs/2504.10903.

59. Allal LB, Lozhkov A, Bakouch E, Blázquez GM, Penedo G, Tunstall L, Marafioti A, Kydlíček H, Lajarín AP, Srivastav V, Lochner J, Fahlgren C, Nguyen XS, Fourrier C, Burtenshaw B, Larcher H, Zhao H, Zakka C, Morlon M, Raffel C, Werra L von, and Wolf T. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737. 2025. arXiv: 2502.02737. URL: https://arxiv.org/abs/2502.02737.

60. Ainslie J, Lee-Thorp J, Jong M de, Zemlyanskiy Y, Lebrón F, and Sanghai S. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245. 2023. arXiv: 2305.13245. URL: https://arxiv.org/abs/2305.13245.

61. Shazeer N, Mirhoseini A, Maziarz K, Davis A, Le Q, Hinton G, and Dean J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In: *5th International Conference on Learning Representations (ICLR)*. Toulon, France: OpenReview.net, 2017. URL: https://openreview.net/forum?id=B1ckMDqlg.

62. Fedus W, Zoph B, and Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. Journal of Machine Learning Research 2022;23:1–39.

63. IBM Granite Team. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models for Enterprise. https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models. 2025.

64. Gu A and Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. 2023. arXiv: 2312.00752. URL: https://arxiv.org/abs/2312.00752.

65. Lieber O, Lenz B, Bata H, Cohen G, Osin J, Dalmedigos I, Safahi E, Meirom S, Belinkov Y, Shalev-Shwartz S, Abend O, Alon R, Asida T, Bergman A, Glozman R, Gokhman M, Manevich A, Ratner N, Rozen N, Shwartz E, Zusman M, and Shoham Y. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887. 2024. arXiv: 2403.19887. URL: https://arxiv.org/abs/2403.19887.

66. NVIDIA. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. arXiv:2508.14444. 2025. arXiv: 2508.14444. URL: https://arxiv.org/abs/2508.14444.

67. Qwen Team, Alibaba Group. Qwen3.5 Small Models for On-Device Applications. https://qwen.ai/blog?id=qwen3.5. Alibaba; 0.8B–9B dense multimodal, Gated DeltaNet hybrid, released 2 March 2026. 2026.

68. Ma S, Wang H, Ma L, Wang L, Wang W, Huang S, Dong L, Wang R, Xue J, and Wei F. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764. 2024. arXiv: 2402.17764. URL: https://arxiv.org/abs/2402.17764.

69. Muralidharan S, Sreenivas ST, Joshi R, Chochowski M, Patwary M, Shoeybi M, Catanzaro B, Kautz J, and Molchanov P. Compact Language Models via Pruning and Knowledge Distillation. Advances in Neural Information Processing Systems (NeurIPS); arXiv:2407.14679. 2024. DOI: 10.52202/079017-1299. URL: https://arxiv.org/abs/2407.14679.

70. Dettmers T, Lewis M, Belkada Y, and Zettlemoyer L. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339. 2022. arXiv: 2208.07339. URL: https://arxiv.org/abs/2208.07339.

71. Xiao G, Lin J, Seznec M, Wu H, Demouth J, and Han S. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438. 2022. arXiv: 2211.10438. URL: https://arxiv.org/abs/2211.10438.

72. Wang J, Zeng Y, Guo J, Ma Y, Liu A, and Liu X. SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment. arXiv:2511.13023. 2025. DOI: 10.1145/3746262.3761973. arXiv: 2511.13023. URL: https://arxiv.org/abs/2511.13023.

73. Frantar E and Alistarh D. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. Honolulu, Hawaii, USA: PMLR, 2023:10323–37. URL: https://proceedings.mlr.press/v202/frantar23a.html.

74. Zhou Z, Kurz S, and Zhao Z. Revisiting Pruning vs Quantization for Small Language Models. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, 2025:12055–70. DOI: 10.18653/v1/2025.findings-emnlp.645. URL: https://aclanthology.org/2025.findings-emnlp.645/.

75. Sreenivas ST, Muralidharan S, Joshi R, Chochowski M, Mahabaleshwarkar AS, Shen G, Zeng J, Chen Z, Suhara Y, Diao S, Yu C, Chen WC, Ross H, Olabiyi O, Aithal A, Kuchaiev O, Korzekwa D, Molchanov P, Patwary M, Shoeybi M, Kautz J, and Catanzaro B. LLM Pruning and Distillation in Practice: The Minitron Approach. arXiv:2408.11796. 2024. arXiv: 2408.11796. URL: https://arxiv.org/abs/2408.11796.

76. Hasan MJ, Rahman F, and Mohammed N. OptimCLM: Optimizing Clinical Language Models for Predicting Patient Outcomes via Knowledge Distillation, Pruning, and Quantization. International Journal of Medical Informatics 2025;195:105764.

77. Blodgett SL, Barocas S, Daumé III H, and Wallach H. Language (Technology) is Power: A Critical Survey of "Bias" in NLP. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, 2020:5454–76. DOI: 10.18653/v1/2020.acl-main.485. URL: https://aclanthology.org/2020.acl-main.485/.

78. Mukherjee S, Mitra A, Jawahar G, Agarwal S, Palangi H, and Awadallah A. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707. 2023. arXiv: 2306.02707. URL: https://arxiv.org/abs/2306.02707.

79. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. 2025. URL: https://arxiv.org/abs/2501.12948.

80. Sun M, Liu Z, Bair A, and Kolter JZ. A Simple and Effective Pruning Approach for Large Language Models. In: *12th International Conference on Learning Representations (ICLR)*. Vienna, Austria: OpenReview.net, 2024. URL: https://openreview.net/forum?id=PxoFut3dWW.

81. Ma X, Fang G, and Wang X. LLM-Pruner: On the Structural Pruning of Large Language Models. arXiv:2305.11627. 2023. arXiv: 2305.11627. URL: https://arxiv.org/abs/2305.11627.

82. Men X, Xu M, Zhang Q, Wang B, Lin H, Lu Y, Han X, and Chen W. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. arXiv:2403.03853. 2024. arXiv: 2403.03853. URL: https://arxiv.org/abs/2403.03853.

83. Xia M, Gao T, Zeng Z, and Chen D. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. arXiv:2310.06694. 2023. arXiv: 2310.06694. URL: https://arxiv.org/abs/2310.06694.

84. Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, et al. Holistic Evaluation of Language Models. Transactions on Machine Learning Research 2023.

85. Hugging Face. Open LLM Leaderboard: About. Leaderboard documentation (version 2). Accessed 3 October 2026. The leaderboard has since been retired. 2024. URL: https://huggingface.co/docs/leaderboards/open_llm_leaderboard/about.

86. Biderman S, Schoelkopf H, Sutawika L, Gao L, Tow J, Abbasi B, Aji AF, Ammanamanchi PS, Black S, Clive J, DiPofi A, Etxaniz J, Fattori B, Forde JZ, Foster C, Hsu J, Jaiswal M, Lee WY, Li H, Lovering C, Muennighoff N, Pavlick E, Phang J, Skowron A, Tan S, Tang X, Wang KA, Winata GI, Yvon F, and Zou A. Lessons from the Trenches on Reproducible Evaluation of Language Models. arXiv:2405.14782. 2024. arXiv: 2405.14782. URL: https://arxiv.org/abs/2405.14782.

87. Microsoft. Phi-4-mini-instruct Model Card. https://huggingface.co/microsoft/Phi-4-mini-instruct. Retrieved 8 October 2026. 2025.

88. Meta AI. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md. Retrieved 8 October 2026. 2024.

89. Aryabumi V, Dang J, Talupuru D, Dash S, Cairuz D, Lin H, Venkitesh B, Smith M, Campos JA, Tan YC, Marchisio K, Bartolo M, Ruder S, Locatelli A, Kreutzer J, Frosst N, Gomez A, Blunsom P, Fadaee M, Üstün A, and Hooker S. Aya 23: Open Weight Releases to Further Multilingual Progress. arXiv:2405.15032. 2024. arXiv: 2405.15032. URL: https://arxiv.org/abs/2405.15032.

90. Qi X, Zeng Y, Xie T, Chen PY, Jia R, Mittal P, and Henderson P. Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To! arXiv:2310.03693. 2023. URL: https://arxiv.org/abs/2310.03693.

91. Google and Apple. Joint Statement from Google and Apple. https://blog.google/company-news/inside-google/company-announcements/joint-statement-google-apple/. Published 12 January 2026. 2026.

92. Xu H, Peng B, Awadalla H, Chen D, Chen YC, Gao M, Kim YJ, Li Y, Ren L, Shen Y, Wang S, Xu W, Gao J, and Chen W. Phi-4-Mini-Reasoning: Exploring the Limits of Small Reasoning Language Models in Math. arXiv:2504.21233. 2025. arXiv: 2504.21233. URL: https://arxiv.org/abs/2504.21233.

93. Laskar MTR, Fu XY, and TN SB. SFT or RL for Tool-Calling Agents? A Controlled Study Across Data, Method, and Scale. arXiv:2609.17848. 2026. arXiv: 2609.17848. URL: https://arxiv.org/abs/2609.17848.

94. Willard BT and Louf R. Efficient Guided Generation for Large Language Models. arXiv:2307.09702. 2023. URL: https://arxiv.org/abs/2307.09702.

95. Dong Y, Ruan CF, Cai Y, Lai R, Xu Z, Zhao Y, and Chen T. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. arXiv:2411.15100. 2024. arXiv: 2411.15100. URL: https://arxiv.org/abs/2411.15100.

96. Patil SG, Mao H, Yan F, Ji CCJ, Suresh V, Stoica I, and Gonzalez JE. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. URL: https://proceedings.mlr.press/v267/patil25a.html.

97. Patil SG, Zhang T, Wang X, and Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs. In: *Advances in Neural Information Processing Systems*. 2024. DOI: 10.52202/079017-4020. arXiv: 2305.15334. URL: https://arxiv.org/abs/2305.15334.

98. Qin Y, Liang S, Ye Y, Zhu K, Yan L, Lu Y, Lin Y, Cong X, Tang X, Qian B, Zhao S, Hong L, Tian R, Xie R, Zhou J, Gerstein M, Li D, Liu Z, and Sun M. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. arXiv: 2307.16789. URL: https://arxiv.org/abs/2307.16789.

99. Li M, Zhao Y, Yu B, Song F, Li H, Yu H, Li Z, Huang F, and Li Y. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2023:3102–16. DOI: 10.18653/v1/2023.emnlp-main.187. URL: https://aclanthology.org/2023.emnlp-main.187/.

100. Chen W and Li Z. Octopus v2: On-device language model for super agent. arXiv:2404.01744. 2024. arXiv: 2404.01744. URL: https://arxiv.org/abs/2404.01744.

101. Erdogan LE, Lee N, Jha S, Kim S, Tabrizi R, Moon S, Hooper C, Anumanchipalli G, Keutzer K, and Gholami A. TinyAgent: Function Calling at the Edge. arXiv:2409.00608. 2024. arXiv: 2409.00608. URL: https://arxiv.org/abs/2409.00608.

102. Lin Q, Wen M, Peng Q, Nie G, Liao J, Wang J, Mo X, Zhou J, Cheng C, Zhao Y, Wang J, and Zhang W. Hammer: Robust Function-Calling for On-Device Language Models via Function Masking. arXiv:2410.04587. 2024. arXiv: 2410.04587. URL: https://arxiv.org/abs/2410.04587.

103. Liu Z, Zhao C, Iandola F, Lai C, Tian Y, Fedorov I, Xiong Y, Chang E, Shi Y, Krishnamoorthi R, Lai L, and Chandra V. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 2024. arXiv: 2402.14905. URL: https://arxiv.org/abs/2402.14905.

104. Prabhakar A, Liu Z, Zhu M, Zhang J, Awalgaonkar T, Wang S, Liu Z, Chen H, Hoang T, Niebles JC, Heinecke S, Yao W, Wang H, Savarese S, and Xiong C. APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay. arXiv:2504.03601. 2025. arXiv: 2504.03601. URL: https://arxiv.org/abs/2504.03601.

105. Kavathekar I, Donakanti R, Kumaraguru P, and Vaidhyanathan K. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. arXiv:2504.19277. 2025. arXiv: 2504.19277. URL: https://arxiv.org/abs/2504.19277.

106. Shen W, Li C, Chen H, Yan M, Quan X, Chen H, Zhang J, and Huang F. Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324. 2024. arXiv: 2401.07324. URL: https://arxiv.org/abs/2401.07324.

107. Geng S, Cooper H, Moskal M, Jenkins S, Berman J, Ranchin N, West R, Horvitz E, and Nori H. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. arXiv:2501.10868. 2025. arXiv: 2501.10868. URL: https://arxiv.org/abs/2501.10868.

108. Park K, Wang J, Berg-Kirkpatrick T, Polikarpova N, and D'Antoni L. Grammar-Aligned Decoding. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. arXiv: 2405.21047. URL: https://arxiv.org/abs/2405.21047.

109. Tam ZR, Wu CK, Tsai YL, Lin CY, Lee Hy, and Chen YN. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. arXiv:2408.02442. 2024. arXiv: 2408.02442. URL: https://arxiv.org/abs/2408.02442.

110. Kurt W. Say What You Mean: A Response to "Let Me Speak Freely". .txt engineering blog, https://blog.dottxt.ai/say-what-you-mean.html. Published 5 November 2024. 2024.

111. Zhou S, Xu FF, Zhu H, Zhou X, Lo R, Sridhar A, Cheng X, Ou T, Bisk Y, Fried D, Alon U, and Neubig G. WebArena: A Realistic Web Environment for Building Autonomous Agents. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. arXiv: 2307.13854. URL: https://arxiv.org/abs/2307.13854.

112. Mialon G, Fourrier C, Swift C, Wolf T, LeCun Y, and Scialom T. GAIA: A Benchmark for General AI Assistants. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. arXiv: 2311.12983. URL: https://arxiv.org/abs/2311.12983.

113. Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, and Narasimhan K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. arXiv: 2310.06770. URL: https://arxiv.org/abs/2310.06770.

114. Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, Gu Y, Ding H, Men K, Yang K, Zhang S, Deng X, Zeng A, Du Z, Zhang C, Shen S, Zhang T, Su Y, Sun H, Huang M, Dong Y, and Tang J. AgentBench: Evaluating LLMs as Agents. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria, 2024. arXiv: 2308.03688. URL: https://arxiv.org/abs/2308.03688.

115. Yao S, Shinn N, Razavi P, and Narasimhan K. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. 2024. arXiv: 2406.12045. URL: https://arxiv.org/abs/2406.12045.

116. Barres V, Dong H, Ray S, Si X, and Narasimhan K. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982. 2025. arXiv: 2506.07982. URL: https://arxiv.org/abs/2506.07982.

117. Cai T, Li Y, Geng Z, Peng H, Lee JD, Chen D, and Dao T. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. arXiv:2401.10774. 2024. arXiv: 2401.10774. URL: https://arxiv.org/abs/2401.10774.

118. Li Y, Wei F, Zhang C, and Zhang H. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. arXiv:2401.15077. 2024. arXiv: 2401.15077. URL: https://arxiv.org/abs/2401.15077.

119. Kim H, Hwang H, Lee J, Park S, Kim D, Lee T, Yoon C, Sohn J, Park J, Reykhart O, Fetherston T, Choi D, Kwak SH, Chen Q, and Kang J. Small Language Models Learn Enhanced Reasoning Skills from Medical Textbooks. npj Digital Medicine 2025;8:240.

120. Guha N, Nyarko J, Ho DE, Ré C, Chilton A, Narayana A, Chohlas-Wood A, Peters A, Waldon B, Rockmore DN, Zambrano D, Talisman D, Hoque E, Surani F, Fagan F, Sarfaty G, Dickinson GM, Porat H, Hegland J, Wu J, Nudell J, Niklaus J, Nay J, Choi JH, Tobia K, Hagan M, Ma M, Livermore M, Rasumov-Rahe N, Holzenberger N, Kolt N, Henderson P, Rehaag S, Goel S, Gao S, Williams S, Gandhi S, Zur T, Iyer V, and Li Z. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023. arXiv: 2308.11462. URL: https://arxiv.org/abs/2308.11462.

121. Chalkidis I, Jana A, Hartung D, Bommarito M, Androutsopoulos I, Katz DM, and Aletras N. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2022. arXiv: 2110.00976. URL: https://arxiv.org/abs/2110.00976.

122. Hendrycks D, Burns C, Chen A, and Ball S. CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2021. arXiv: 2103.06268. URL: https://arxiv.org/abs/2103.06268.

123. Rozière B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, Adi Y, Liu J, Sauvestre R, Remez T, Rapin J, Kozhevnikov A, Evtimov I, Bitton J, Bhatt M, Ferrer CC, Grattafiori A, Xiong W, Défossez A, Copet J, Azhar F, Touvron H, Martin L, Usunier N, Scialom T, and Synnaeve G. Code Llama: Open Foundation Models for Code. arXiv:2308.12950. 2023. arXiv: 2308.12950. URL: https://arxiv.org/abs/2308.12950.

124. Lozhkov A, Li R, Allal LB, Cassano F, Lamy-Poirier J, Tazi N, Tang A, Pykhtar D, Liu J, Wei Y, Liu T, Tian M, Kocetkov D, Zucker A, Belkada Y, Wang Z, Liu Q, Abulkhanov D, Paul I, Li Z, Li WD, Risdal M, Li J, Zhu J, Zhuo TY, Zheltonozhskii E, Dade NOO, Yu W, Krauß L, Jain N, et al. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173. 2024. arXiv: 2402.19173. URL: https://arxiv.org/abs/2402.19173.

125. Guo D, Zhu Q, Yang D, Xie Z, Dong K, Zhang W, Chen G, Bi X, Wu Y, Li YK, Luo F, Xiong Y, and Liang W. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196. 2024. URL: https://arxiv.org/abs/2401.14196.

126. Zhang Y, Li Y, Cui L, Cai D, Liu L, Fu T, Huang X, Zhao E, Zhang Y, Xu C, Chen Y, Wang L, Luu AT, Bi W, Shi F, and Shi S. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. Computational Linguistics 2025.

127. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih Wt, Rocktäschel T, Riedel S, and Kiela D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020:9459–74. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

128. Microsoft Research. Phi-2: The Surprising Power of Small Language Models. Microsoft Research Blog. 2023. URL: https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/.

129. Microsoft. Phi-3.5-mini-instruct Model Card. https://huggingface.co/microsoft/Phi-3.5-mini-instruct. Phi-3.5 is also described in the later versions of the Phi-3 technical report. 2024.

130. Abdin M, Aneja J, Behl H, Bubeck S, Eldan R, Gunasekar S, Harrison M, Hewett RJ, Javaheripi M, Kauffmann P, Lee JR, Lee YT, Li Y, Liu W, Mendes CCT, Nguyen A, Price E, Rosa G de, Saarikivi O, Salim A, Shah S, Wang X, Ward R, Wu Y, Yu D, Zhang C, and Zhang Y. Phi-4 Technical Report. arXiv:2412.08905. 2024. arXiv: 2412.08905. URL: https://arxiv.org/abs/2412.08905.

131. Microsoft, Abouelenin A, Ashfaq A, Atkinson A, Awadalla H, Bach N, Bao J, Benhaim A, Cai M, Chaudhary V, Chen C, Chen D, Chen D, Chen J, Chen W, Chen YC, Chen Yl, Dai Q, Dai X, Fan R, Gao M, Gao M, Garg A, Goswami A, Hao J, Hendy A, Hu Y, Jin X, Khademi M, Kim D, et al. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv:2503.01743. 2025. arXiv: 2503.01743. URL: https://arxiv.org/abs/2503.01743.

132. Abdin M, Agarwal S, Awadallah A, Balachandran V, Behl H, Chen L, Rosa G de, Gunasekar S, Javaheripi M, Joshi N, Kauffmann P, Lara Y, Mendes CCT, Mitra A, Nushi B, Papailiopoulos D, Saarikivi O, Shah S, Shrivastava V, Vineet V, Wu Y, Yousefi S, and Zheng G. Phi-4-Reasoning Technical Report. arXiv:2504.21318. 2025. arXiv: 2504.21318. URL: https://arxiv.org/abs/2504.21318.

133. Gemma Team, Google DeepMind. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. 2024. arXiv: 2408.00118. URL: https://arxiv.org/abs/2408.00118.

134. Yang A, Yang B, Hui B, Zheng B, Yu B, Zhou C, Li C, Li C, Liu D, Huang F, Dong G, Wei H, Lin H, Tang J, Wang J, Yang J, Tu J, Zhang J, Ma J, Yang J, Xu J, Zhou J, Bai J, He J, Lin J, Dang K, Lu K, Chen K, Yang K, Li M, et al. Qwen2 Technical Report. arXiv:2407.10671. 2024. URL: https://arxiv.org/abs/2407.10671.

135. Zhang P, Zeng G, Wang T, and Lu W. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385. 2024. URL: https://arxiv.org/abs/2401.02385.

136. Jiang AQ, Sablayrolles A, Mensch A, Bamford C, Chaplot DS, Casas D de las, Bressand F, Lengyel G, Lample G, Saulnier L, Lavaud LR, Lachaux MA, Stock P, Scao TL, Lavril T, Wang T, Lacroix T, and Sayed WE. Mistral 7B. arXiv:2310.06825. 2023. arXiv: 2310.06825. URL: https://arxiv.org/abs/2310.06825.

137. Singer P, Pfeiffer P, Babakhin Y, Jeblick M, Dhankhar N, Fodor G, and Ambati SS. H2O-Danube-1.8B Technical Report. arXiv:2401.16818. 2024. arXiv: 2401.16818. URL: https://arxiv.org/abs/2401.16818.

138. Team OLMo, Walsh P, Soldaini L, Groeneveld D, Lo K, Arora S, Bhagia A, Gu Y, Huang S, Jordan M, Lambert N, Schwenk D, Tafjord O, Anderson T, Atkinson D, Brahman F, Clark C, Dasigi P, Dziri N, Ettinger A, Guerquin M, Heineman D, Ivison H, Koh PW, Liu J, Malik S, Merrill W, Miranda LJV, Morrison J, Murray T, et al. 2 OLMo 2 Furious. arXiv:2501.00656. 2025. arXiv: 2501.00656. URL: https://arxiv.org/abs/2501.00656.

139. Stability AI. Stable LM 2 1.6B Technical Report. arXiv:2402.17834. 2024. arXiv: 2402.17834. URL: https://arxiv.org/abs/2402.17834.

140. BigScience Workshop. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv:2211.05100. 2022. URL: https://arxiv.org/abs/2211.05100.

141. Romero A, Ballas N, Kahou SE, Chassang A, Gatta C, and Bengio Y. FitNets: Hints for Thin Deep Nets. arXiv:1412.6550. 2015. arXiv: 1412.6550. URL: https://arxiv.org/abs/1412.6550.

142. Mitra A, Corro LD, Mahajan S, Codas A, Simoes C, Agarwal S, Chen X, Razdaibiedina A, Jones E, Aggarwal K, Palangi H, Zheng G, Rosset C, Khanpour H, and Awadallah A. Orca 2: Teaching Small Language Models How to Reason. arXiv:2311.11045. 2023. URL: https://arxiv.org/abs/2311.11045.

143. Nguyen T and Nguyen T. An Evaluation of LLMs Inference on Popular Single-board Computers. arXiv:2511.07425. 2025. arXiv: 2511.07425. URL: https://arxiv.org/abs/2511.07425.

144. Microsoft. ONNX Runtime. GitHub repository: https://github.com/microsoft/onnxruntime. 2019.

145. MLC Team. WebLLM: High-Performance In-Browser LLM Inference Engine. GitHub repository. 2024. URL: https://arxiv.org/abs/2412.15803.

146. Google Developers. LiteRT with QNN Acceleration. Google AI Blog. 2025. URL: https://ai.google.dev/edge/litert.

147. Xu J, Li Z, Chen W, Wang Q, Gao X, Cai Q, and Ling Z. On-Device Language Models: A Comprehensive Review. arXiv:2409.00088. 2024. arXiv: 2409.00088. URL: https://arxiv.org/abs/2409.00088.

148. Bakouch E, Allal LB, Lozhkov A, Tazi N, Tunstall L, Patiño CM, Beeching E, Roucher A, Reedi AJ, Gallouédec Q, Rasul K, Habib N, Fourrier C, Kydlicek H, Penedo G, Larcher H, Morlon M, Srivastav V, Lochner J, Nguyen XS, Raffel C, Werra L von, and Wolf T. SmolLM3: smol, Multilingual, Long-Context Reasoner. Hugging Face blog, https://huggingface.co/blog/smollm3. 2025.

149. Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, Chaplot DS, Casas D de las, Hanna EB, Bressand F, Lengyel G, Bour G, Lample G, Lavaud LR, Saulnier L, Lachaux MA, Stock P, Subramanian S, Yang S, Antoniak S, Scao TL, Gervet T, Lavril T, Wang T, Lacroix T, and Sayed WE. Mixtral of Experts. arXiv:2401.04088. 2024. URL: https://arxiv.org/abs/2401.04088.

150. Dai D, Deng C, Zhao C, Xu RX, Gao H, Chen D, Li J, Zeng W, Yu X, Wu Y, Xie Z, Li YK, Huang P, Luo F, Ruan C, Sui Z, and Liang W. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024. DOI: 10.18653/v1/2024.acl-long.70. URL: https://arxiv.org/abs/2401.06066.

151. Meta AI. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/. 2025.

152. Li Y, Bubeck S, Eldan R, Giorno AD, Gunasekar S, and Lee YT. Textbooks Are All You Need II: phi-1.5 technical report. arXiv:2309.05463. 2023. URL: https://arxiv.org/abs/2309.05463.

153. Eldan R and Li Y. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759. 2023. URL: https://arxiv.org/abs/2305.07759.

154. Beyer L, Steiner A, Pinto AS, Kolesnikov A, Wang X, Salz D, Neumann M, Alabdulmohsin I, Tschannen M, Bugliarello E, Unterthiner T, Keysers D, Koppula S, Liu F, Grycner A, Gritsenko A, Houlsby N, Kumar M, Rong K, Eisenschlos J, Kabra R, Bauer M, Bošnjak M, Chen X, Minderer M, Voigtlaender P, Bica I, Balazevic I, Puigcerver J, Papalampidi P, et al. PaliGemma: A Versatile 3B VLM for Transfer. arXiv:2407.07726. 2024. arXiv: 2407.07726. URL: https://arxiv.org/abs/2407.07726.

155. Bai S, Chen K, Liu X, Wang J, Ge W, Song S, Dang K, Wang P, Wang S, Tang J, Zhong H, Zhu Y, Yang M, Li Z, Wan J, Wang P, Ding W, Fu Z, Xu Y, Ye J, Zhang X, Xie T, Cheng Z, Zhang H, Yang Z, Xu H, and Lin J. Qwen2.5-VL Technical Report. arXiv:2502.13923. 2025. arXiv: 2502.13923. URL: https://arxiv.org/abs/2502.13923.

156. Google. Gemma 3n: On-Device Multimodal Models. Google AI Blog. 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/.

157. Liu AH, Ehrenberg A, Lo A, et al. Voxtral. arXiv:2507.13264. 2025. arXiv: 2507.13264. URL: https://arxiv.org/abs/2507.13264.

158. Üstün A, Aryabumi V, Yong ZX, Ko WY, D'souza D, Onilude G, Bhandari N, Singh S, Ooi HL, Kayid A, Vargus F, Blunsom P, Longpre S, Muennighoff N, Fadaee M, Kreutzer J, and Hooker S. Aya Model: An Instruction Finetuned Open-Access Multilingual Language Model. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024. DOI: 10.18653/v1/2024.acl-long.845. URL: https://arxiv.org/abs/2402.07827.

159. Kim S, Hooper C, Gholami A, Dong Z, Li X, Shen S, Mahoney MW, and Keutzer K. SqueezeLLM: Dense-and-Sparse Quantization. arXiv:2306.07629. 2024. arXiv: 2306.07629. URL: https://arxiv.org/abs/2306.07629.

160. Yang C, Lu W, Zhu Y, Wang Y, Chen Q, Gao C, Yan B, and Chen Y. Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application. ACM Transactions on Intelligent Systems and Technology 2025.

161. Jiao X, Yin Y, Shang L, Jiang X, Chen X, Li L, Wang F, and Liu Q. TinyBERT: Distilling BERT for Natural Language Understanding. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, 2020:4163–74. DOI: 10.18653/v1/2020.findings-emnlp.372. URL: https://aclanthology.org/2020.findings-emnlp.372/.

162. Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi E, Le Q, and Zhou D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*. Vol. 35. New Orleans, Louisiana, USA: Curran Associates, Inc., 2022. DOI: 10.52202/068431-1800. arXiv: 2201.11903. URL: https://arxiv.org/abs/2201.11903.

163. Feng T, Li Y, Chenglin L, Chen H, Yu F, and Zhang Y. Teaching Small Language Models Reasoning through Counterfactual Distillation. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, 2024:5831–42. DOI: 10.18653/v1/2024.emnlp-main.333. URL: https://aclanthology.org/2024.emnlp-main.333/.

164. Liao H, He S, Xu Y, Zhang Y, Liu K, and Zhao J. Neural-Symbolic Collaborative Distillation: Advancing Small Language Models for Complex Reasoning Tasks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2025. DOI: 10.1609/aaai.v39i23.34636. arXiv: 2409.13203. URL: https://arxiv.org/abs/2409.13203.

165. Xu X, Li M, Tao C, Shen T, Cheng R, Li J, Xu C, Tao D, and Zhou T. A Survey on Knowledge Distillation of Large Language Models. arXiv:2402.13116. 2024. arXiv: 2402.13116. URL: https://arxiv.org/abs/2402.13116.

166. Ballout M, Krumnack U, Heidemann G, and Kühnberger KU. Efficient Knowledge Distillation: Empowering Small Language Models with Teacher Model Insights. In: *Natural Language Processing and Information Systems (NLDB 2024)*. 2024. DOI: 10.1007/978-3-031-70239-6_3. arXiv: 2409.12586. URL: https://arxiv.org/abs/2409.12586.

167. Fu Y, Yu Y, Han X, Li R, Long X, Yu H, and Li P. Dynamic Self-Distillation via Previous Mini-batches for Fine-tuning Small Language Models. arXiv:2411.16991. 2024. arXiv: 2411.16991. URL: https://arxiv.org/abs/2411.16991.

168. Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, and Steinhardt J. Measuring Massive Multitask Language Understanding. In: *9th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2021. URL: https://openreview.net/forum?id=d7KBjmI3GmQ.

169. Clark P, Cowhey I, Etzioni O, Khot T, Sabharwal A, Schoenick C, and Tafjord O. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457. 2018. arXiv: 1803.05457. URL: https://arxiv.org/abs/1803.05457.

170. Zellers R, Holtzman A, Bisk Y, Farhadi A, and Choi Y. HellaSwag: Can a Machine Really Finish Your Sentence? In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019:4791–800. DOI: 10.18653/v1/P19-1472. URL: https://aclanthology.org/P19-1472/.

171. Cobbe K, Kosaraju V, Bavarian M, Chen M, Jun H, Kaiser L, Plappert M, Tworek J, Hilton J, Nakano R, Hesse C, and Schulman J. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168. 2021. arXiv: 2110.14168. URL: https://arxiv.org/abs/2110.14168.

172. Hendrycks D, Burns C, Kadavath S, Arora A, Basart S, Tang E, Song D, and Steinhardt J. Measuring Mathematical Problem Solving With the MATH Dataset. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 34. Curran Associates, Inc., 2021. arXiv: 2103.03874. URL: https://arxiv.org/abs/2103.03874.

173. Chen M, Tworek J, Jun H, Yuan Q, Oliveira Pinto HP de, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, Ray A, Puri R, Krueger G, Petrov M, Khlaaf H, Sastry G, Mishkin P, Chan B, Gray S, Ryder N, Pavlov M, Power A, Kaiser L, Bavarian M, Winter C, Tillet P, Such FP, Cummings D, Plappert M, Chantzis F, et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. 2021. arXiv: 2107.03374. URL: https://arxiv.org/abs/2107.03374.

174. Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, Jiang E, Cai C, Terry M, Le Q, and Sutton C. Program Synthesis with Large Language Models. arXiv:2108.07732. 2021. arXiv: 2108.07732. URL: https://arxiv.org/abs/2108.07732.

175. Joshi M, Choi E, Weld DS, and Zettlemoyer L. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, 2017:1601–11. DOI: 10.18653/v1/P17-1147. URL: https://aclanthology.org/P17-1147/.

176. Rajpurkar P, Jia R, and Liang P. Know What You Don't Know: Unanswerable Questions for SQuAD. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Melbourne, Australia: Association for Computational Linguistics, 2018:784–9. DOI: 10.18653/v1/P18-2124. URL: https://aclanthology.org/P18-2124/.

177. Williams A, Nangia N, and Bowman SR. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, 2018:1112–22. DOI: 10.18653/v1/N18-1101. URL: https://aclanthology.org/N18-1101/.

178. Socher R, Perelygin A, Wu J, Chuang J, Manning CD, Ng A, and Potts C. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In: *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Seattle, Washington, USA: Association for Computational Linguistics, 2013:1631–42. DOI: 10.18653/v1/D13-1170. URL: https://aclanthology.org/D13-1170/.

179. Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, Lin Z, Li Z, Li D, Xing EP, Zhang H, Gonzalez JE, and Stoica I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. Vol. 36. New Orleans, Louisiana, USA: Curran Associates, Inc., 2023. DOI: 10.52202/075280-2020. arXiv: 2306.05685. URL: https://arxiv.org/abs/2306.05685.

180. Gao L, Tow J, Abbasi B, et al. A Framework for Few-Shot Language Model Evaluation. Zenodo. 2023. URL: https://github.com/EleutherAI/lm-evaluation-harness.

181. Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez JE, Zhang H, and Stoica I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz, Germany: Association for Computing Machinery, 2023:611–26. DOI: 10.1145/3600006.3613165. URL: https://doi.org/10.1145/3600006.3613165.

182. Apple Machine Learning Research. MLX: An Array Framework for Apple Silicon. Software, GitHub repository. 2023. URL: https://github.com/ml-explore/mlx.

183. NVIDIA Corporation. TensorRT-LLM: Accelerating Large Language Model Inference. NVIDIA Developer documentation. 2024. URL: https://developer.nvidia.com/tensorrt.

184. Apple Inc. Apple Intelligence Foundation Language Models. Tech. rep. Apple Machine Learning Research, 2024. URL: https://arxiv.org/abs/2407.21075.

185. Maginative. Samsung Unveils Gauss, Its New Generative AI System. News report on the Samsung AI Forum 2023 announcement. Accessed 3 October 2026. Samsung has published no technical report, parameter count, or architecture for Gauss. 2023. URL: https://www.maginative.com/article/samsung-unveils-gauss-its-new-generative-ai-system/.

186. Lu Z, Li X, Cai D, Yi R, Liu F, Liu W, Luan J, Zhang X, Lane ND, and Xu M. Demystifying Small Language Models for Edge Deployment. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025:14747–64. DOI: 10.18653/v1/2025.acl-long.718. URL: https://aclanthology.org/2025.acl-long.718/.

187. Mistral AI. Codestral: Hello, World! Mistral AI Blog. 2024. URL: https://mistral.ai/news/codestral.

188. Qualcomm Technologies. On-Device AI: Enabling the Next Generation of Mobile Intelligence. Qualcomm white paper. 2024. URL: https://www.qualcomm.com/research/artificial-intelligence/on-device-ai.

189. Strubell E, Ganesh A, and McCallum A. Energy and Policy Considerations for Deep Learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019:3645–50. DOI: 10.18653/v1/P19-1355. URL: https://aclanthology.org/P19-1355/.

190. Patterson D, Gonzalez J, Hölzle U, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, and Dean J. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. Computer 2022;55:18–28.

191. Ren L, Chen C, Xu H, Kim YJ, Atkinson A, Zhan Z, Sun J, Peng B, Liu L, Wang S, Cheng H, Gao J, Chen W, and Shen Y. Decoder-Hybrid-Decoder Architecture for Efficient Reasoning with Long Generation. In: *Advances in Neural Information Processing Systems*. Introduces the SambaY architecture (Phi-4-mini-flash). 2025. DOI: 10.52202/085713-3591. arXiv: 2507.06607. URL: https://arxiv.org/abs/2507.06607.

192. Yang S, Kautz J, and Hatamizadeh A. Gated Delta Networks: Improving Mamba2 with Delta Rule. In: *International Conference on Learning Representations (ICLR)*. 2025. arXiv: 2412.06464. URL: https://arxiv.org/abs/2412.06464.